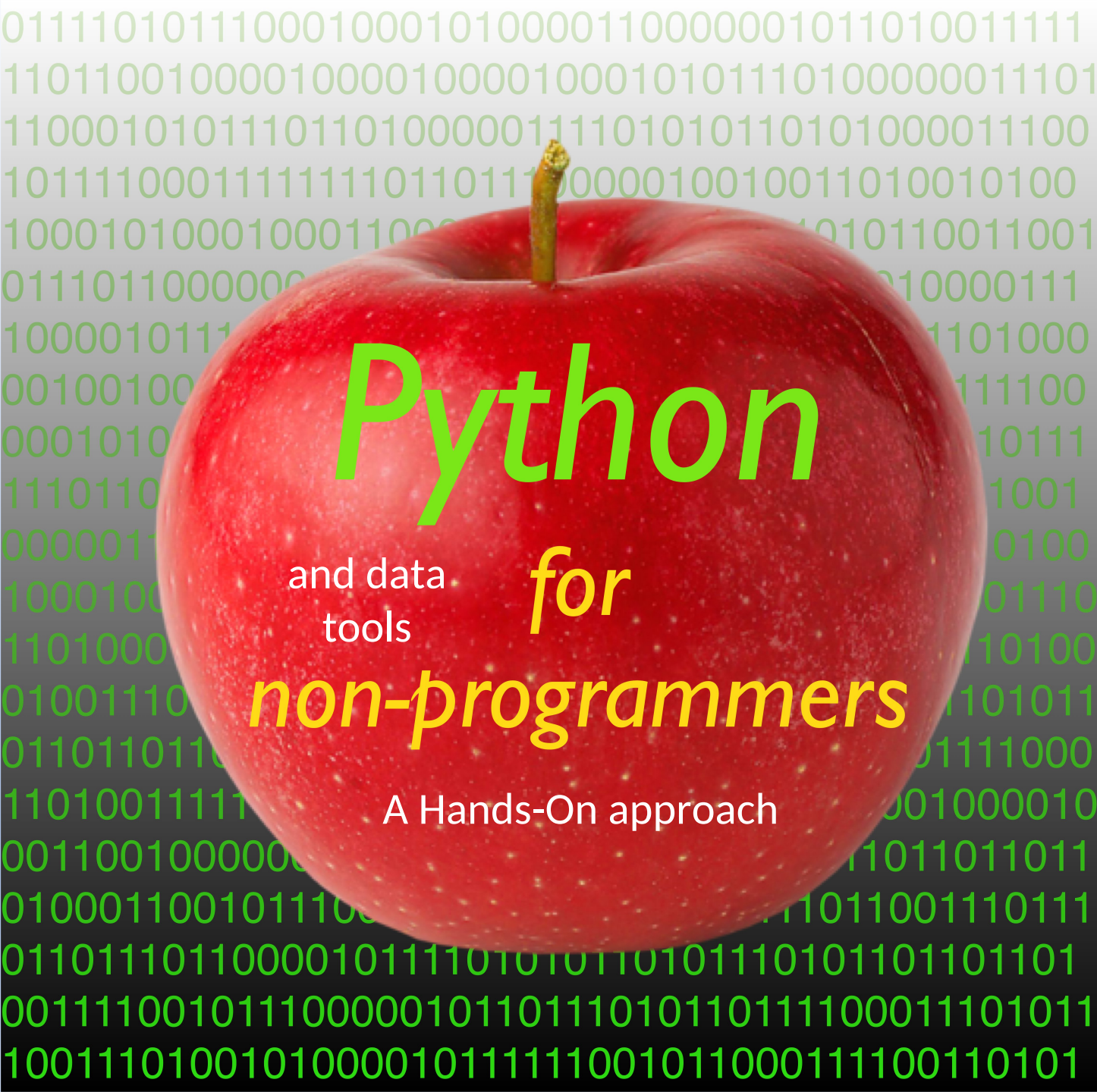




UNIVERSITÄT
BAYREUTH

Delivered by

Dr. Rodrigo Albuquerque

(AG Ruckdäschel)

&

AG Künneth

Part of the project
“digitale Methoden”
(bidt-digikollegs)



I - Basics of Python

- Brief overview of Python
- Installation and graphical interfaces
- Data types and operators
- Objects, modules and classes
- Strings
- Sets, lists, tuples
- Dictionary
- Arrays and matrices
- Loops (for, while)
- If statements
- Functions, Files
- Plotting graphics
- Pandas

II – Python Projects

- Small projects to use the concepts learned

Online material:



<https://rqawork.github.io/teach>

or repository <https://github.com/rqawork/teach> to
download the whole material & infos

- Brief overview of Python
- Installation and graphical environments
- Data types and operators
- Objects, modules and classes
- Strings
- Lists, sets and tuples
- Dictionary
- Arrays and matrices
- Loops (for, while)
- If statements
- Functions, pandas
- Files
- Plotting graphics

Python

Easy, versatile and powerful object-oriented programming language.
It is adopted by e.g. **YouTube**, **Instagram**, **Dropbox**, **Spotify**, **Quora**.

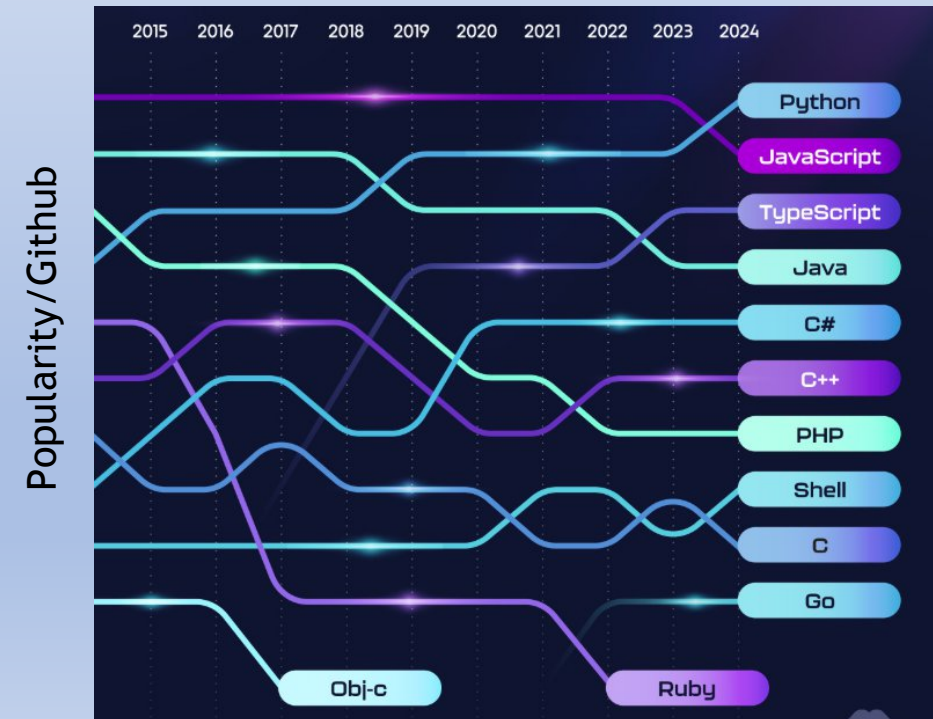
Applications:

AI, Machine Learning, data science, web, database, scientific simulations/calculations.

Visit our ML course at UBT!
“Machine Learning for Beginners”

Advantages of Python:

- It runs in Linux, Mac, Windows, Unix (already installed)
- It is free and open-source
- It has a **GIANT** community helping and further developing
- It has a large (and growing) number of **very useful libraries**
- It has excellent readability and is well-structured (**good for beginners!**)



History and general information

Development: 1991 Guido van Rossum (named after the British comedy 'Monty Python')

Versions: 2 (year 2000), 3 (year 2008)

Object oriented: Objects have **attributes** + **methods**

Case sensitive: 'Tx' is not the same as 'tx'.

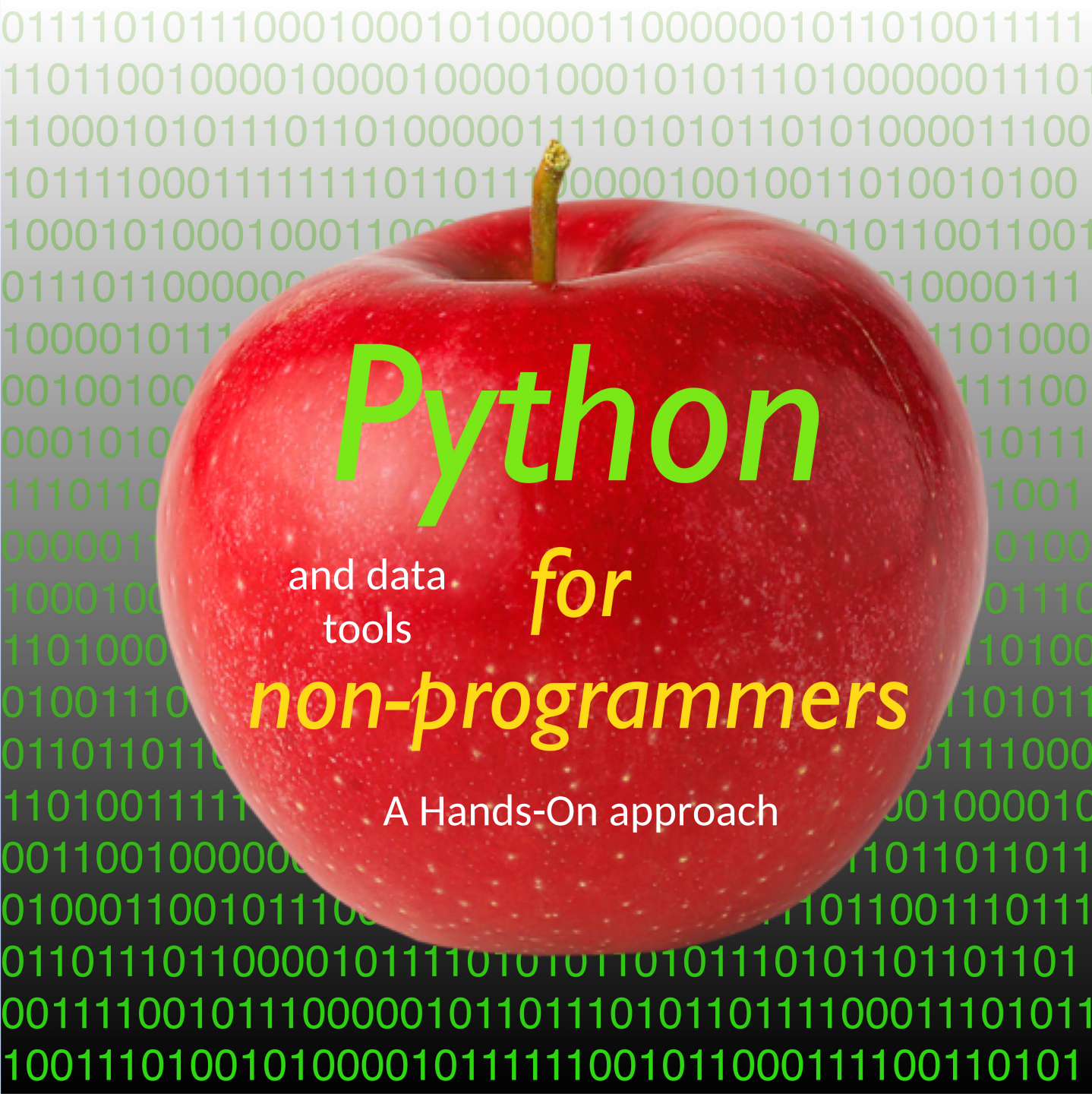
Python homepage: www.python.org

Online tutorials:

<https://www.w3schools.com/python> **and** <https://automatetheboringstuff.com/2e/>

Good book for beginners: [B. Lubanovic, Introducing Python. 2. ed., O'REILLY, 2019.](#)

Good e-Book: <https://pythonnumericalmethods.berkeley.edu/notebooks/Index.html>



UNIVERSITÄT
BAYREUTH

Delivered by

Dr. Rodrigo Albuquerque

(AG Ruckdäschel)

&

AG Künneth

Part of the project
“digitale Methoden”
(bidt-digikollegs)



- Brief overview of Python
- **Installation and graphical interfaces**
- Data types and operators
- Objects, modules and classes
- Strings
- Lists, sets and tuples
- Dictionary
- Arrays and matrices
- Loops (for, while)
- If statements
- Functions, pandas
- Files
- Plotting graphics

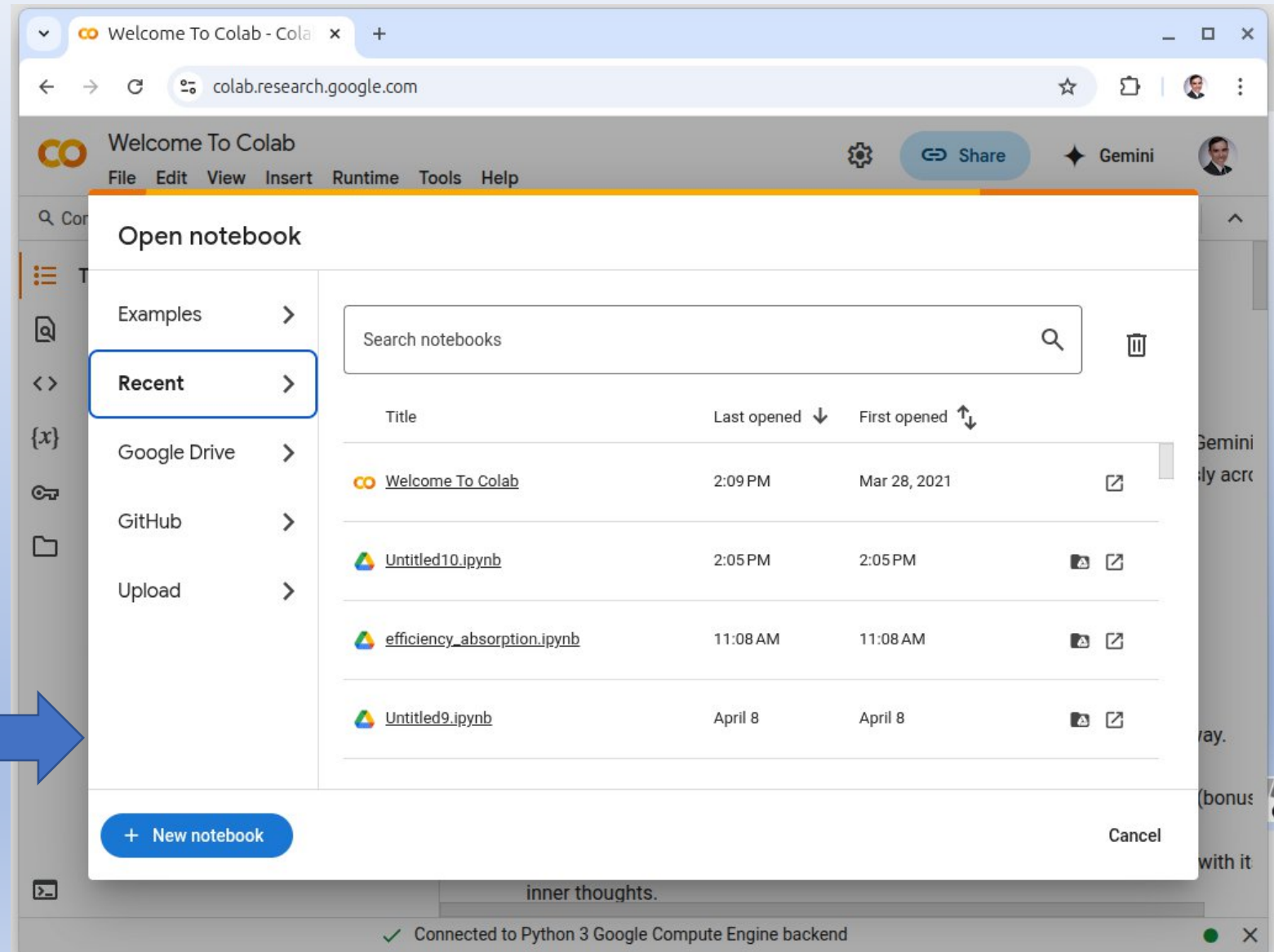
Google colab

- It works out of the box
- You will need a gmail account
- It works only online

At longer term, please, install jupyter notebook locally to have offline access to Python

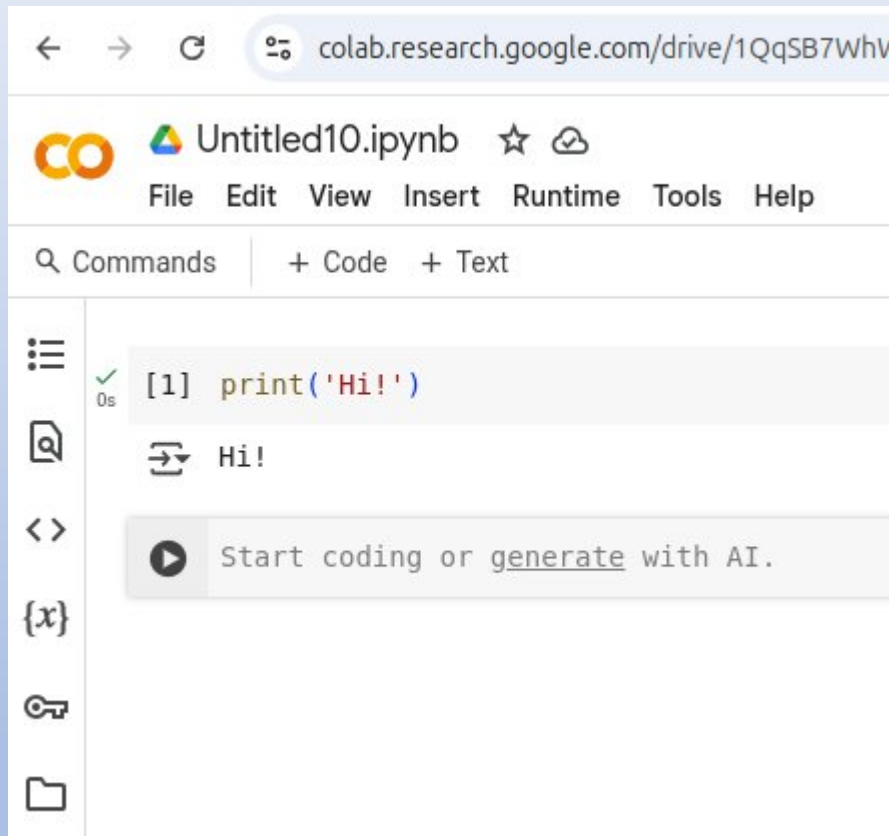
Upload a new jupyter notebook (.ipynb) file

Create a new (empty) notebook



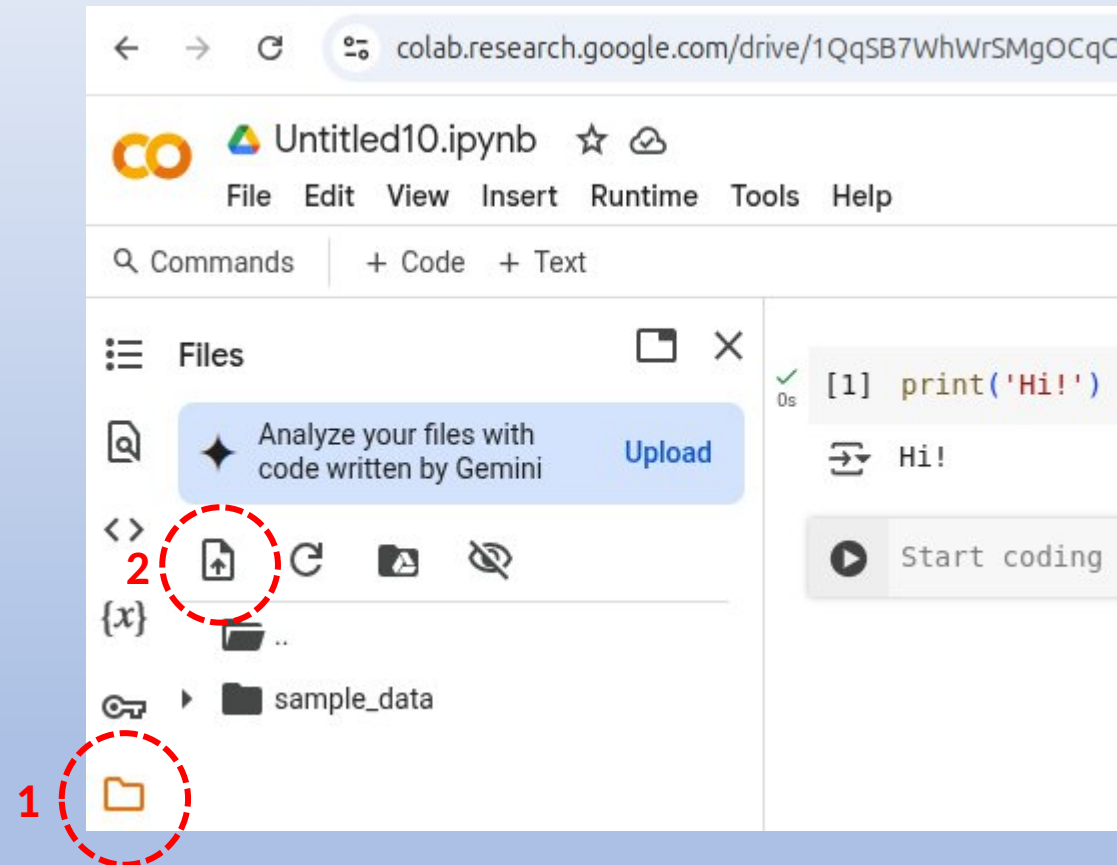
Google colab

New (blank) notebook



Uploading extra files (other than .ipynb)

Click on the files icon below (1), then in the arrow up icon (2)



For the beginners: Please go to *Tools => Settings => AI Assistance* and disable the first two options (otherwise, it becomes very confusing to write anything). Afterwards you can enable it again.

1) Download and execute the installation file, then follow the instructions there.

<https://docs.conda.io/en/latest/miniconda.html>

Click on [Download](#) (upper/right corner), then on [Download Miniconda installer](#) at the bottom of the page.

2) Windows: Launch the *Anaconda Powershell Prompt*
Mac & Linux: Open a *terminal*



```
(base) PS C:\Users\don>
```

This is a black/gray screen containing "(base)"

3) Install Jupyter notebook and some needed libraries:

conda install jupyter numpy matplotlib pandas

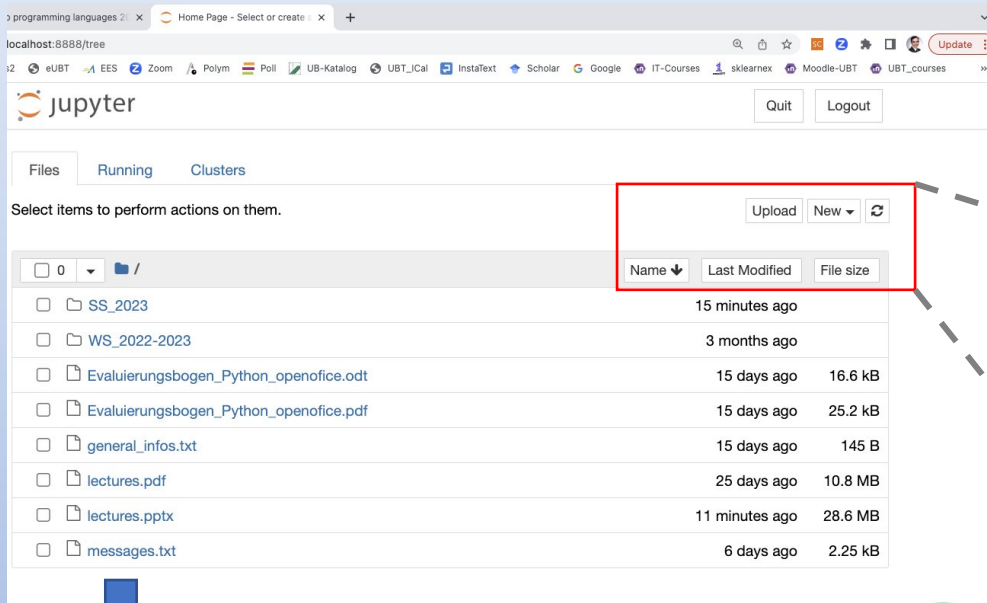
4) Launch the jupyter notebook (having google chrome is preferred over firefox):

jupyter-notebook

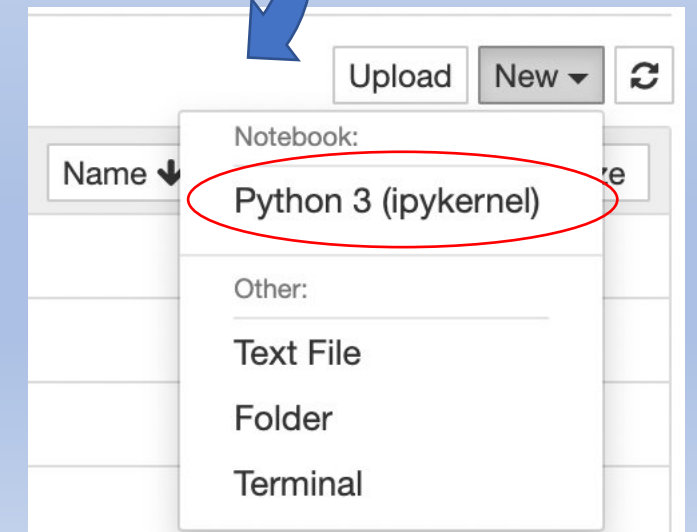
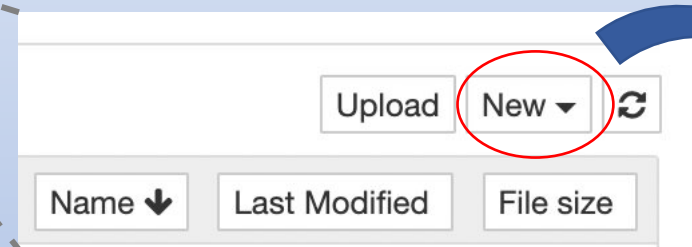
5) After using Jupyter Notebook, close its window (previously open in Google Chrome or Firefox), then click on the black Anaconda Powershell Prompt terminal and press CONTROL + C to close the prompt (you should see "(base)" again)

Local installation via Anaconda (or Miniconda)

After launching Jupyter Notebook, you will see:



Upper/Right corner



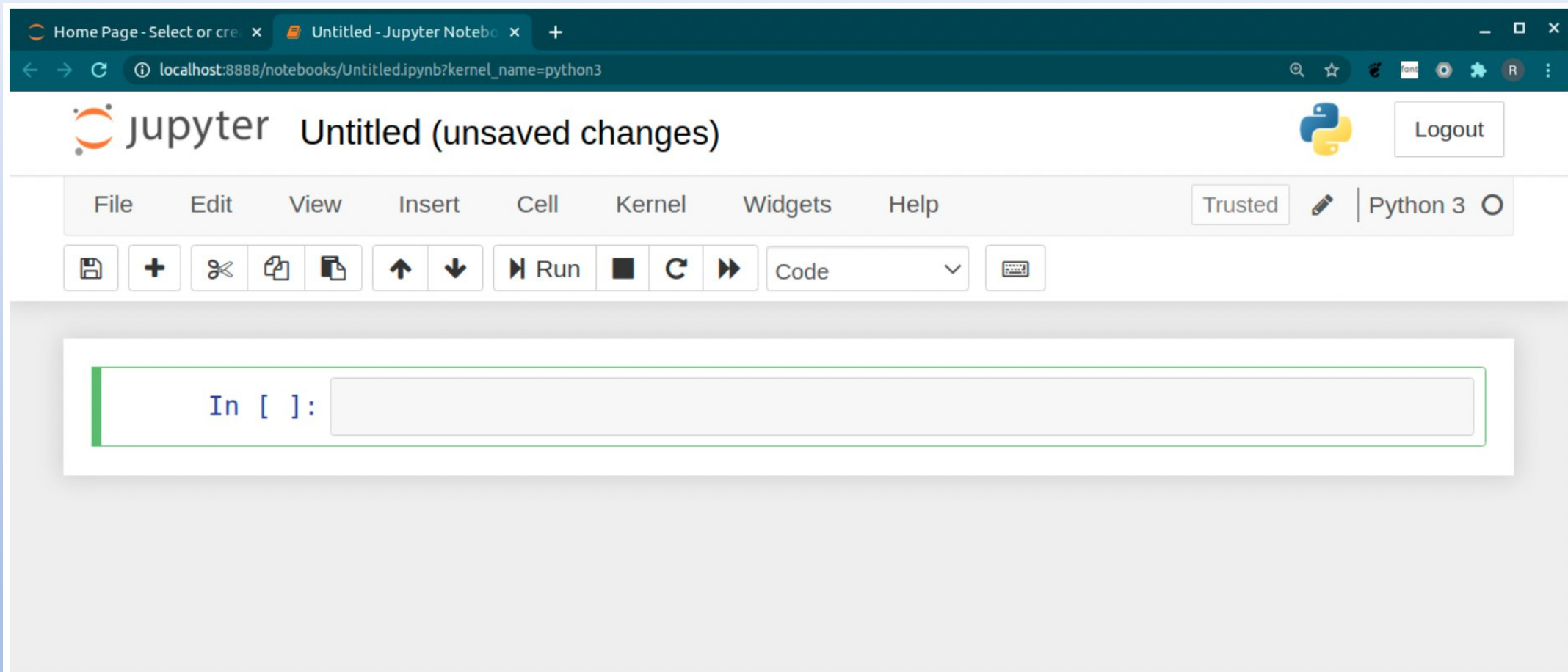
Your folders are here. You can either:

- 1) Search for your .ipynb file and click on it, **or**
- 2) Open a new, empty Jupyter notebook

Some other Python editors: Spyder, VSCode, PyCharm

Python session (after entering the command `python`): `>>> a = 2`

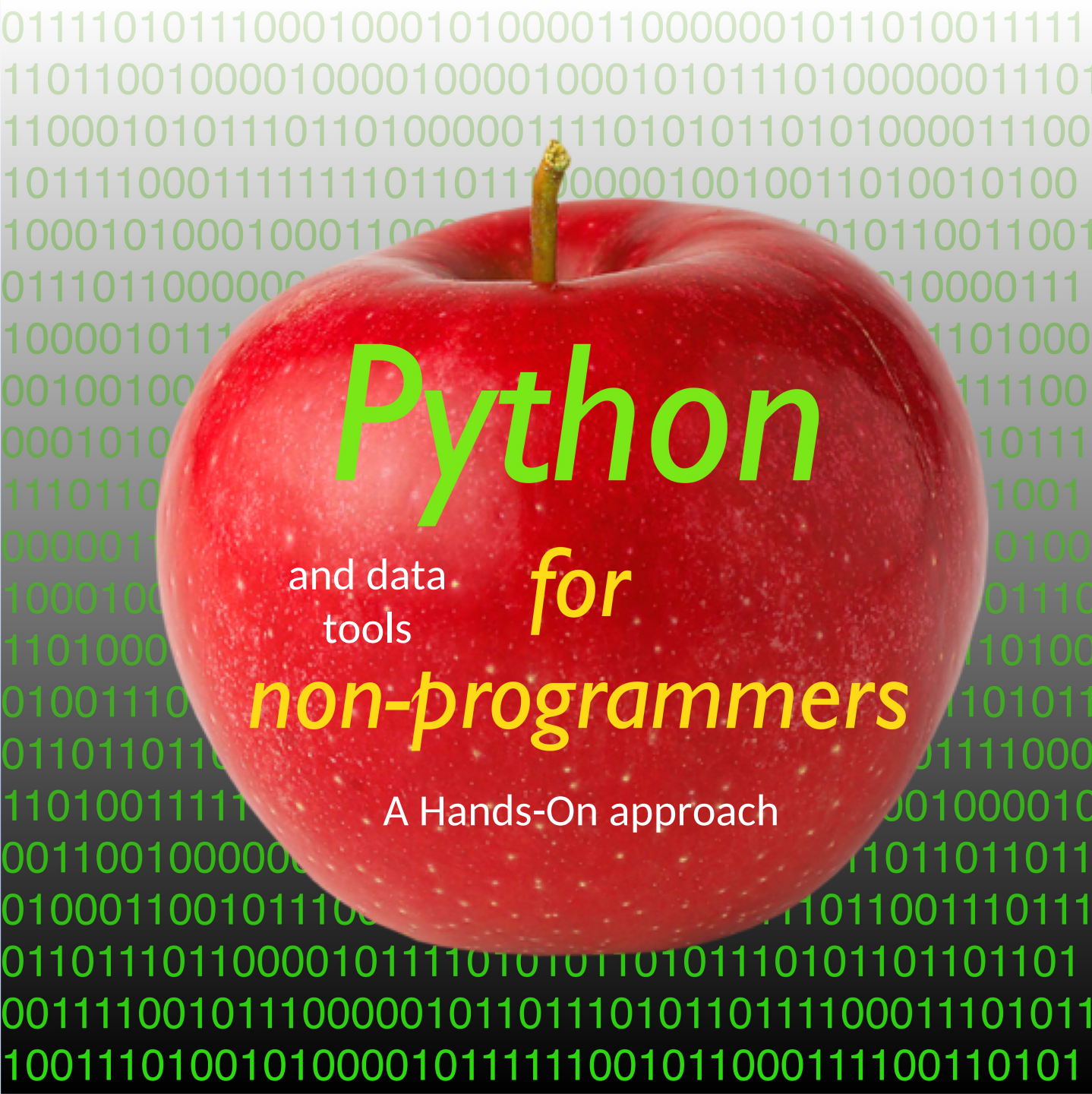
IPython session (after entering the command `ipython`): `In [1]:`



All Python codes discussed here can be written in the cells of the Jupyter Notebook and saved for later use.

Tip: press SHIFT + ENTER to execute a cell





UNIVERSITÄT
BAYREUTH

Delivered by

Dr. Rodrigo Albuquerque

(AG Ruckdäschel)

&

AG Künneth

Part of the project
“digitale Methoden”
(bidt-digikollegs)



- Brief overview of Python
- Installation and graphical interfaces
- Data types and operators
- Objects, modules and classes
- Strings
- Lists, sets and tuples
- Dictionary
- Arrays and matrices
- Loops (for, while)
- If statements
- Functions, pandas
- Files
- Plotting graphics

Please write / review all codes discussed here in your own notebook.
There are **MANY** concepts to learn that need to be **practiced!**

=> Open an empty jupyter notebook in your laptop

Some data types

a) Numeric (float, integer, complex)

Possible variable names:

a5, a_b, abc7, a_4, _4a

Wrong variable names:

5a, a-b

```
1 >>> a = 2
2 >>> type(a)
3 <class 'int'>
4 >>> b = 5.634
5 >>> type(b)
6 <class 'float'>
7 >>> c = 2 + 5j      # Comment: In Python, the complex number  $(-1)^{1/2}$  is "j" and not "i")
8 >>> type(c)
9 <class 'complex'>
```

only write "a = 2" (no need to write the symbol ">>>")

This is the output (do not write it!)

Comment: In Python, the complex number $(-1)^{1/2}$ is "j" and not "i")

Note: The symbol ">>>" will be very often found by you in descriptions of Python sessions. Inside Jupyter Notebook, we will not see these symbols.

Line numbers

Question: Which data type has d , where d is given by a multiplied by b ? Check by yourself...



Some data types

b) Boolean (= True or False)

```
>>> a=True
>>> type(a)
<class 'bool'>
>>> b = False
>>> type(b)
<class 'bool'>
```

Boolean expressions

```
>>> a=2
>>> b=3
>>> a == b
False
>>> a < b
True
>>> b >= a
True
```

Is *a* equal to *b*?

Is *a* smaller than *b*?

Is *b* larger than or equal to *a*?

* comparison operators: *>*, *<*, *>=*, *<=*, *==*, *!=*

Note: Comments (*#*)

```
>>> # This is a comment (ignored by Python)
>>>
```

Some data types

c) String

```
>>> a='Study'
>>> b=' Python!'
>>> type(a)
<class 'str'>
>>> a+b
'Study Python!'
>>> 3*b
' Python! Python! Python!'
>>> c='hello 2023'
>>> c
'hello 2023'
```

Convert string to other types:

```
>>> a = '10'
>>> a+a
'1010'
>>> float(a)
10.0
>>> int(a)
10
>>> complex(a)
(10+0j)
```

Convert other types to string:

```
>>> a=2022
>>> str(a)
'2022'
```

Direct the output:

```
>>> b = str(a)
```

Note: ' ' and " " are equivalent to define strings

Simple operations with numerical types

```
1 >>> a, b, c = 1, 2, 3
2 >>> a
3 1
4 >>> a + b * c
5 7
6 >>> a - b / c
7 0.3333333333333333
8 >>> c ** b
9 9
10 >>> a += 1
11 >>> a
12 2
13 >>> b *= 5
14 >>> b
15 10
```

Assign values to multiple variables

(other multi-variable assignment: $a = b = c = 1$)

"*" has precedence over "+"

"/" has precedence over "-"

c^b (Note: $**$ has precedence than $*$ and $/$)

new $a = \text{old } a + 1$ (a is redefined)

new $b = \text{old } b * 5$ (b is redefined)

Arithmetic operators

$+$, $-$, $*$, $**$, $/$, $//$, $\%$

Assignment operators

$=$, $+=$, $-=$, $*=$, $/=$, etc.

Try yourself different commands to learn more about operations and precedence!



Common operators (arithmetic and assignment)

Operator	Operation	Result
+	$2 + 3$	5
-	$5 - 1$	4
*	$2 * 3$	6
/	$9 / 4$	2.25
**	$2 ** 3$	8
//	$9 // 4$	2
%	$9 \% 4$	1
+=	$a += 1$	$a = a + 1$
-=	$a -= 1$	$a = a - 1$
*=	$a *= 3$	$a = a * 3$

Note: some operators work with strings too!

****:** Exponent

//: Integer division

%: Modulus (or remainder)

Tip: Check the suggested bibliography for all operators.

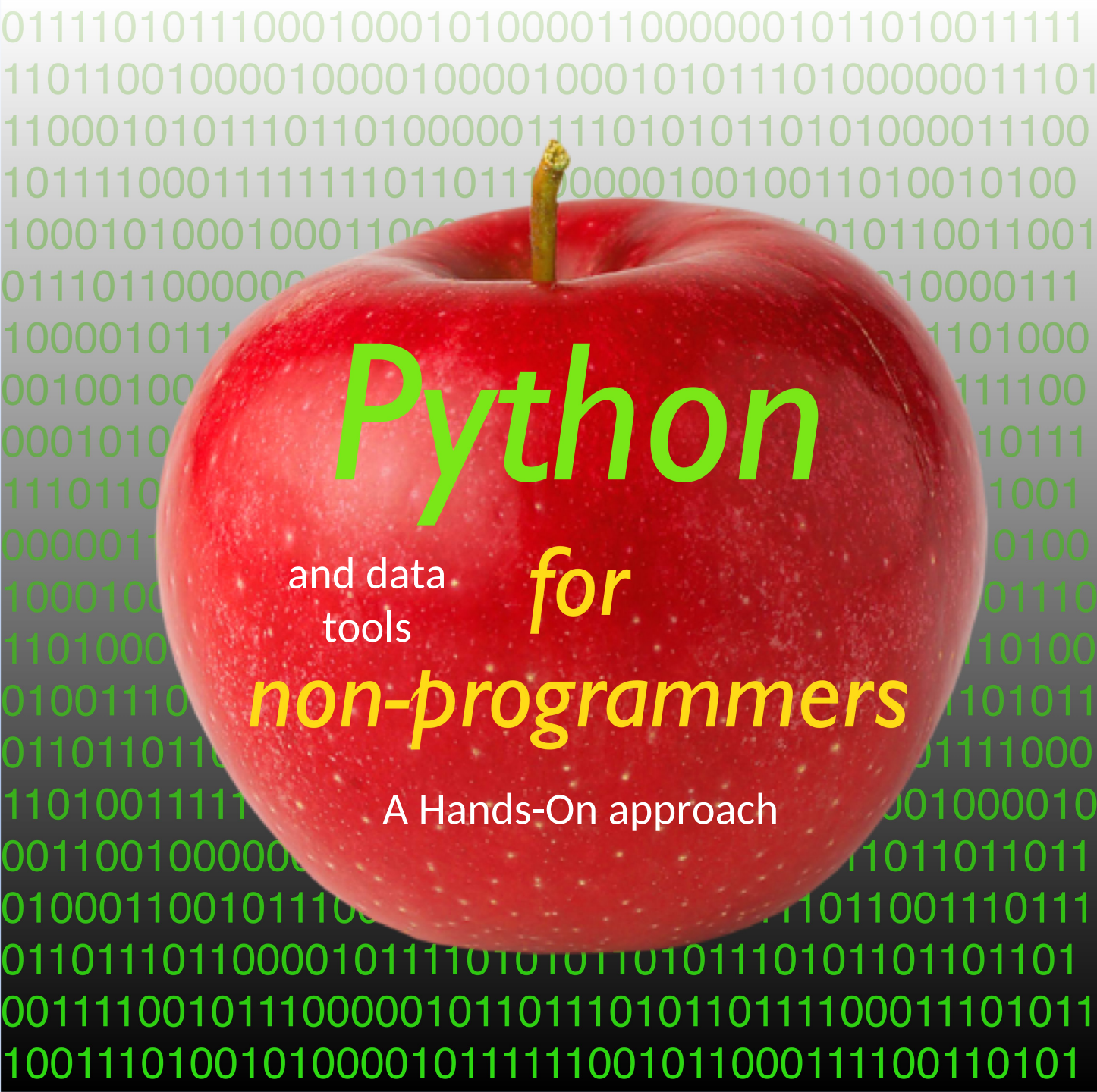
There are many classes of operators in Python!

- Arithmetic (+, -, *, **, /, //, %)
- Assignment (=, +=, -=, *=, **=, /=, //=, %=, etc.)
- Membership (in, not in)
- Comparison (>, <, >=, <=, ==, !=)
- Logical (and, or, not)
- Identity (is, is not)
- Bitwise (&, |, ^, ~, >>, <<)

Many of them will be discussed during the next lectures.

Check them e.g. here: https://www.w3schools.com/python/python_operators.asp





UNIVERSITÄT
BAYREUTH

Delivered by

Dr. Rodrigo Albuquerque

(AG Ruckdäschel)

&

AG Künneth

Part of the project
“digitale Methoden”
(bidt-digikollegs)



- Brief overview of Python
- Installation and graphical interfaces
- Data types and operators
- **Objects, modules and classes**
- Strings
- Lists, sets and tuples
- Dictionary
- Arrays and matrices
- Loops (for, while)
- If statements
- Functions, pandas
- Files
- Plotting graphics

In Python, almost everything is an object!

Classes define how instantiated objects behave

- Objects belong to **classes** (e.g. integers belong to class “**int**”, see previous slides)
- An object contains data and functions that can operate on the data (**attributes** + **methods**)
- Objects can be instantiated (created) from classes: `>>> obj = Class()`

“factory of objects”

A very simple object (automatically instantiated from class ‘str’):

```
>>> a = 'Hello!'
>>> type(a)
<class 'str'>
>>> a.upper()
'HELLO!'
>>> a.__len__()
6
>>> len(a)
6
```

The data ‘Hello!’ is stored in object “a”

Method: Convert the data stored in object “a” to uppercase

Attribute: Get the length attribute (number of characters stored in object “a”)

Built-in function `len()`

Tip: In Jupyter, press `a.` + **TAB** and `a._` + **TAB** to see a list of available methods and attributes of object `a`. `dir(a)` also helps!



Exercise

Use NumPy to solve the following: $\frac{e^{\cos(\frac{\pi}{2})}}{\ln 9}$

```
In [1]: from numpy import log, pi, cos, exp
```

```
In [2]: a = cos(pi/2)
        b = exp(a)
```

```
In [3]: b / log(9)
```

```
Out[3]: 0.45511961331341866
```

or simply `exp(cos(pi/2)) / log(9)`

However, **stepwise calculations** prevent errors!

Tip: Check what is stored in the temporary variable `_` (underscore)

Some interesting modules of the library NumPy

```
>>> import numpy as np
```

random: generate random numbers and statistical distributions (e.g. multivariate gaussian)

```
>>> np.random.rand()  
0.43077007514553145
```

```
>>> np.random.normal(size = 3)  
array([-0.0596406 , -0.73070764, -0.82219425])
```

linalg: Linear algebra operations with matrices and vectors (e.g. determinants, eigenvectors)

$$X = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$$

```
>>> np.linalg.det(X)  
-2.0000000000000004
```

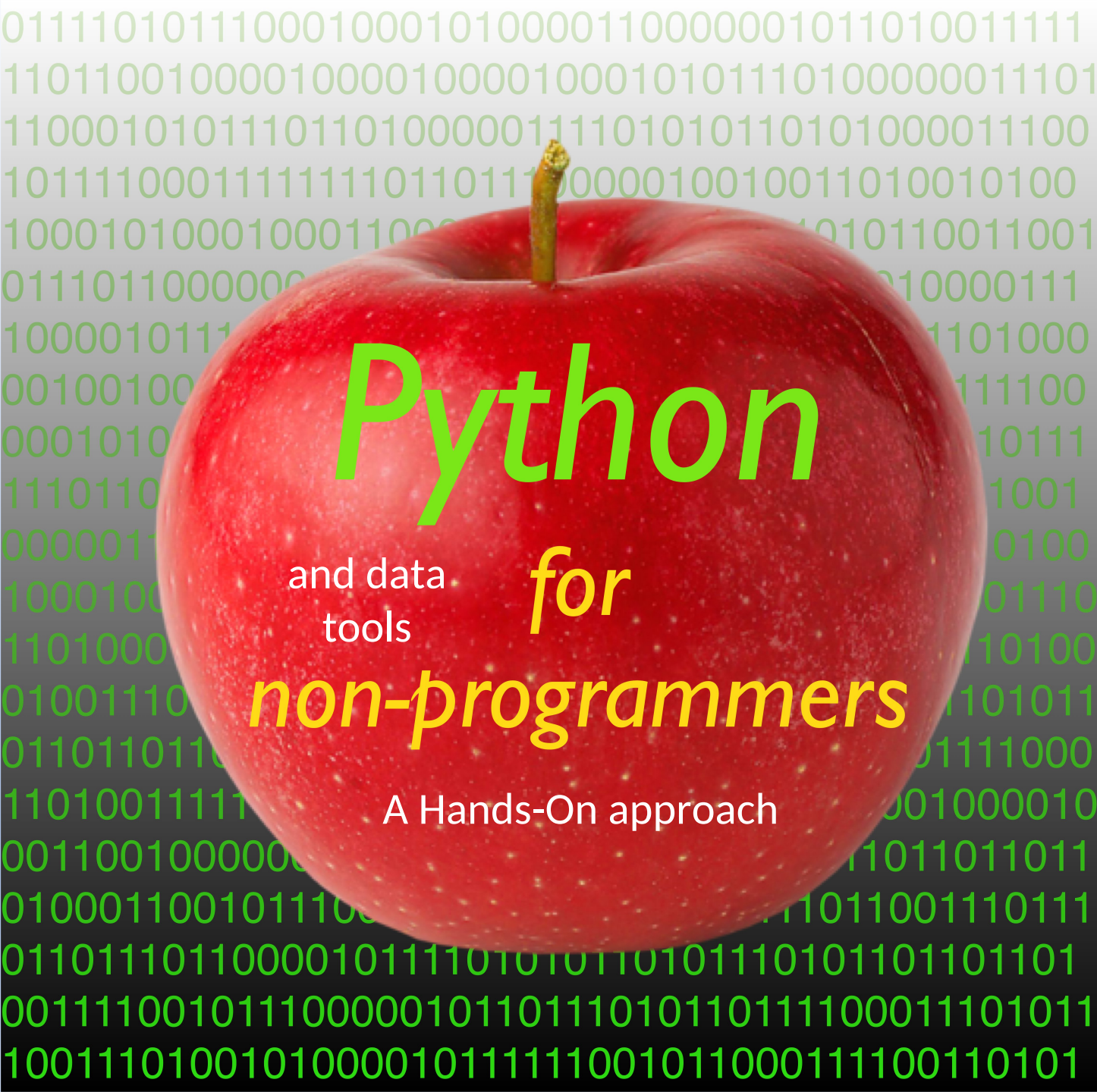
```
>>> np.linalg.eig(X)  
(array([-0.37228132, 5.37228132]),  
 matrix([[ -0.82456484, -0.41597356],  
         [ 0.56576746, -0.90937671]]))
```

NumPy has also many **mathematical and statistical functions**, **constants** and other **general functions**.

Other important libraries (> 200 are available per default)

- **matplotlib**: For making nice plots
- **pandas**: For the easy manipulation of datasets
- **scipy**: For optimization, signal processing, statistics
- **scikit-learn**: For machine learning applications
- **opencv**: For manipulating images (computer vision)

Thousands of free external libraries for a **large** number of applications are available! (This number keeps increasing...)



UNIVERSITÄT
BAYREUTH

Delivered by

Dr. Rodrigo Albuquerque

(AG Ruckdäschel)

&

AG Künneth

Part of the project
“digitale Methoden”
(bidt-digikollegs)



- Brief overview of Python
- Installation and graphical interfaces
- Data types and operators
- Objects, modules and classes
- **Strings**
- Lists, sets and tuples
- Dictionary
- Arrays and matrices
- Loops (for, while)
- If statements
- Functions, pandas
- Files
- Plotting graphics

Strings are also considered a **data structure**!

Errors: i) Check the element `a[20]`; ii) Try `a[0] = 'h'`

String: Sequence of letters and other characters enclosed in quotes.

(strings are **immutable!**)

```
In [1]: a = 'Hi. My name is John!'
```

```
In [2]: len(a) # 20 characters, including empty spaces
```

```
Out[2]: 20
```

```
In [3]: a[0] # localization index goes from 0 to 19
```

```
Out[3]: 'H'
```

```
In [4]: a[19]
```

```
Out[4]: '!'
```

```
In [5]: a[0:2] # element at position 2 is excluded! (Slice)
```

```
Out[5]: 'Hi'
```

```
In [6]: a[-1] # last element: from the end, the indices go from -1 to -20
```

```
Out[6]: '!'
```

```
In [7]: a[-20]
```

```
Out[7]: 'H'
```

```
>>> b = "hello!"
```

index →

0 1 2 3 4 5

H	e	l	l	o	!
---	---	---	---	---	---

-6 -5 -4 -3 -2 -1

← index

Note: `a[0:2]` and `a[:2]`
are equivalent.

Challenge: How to slice `b` (above)
using only negative indices to
extract "ello"? `b[-5:-1]`

Note: To include "!", use `b[-5:]`

Some string methods:

Exercise: Use the **Synopsis** section of the “Romeo and Juliet” entry in wikipedia to count how many times the words ‘Romeo’ and “Juliet” appeared. **Important:** Define your string using 3 quotes instead of 1.

Answer:

Romeo: 25, Juliet: 15

```
In [9]: a.count("i")           # a = 'Hi. My name is John!'
```

```
Out[9]: 2
```

```
In [10]: a.upper()
```

```
Out[10]: 'HI. MY NAME IS JOHN!'
```

```
In [11]: a.split()
```

```
Out[11]: ['Hi.', 'My', 'name', 'is', 'John!']
```

```
In [12]: a.split("i") # we defined "i" as the separator
```

```
Out[12]: ['H', '. My name ', 's John!']
```

```
In [13]: a.replace("i","O") # Note: a was not redefined (strings are immutable)
```

```
Out[13]: 'HO. My name Os John!'
```

Tip1: Check other string methods via `a.` + **TAB**

Tip2: Try out arithmetic operators (e.g. `a+b` or `2*a`) to learn more!

Try `a[::-1]` What happened here?



Formating strings

Great for:

- Displaying results in an organized way
- Customizing general text inside plots (\$ \$ is then available!)
- Writing files in Python

```
In [1]: var1, var2 = "Hello!", 1.4325346
```

```
In [2]: print('I have two variables: var1 and var2')
I have two variables: var1 and var2
```

```
In [3]: print(f"var1 = {var1} and var2 = {var2}") # string formating with f' {variable}'
var1 = Hello! and var2 = 1.4325346
```

```
In [4]: print(f"var1 = {var1:<10} and var2 = {var2:~.2f}") # {variable:formating options}
var1 = Hello!      and var2 = 1.43
```

```
In [5]: print(f"var1 = {var1:^10} and var2 = {var2:+010.2f}")
var1 =   Hello!    and var2 = +000001.43
```

```
In [6]: var3 = f"var1 = {var1:^10} and var2 = {var2:+010.2f}"
var3
```

```
Out[6]: 'var1 =   Hello!    and var2 = +000001.43'
```

Alignment: <, >, ^ (left, right or center)
 Add explicit signal: +
 Use 2 decimal places: .2f
 Use 10 characters/places in total: 10
 Add zeros on the left: 0



Exercise

Generate string variables with the information below and use the `print()` command to show in an organized way the name, age, number of followers and salary of the respective users:

User	Age	# Followers	Salary(Eur)
Mike	30	139284	100230.00
John	40	4738475	89780.00
Stephen	35	1838484	93250.00

Tip1: place the `print()` commands above each other and inside the same cell of Jupyter.

**Possible
Answer:**

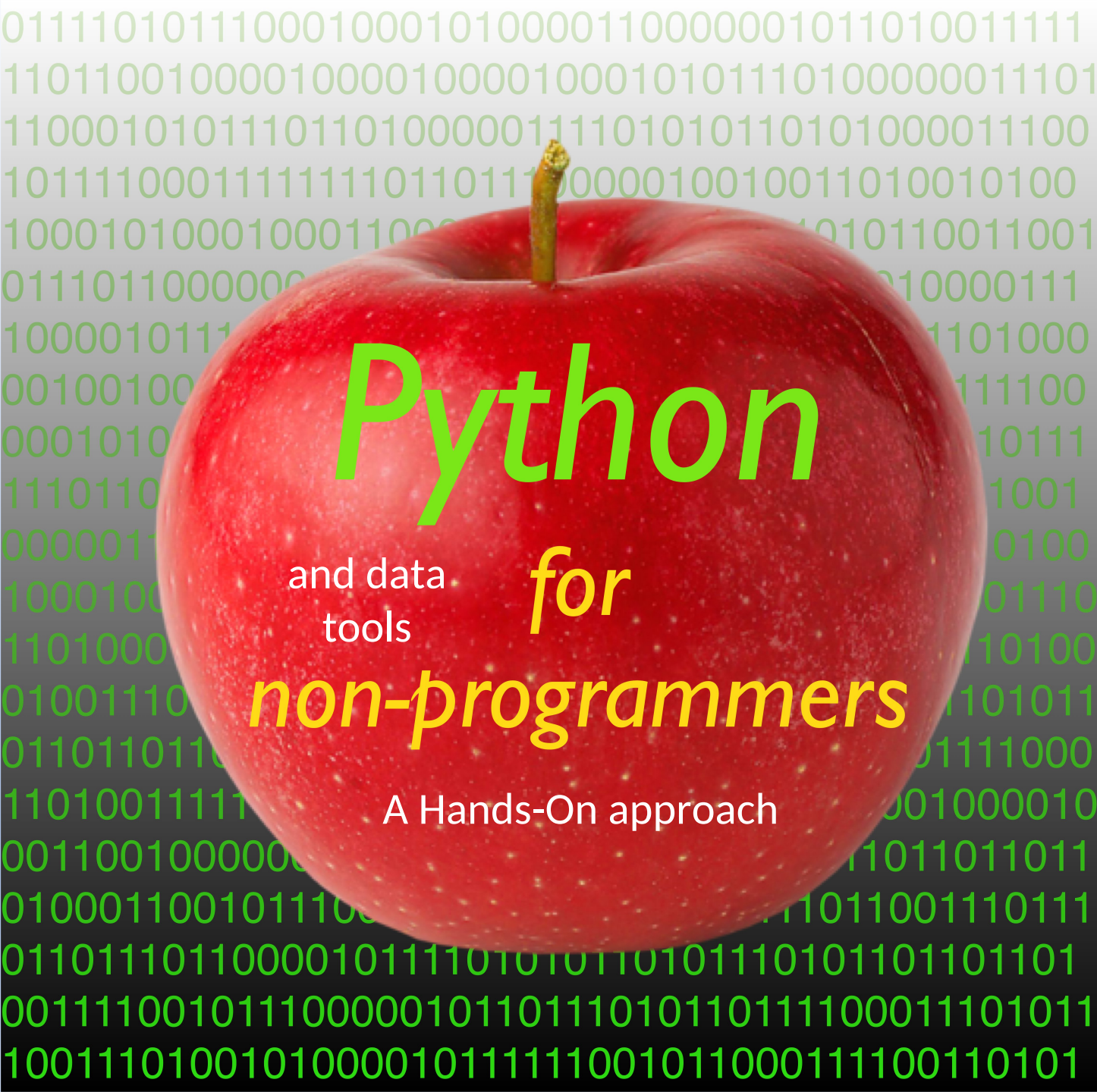
```
In [1]: name1, name2, name3 = "Mike", "John", "Stephen"
        age1, age2, age3 = 30, 40, 35
        follow1, follow2, follow3 = 139284, 4738475, 1838484
        salary1, salary2, salary3 = 100230.00, 89780.00, 93250.00
        name, age, follow, salary = "Name", "Age", "# Followers", "Salary (Eur)"
```

```
In [8]: print(f"{name:<9}{age:>5}{follow:>15}{salary:>15}")
        print(f"{name1:<9}{age1:>5}{follow1:>15}{salary1:>15.2f}")
        print(f"{name2:<9}{age2:>5}{follow2:>15}{salary2:>15.2f}")
        print(f"{name3:<9}{age3:>5}{follow3:>15}{salary3:>15.2f}")
```

Name	Age	# Followers	Salary (Eur)
Mike	30	139284	100230.00
John	40	4738475	89780.00
Stephen	35	1838484	93250.00

Tip2: To show comma as thousands separator: `{salary1:>15,.2f}`





UNIVERSITÄT
BAYREUTH

Delivered by

Dr. Rodrigo Albuquerque

(AG Ruckdäschel)

&

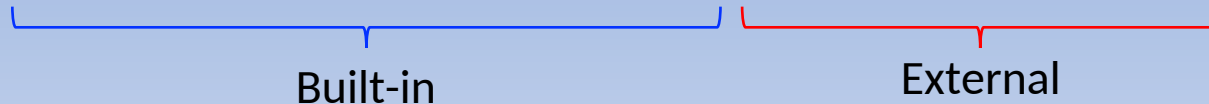
AG Künneth

Part of the project
“digitale Methoden”
(bidt-digikollegs)



- Brief overview of Python
- Installation and graphical interfaces
- Data types and operators
- Objects, modules and classes
- Strings
- **Lists**, sets and tuples
- Dictionary
- Arrays and matrices
- Loops (for, while)
- If statements
- Functions, pandas
- Files
- Plotting graphics

In Python, data can be grouped using **DATA STRUCTURES**:
lists, sets, tuples, dictionaries, arrays, matrices, etc



List: collection of objects of any data type (separated by comma)

```
In [45]: c = [10, 'hello', 3.14]
```

```
In [46]: len(c)
```

```
Out[46]: 3
```

```
In [47]: type(c)
```

```
Out[47]: list
```

```
In [48]: c[0]
```

```
Out[48]: 10
```

```
In [49]: c[0] + c[2]
```

```
Out[49]: 13.14
```

```
In [50]: c[:2]
```

```
Out[50]: [10, 'hello']
```

```
In [51]: c[2] = 5
```

```
In [52]: c
```

```
Out[52]: [10, 'hello', 5]
```

```
In [53]: c[1][4]
```

```
Out[53]: 'o'
```

List in a list! (**nested** list) 

```
In [54]: d = [1, 'Hi', [20, 40, 60]]
```

```
In [55]: len(d)
```

```
Out[55]: 3
```

```
In [56]: d[2][1]
```

```
Out[56]: 40
```

lists are
mutable!

multiple
indexing!

Tip 1: Command `list(X)` converts
X (e.g. array or set) into a list.

Tip 2: Check if 'ho' is in the list `a`:

```
>>> 'ho' in a
True
```

`in` and `not in`: membership operators

methods & commands

```
In [38]: a = [1, 2, 3, 3, 'ho']
```

```
In [39]: a.append(5.4)
```

```
In [40]: a
```

```
Out[40]: [1, 2, 3, 3, 'ho', 5.4]
```

```
In [41]: a.remove(5.4)
```

```
In [42]: a
```

```
Out[42]: [1, 2, 3, 3, 'ho']
```

```
In [43]: a.count(3)
```

```
Out[43]: 2
```

```
In [44]: a.index('ho')
```

```
Out[44]: 4
```

```
In [45]: print(a)
```

```
[1, 2, 3, 3, 'ho']
```



List comprehension

iterator: gives one value at a time (here, 10 integers, from 0 to 9)

```
In [2]: [i for i in range(10)]
```

```
Out[2]: [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

```
In [4]: [2*i for i in range(10)]
```

```
Out[4]: [0, 2, 4, 6, 8, 10, 12, 14, 16, 18]
```

Tip: Create an empty list as:

```
>>> a = []
```

or

```
>>> a = list()
```

Tip: Try out arithmetic operators (e.g. `a+b` or `2*a`) to learn more!

for loop: iterates over each element of the iterator `range()`

Iterating over elements of existing lists:

```
In [30]: a = [1,2,3,3,'ho']
         [item*2 for item in a]
```

```
Out[30]: [2, 4, 6, 6, 'hoho']
```

“item” = could be any other name instead...

```
In [31]: a = [1,2,3,3,'ho']
         for whatever in a:
             print(whatever)
```

```
1
2
3
3
ho
```


Exercise

Use the nested lists below to show in an organized way (via the command `print()`) the name, age, number of followers and salary of the respective users (use part of the code of the previous exercise!)

```
info = ["User", "Age", "# Followers", "Salary (Eur)",
        ["Mike", 30, 139284, 100230.00],
        ["John", 40, 4738475, 89780.00],
        ["Stephen", 35, 1838484, 93250.00]]
```

Tip: First print the first sub-list, then iterate over the remaining lists and print them too.

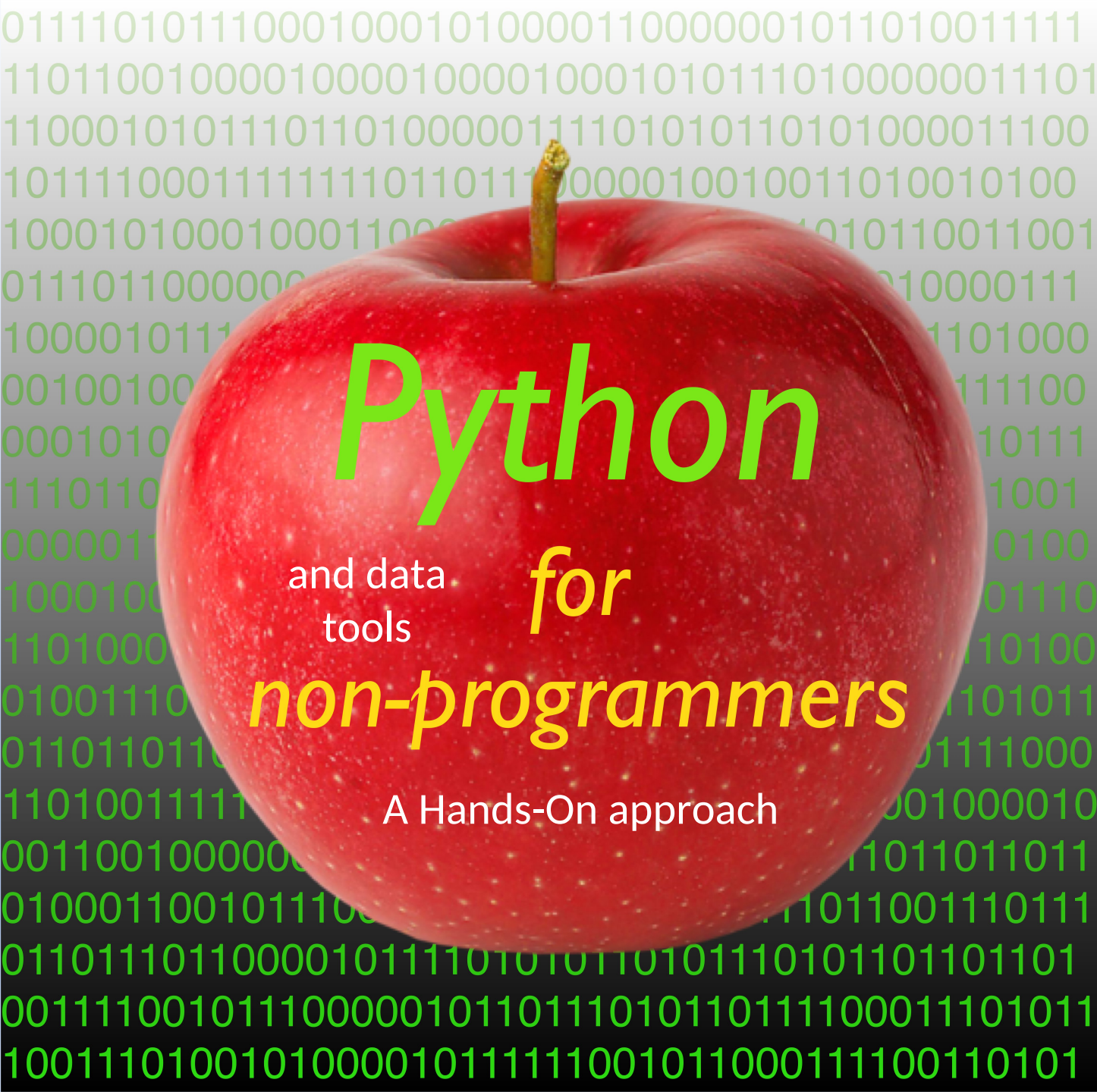
```
In [1]: info = ["User", "Age", "# Followers", "Salary (Eur)",
                ["Mike", 30, 139284, 100230.00],
                ["John", 40, 4738475, 89780.00],
                ["Stephen", 35, 1838484, 93250.00]]

In [2]: print(f"{info[0][0]:<9}{info[0][1]:>5}{info[0][2]:>15}{info[0][3]:>15}")
        for item in info[1:]: # first element does not participate
            user = item[0]
            age = item[1]
            follow = item[2]
            salary = item[3]
            print(f"{user:<9}{age:>5}{follow:>15}{salary:>15.2f}")
```

User	Age	# Followers	Salary (Eur)
Mike	30	139284	100230.00
John	40	4738475	89780.00
Stephen	35	1838484	93250.00

Note: Lines 3-6 of `In [2]` could have been written as one single line.
e.g. `us, ag, fo, sa = item` (**unpacking** the list "item")





UNIVERSITÄT
BAYREUTH

Delivered by

Dr. Rodrigo Albuquerque

(AG Ruckdäschel)

&

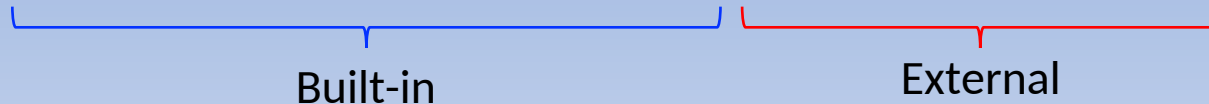
AG Künneth

Part of the project
“digitale Methoden”
(bidt-digikollegs)



- Brief overview of Python
- Installation and graphical interfaces
- Data types and operators
- Objects, modules and classes
- Strings
- Lists, **sets** and tuples
- Dictionary
- Arrays and matrices
- Loops (for, while)
- If statements
- Functions, pandas
- Files
- Plotting graphics

In Python, data can be grouped using **DATA STRUCTURES**:
lists, sets, tuples, dictionaries, arrays, matrices, etc



Tip: To create an empty set:

```
>>> a = set()
```

Set: Unordered group of *unique* elements (data types can be different)
 They are mutable (use `frozenset()` for immutable sets)
 Sets are non-subscriptable (E.g., `a[0]` gives an error!)

```
In [72]: a = [1,2,3,3,'ho',1]
         set(a)
```

```
Out[72]: {1, 2, 3, 'ho'}
```

The order of the elements can change!

```
In [73]: b = set(a)
         type(b)
```

```
Out[73]: set
```

```
In [74]: len(b)
```

```
Out[74]: 4
```

```
In [75]: b.add("hi")
         b
```

```
Out[75]: {1, 2, 3, 'hi', 'ho'}
```

Note: method `append()` does not work!

```
In [76]: b.remove("ho")
         b
```

```
Out[76]: {1, 2, 3, 'hi'}
```

Important operations with sets:

```
In [1]: a=set([1,2,3])
        b=set([3,4,5])
```

```
In [2]: a - b # elements of "a" not present in "b"
        # alternatively, a.difference(b)
```

```
Out[2]: {1, 2}
```

```
In [3]: a | b # merge sets (union)
        # alternatively, a.union(b)
```

```
Out[3]: {1, 2, 3, 4, 5}
```

```
In [4]: a & b # common elements (intersection)
        # or a.intersection(b)
```

```
Out[4]: {3}
```

```
In [5]: a ^ b # elements NOT present on both sets
        # or a.symmetric_difference(b)
```

```
Out[5]: {1, 2, 4, 5}
```

➡ Search the suggested bibliography for **many** more methods!



Exercise1:

Use the **Synopsis** section of the “Romeo and Juliet” entry in wikipedia to roughly* count how many words in total are there and how many unique words are there. **Important:** Define your string using **3 quotes instead of 1**. Remember to use the string method `split()` and the built-in functions `len()` and `set()`.

*The final set will contain elements like “/” (it is not a word, but OK!)

Answer:

```
In [82]: text = text.split()
         text_unique = set(text)
         len(text), len(text_unique)
```

```
Out[82]: (652, 356)
```

Alternative (worse readability):

```
>>> len( set(synopsys.split()) & set(dateandtext.split()) )
```

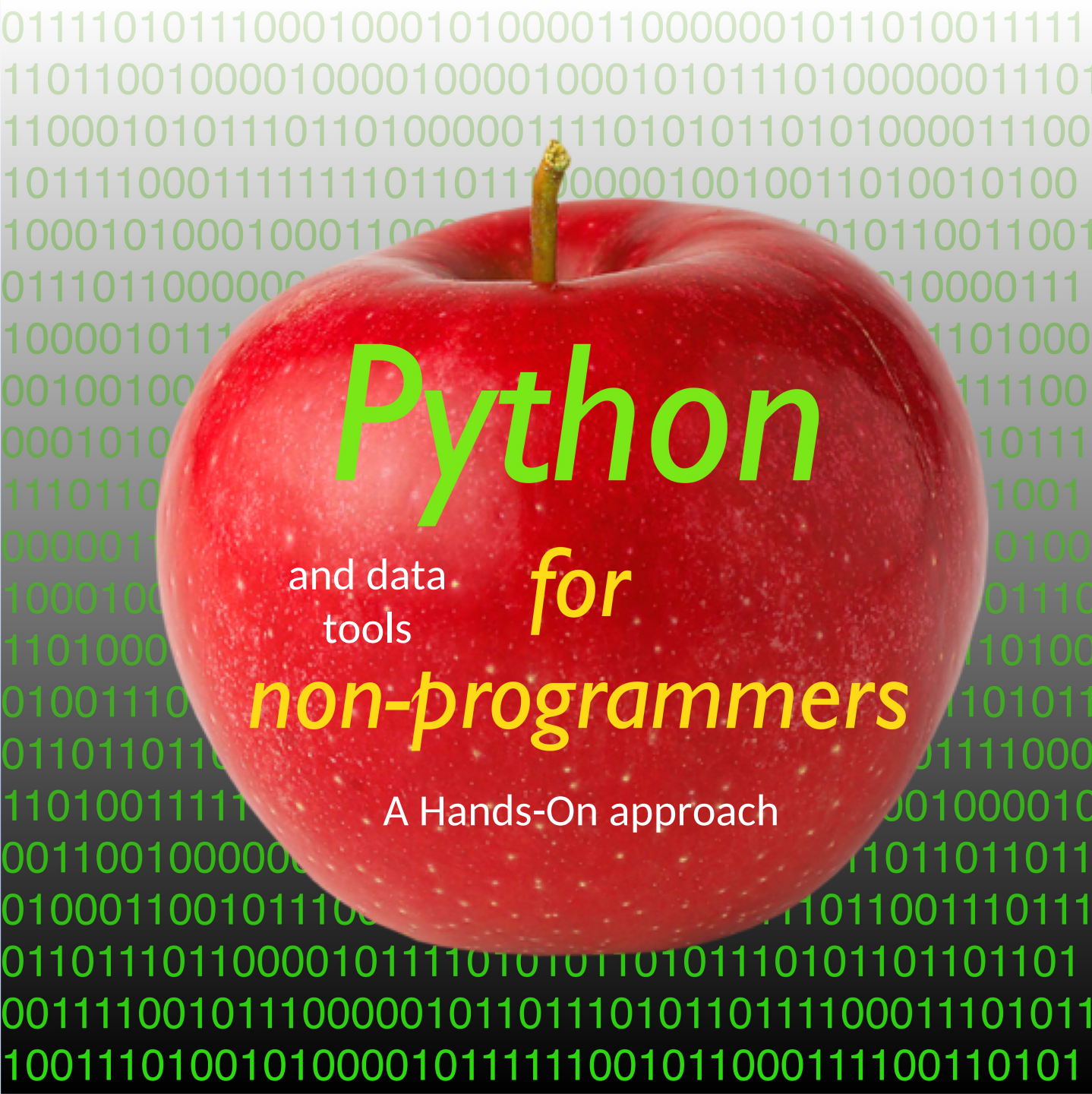
Exercise2:

Generate a set with the **Synopsis** section of the “Romeo and Juliet” entry in wikipedia and another set with the **Date and text** section. How many “words” appear on both sets? **Important:** Define your string using **3 quotes instead of 1**. Remember to use the string method `split()`, the built-in function `set()` and the operator `&`.

```
>>> synopsys = """ blabla """
>>> dateandtext = """ blabla """
>>> synopsys = synopsys.split()
>>> dateandtext = dateandtext.split()
>>> synopsys = set(synopsys)
>>> dateandtext = set(dateandtext)
>>> len(synopsys & dateandtext)
```

Answer: 48 words are present in both sets

{'However,', 'Juliet', 'Juliet's', 'Juliet.', 'Nurse', 'Romeo', 'The', 'a', 'about', 'all', 'an', 'and', 'are', 'as', 'be', 'before', 'between', 'but', 'by', 'for', 'from', 'had', 'he', 'his', 'in', 'into', 'is', 'it', 'makes', 'more', 'not', 'now', 'of', 'on', 'one', 'or', 'play', 'she', 'that', 'the', 'through', 'to', 'two', 'under', 'was', 'when', 'with', 'years'}



UNIVERSITÄT
BAYREUTH

Delivered by

Dr. Rodrigo Albuquerque

(AG Ruckdäschel)

&

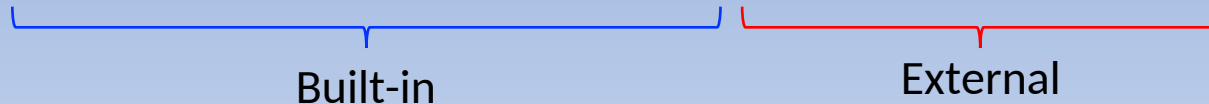
AG Künneth

Part of the project
“digitale Methoden”
(bidt-digikollegs)



- Brief overview of Python
- Installation and graphical interfaces
- Data types and operators
- Objects, modules and classes
- Strings
- Lists, sets and **tuples**
- Dictionary
- Arrays and matrices
- Loops (for, while)
- If statements
- Functions, pandas
- Files
- Plotting graphics

In Python, data can be grouped using **DATA STRUCTURES**:
lists, sets, tuples, dictionaries, arrays, matrices, etc



Tuple is an immutable collection of elements of any data type just used to store the data to be consulted later.
 Tuple elements are inside parantheses.

Note: If one of the tuple elements is a (mutable) list, we can change the value of any element of that list, therefore indirectly changing the tuple!

```
In [1]: a = 3, "hello", 1.7

In [2]: a
Out[2]: (3, 'hello', 1.7)

In [3]: type(a)
Out[3]: tuple

In [4]: a[0]
Out[4]: 3

In [5]: 2*a
Out[5]: (3, 'hello', 1.7, 3, 'hello', 1.7)

In [6]: a.count("hello"), a.index(1.7)
Out[6]: (1, 2)

In [7]: x, y, z = a # unpacking tuple "a"

In [8]: y
Out[8]: 'hello'

In [9]: tuple([1,2,3]) # convert list to tuple
Out[9]: (1, 2, 3)
```

```
In [1]: a = (2, 3, ["a", "b"])

In [2]: a
Out[2]: (2, 3, ['a', 'b'])

In [3]: len(a)
Out[3]: 3

In [4]: a[2][0] = "z"

In [5]: a
Out[5]: (2, 3, ['z', 'b'])

In [6]: a[0] = "z"

-----
TypeError                                Traceback (most recent call last)
/var/folders/g0/9w3xr1nd031ftmvx29wmvw9w0000gn/T/ipykernel_9871/3545248358.py in <module>
----> 1 a[0] = "z"

TypeError: 'tuple' object does not support item assignment
```

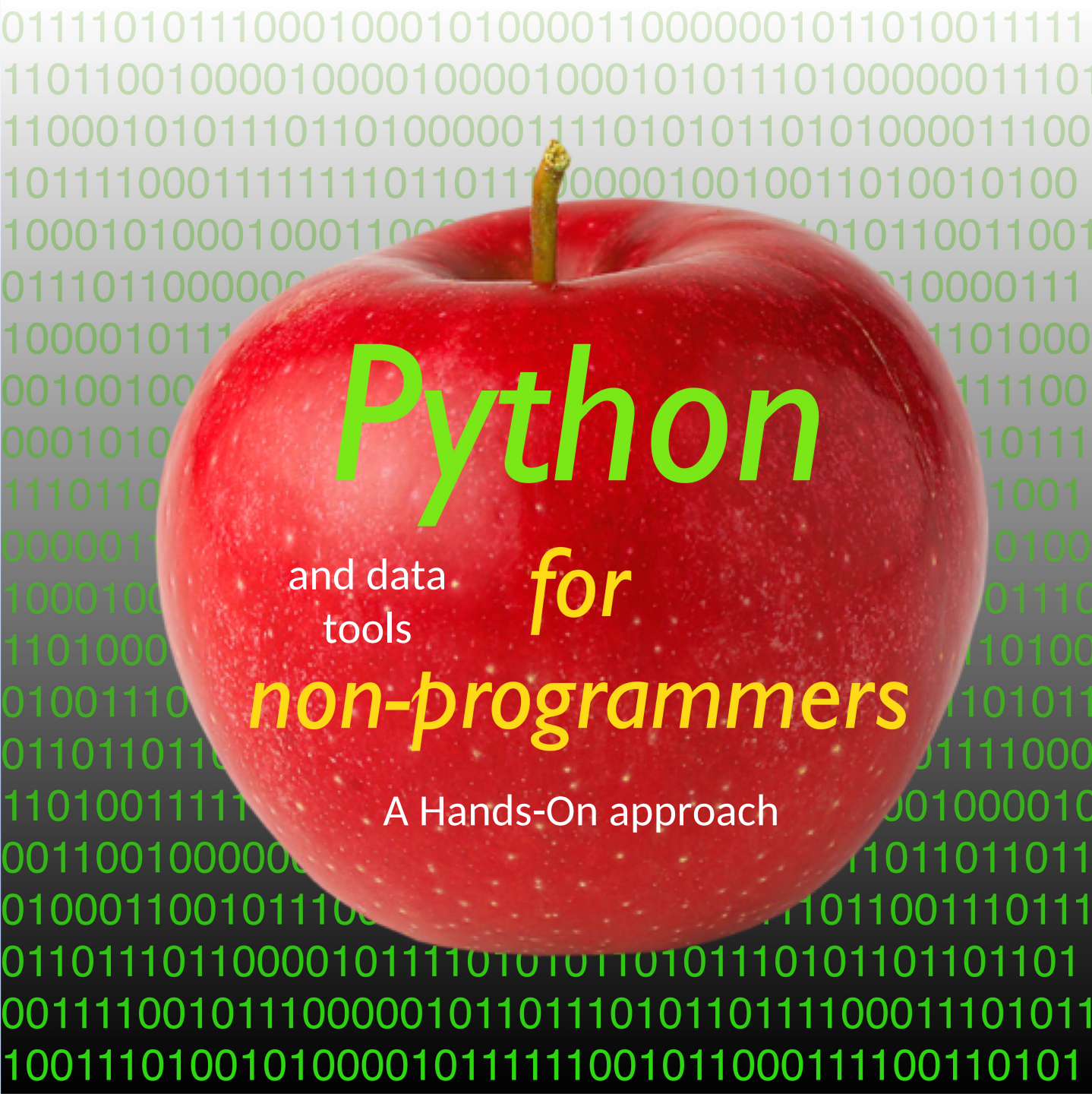
Iterate over the tuple elements:

```
>>> for item in a:
    print(item)
```

Generate tuples:

```
>>> tuple(i for i in range(5))
(0, 1, 2, 3, 4)
```





UNIVERSITÄT
BAYREUTH

Delivered by

Dr. Rodrigo Albuquerque

(AG Ruckdäschel)

&

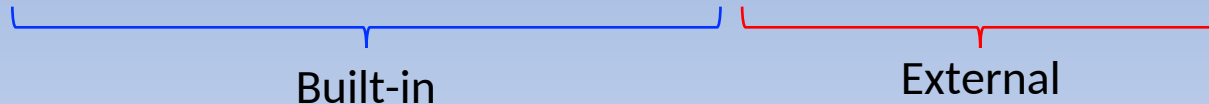
AG Künneth

Part of the project
“digitale Methoden”
(bidt-digikollegs)



- Brief overview of Python
- Installation and graphical interfaces
- Data types and operators
- Objects, modules and classes
- Strings
- Lists, sets and tuples
- Dictionary
- Arrays and matrices
- Loops (for, while)
- If statements
- Functions, pandas
- Files
- Plotting graphics

In Python, data can be grouped using **DATA STRUCTURES:**
lists, sets, tuples, dictionaries, arrays, matrices, etc



Dictionary is a mutable collection of pairs of **key : value** (keys are unique) of any data type.

```
>>> Z = {'H': 1, 'He': 2, 'Li': 3}
>>> type(Z)
<class 'dict'>
>>> Z['H']
1
>>> Z['Be'] = 4
>>> Z
{'H': 1, 'He': 2, 'Li': 3, 'Be': 4}
>>> Z.pop('Be')
4
>>> Z
{'H': 1, 'He': 2, 'Li': 3}
>>> len(Z)
3
>>> Z.keys()
dict_keys(['H', 'He', 'Li'])
>>> Z.values()
dict_values([1, 2, 3])
>>> Z1 = {'H': 1, 'Be': 4}
>>> Z.update(Z1) # Repeated key is overwritten!
>>> Z
{'H': 1, 'He': 2, 'Li': 3, 'Be': 4}
```

Iterating over elements

```
In [1]: a = {"H":1, "He":2}
```

```
In [2]: for item in a.keys():
        print(item)
```

```
H
He
```

```
In [3]: a, b = ["H", "He"], [1, 2]
```

```
In [4]: mydict = {} # empty dictionary
        for key, value in zip(a,b):
            mydict[key] = value
```

```
In [5]: mydict
```

```
Out[5]: {'H': 1, 'He': 2}
```

Tip: Create an empty dictionary with e.g. `>>> a = dict()`



Exercise 1

Create a dictionary having the first names of the Beatles as keys and the length of their first names as values.

```
In [1]: names = ["John", "Paul", "Ringo", "George"]
        mydict = {}
        for name in names:
            mydict[name] = len(name)

In [2]: mydict

Out[2]: {'John': 4, 'Paul': 4, 'Ringo': 5, 'George': 6}
```

Exercise 2

Create a dictionary having the keys "Milky Way" and "Andromeda". Each value stored is a new dictionary having "Planets" and "Stars" as keys and the following numbers as values, respectively: 3E12 and 1E12 (Andromeda) and 5E9 and 2.5E9 (Milky Way). How to consult the number of planets of Andromeda?

Possible answer:

```
In [3]: milky = {"Planets": int(5E9), "Stars": int(2.5E9)}
        androm = {"Planets": int(3E12), "Stars": int(1E12)}
        galaxies = {"Milky Way": milky, "Andromeda": androm}

In [4]: galaxies

Out[4]: {'Milky Way': {'Planets': 5000000000, 'Stars': 2500000000},
        'Andromeda': {'Planets': 3000000000000, 'Stars': 1000000000000}}

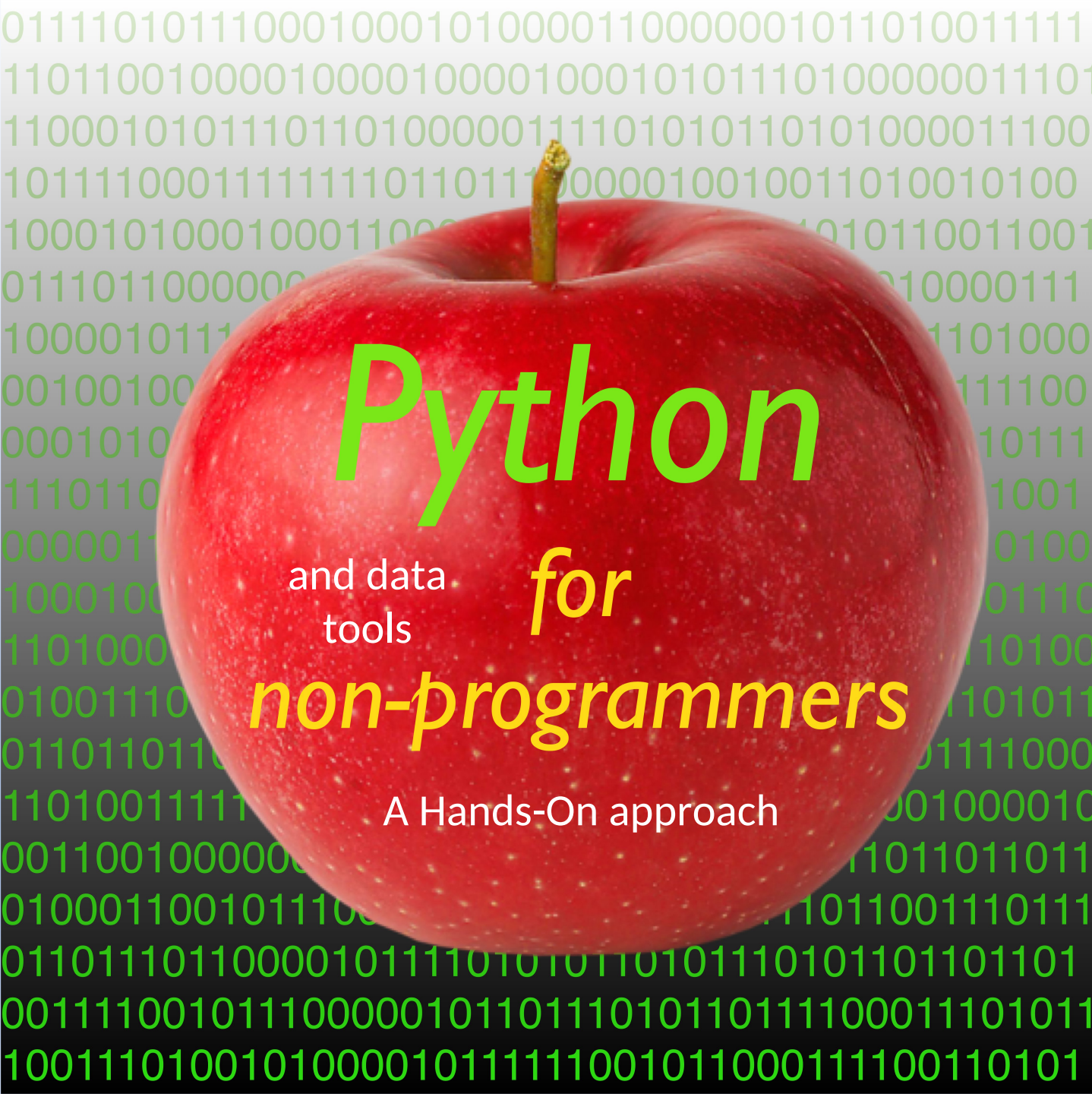
In [5]: galaxies["Andromeda"]["Planets"]

Out[5]: 3000000000000
```

Dictionary of dictionaries: very common!

Dictionary key types = "City" "School" "Class" "Name" "Discipline" "Exam Type"
 Another example: `GE_School_Grades['Bayreuth']['Graser']['3C']['Will Smith']['Math']['Second Exam']`





UNIVERSITÄT
BAYREUTH

Delivered by

Dr. Rodrigo Albuquerque

(AG Ruckdäschel)

&

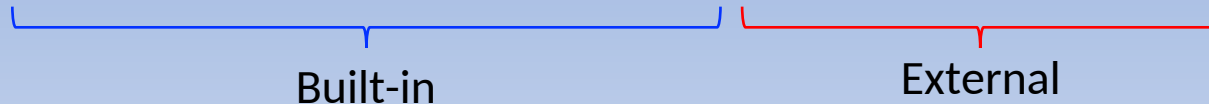
AG Künneth

Part of the project
“digitale Methoden”
(bidt-digikollegs)



- Brief overview of Python
- Installation and graphical interfaces
- Data types and operators
- Objects, modules and classes
- Strings
- Lists, sets and tuples
- Dictionary
- **Arrays** and matrices
- Loops (for, while)
- If statements
- Functions, pandas
- Files
- Plotting graphics

In Python, data can be grouped using **DATA STRUCTURES:**
lists, sets, tuples, dictionaries, arrays, matrices, etc



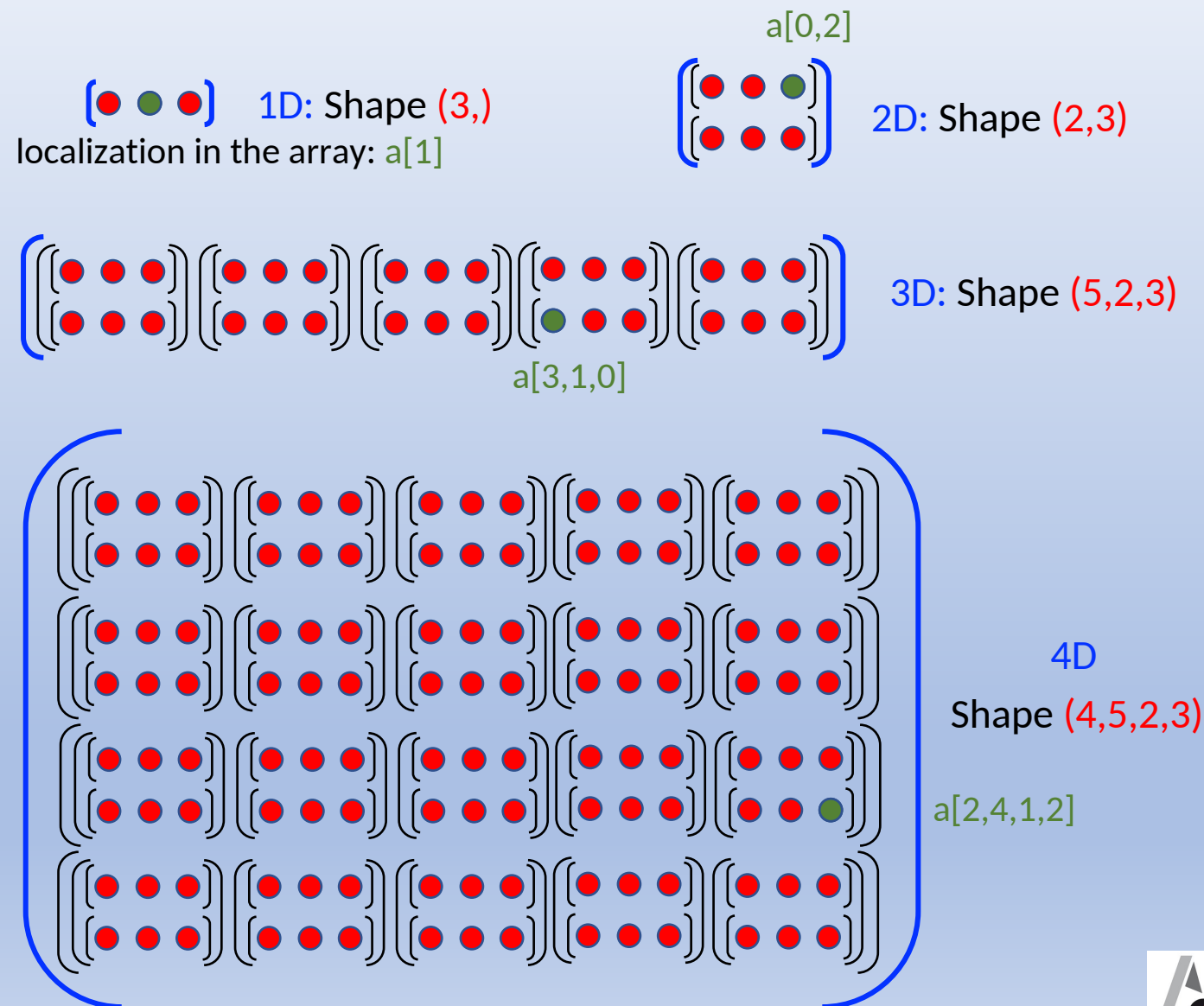
Tip: `np.array(a, dtype = int)`

int, float, complex, str, np.int32, etc

Array: Collection of elements of the same data type and with dimension d ($d = 1, 2, \dots, N$)

```

1  >>> import numpy as np
2  >>> a = [1, 2, 3]
3  >>> a = np.array(a)
4  >>> a
5  array([1, 2, 3])
6  >>> a.shape
7  (3,)
8  >>> b = [[1,2,3],[4,5,6]]
9  >>> b = np.array(b)
10 >>> b
11 array([[1, 2, 3],
12        [4, 5, 6]])
13 >>> b.shape
14 (2, 3)
15 >>> b.size
16 6
17 >>> b[0,1]
18 2
19 >>> b[1,:]
20 array([4, 5, 6])
    
```



Other ways to generate arrays

Tip: In Jupyter, press **SHIFT + TAB** near a command to get info.

```
In [1]: import numpy as np
```

```
In [2]: np.linspace(1, 2, 5) # 5 points in the range [1,2]
```

```
Out[2]: array([1. , 1.25, 1.5 , 1.75, 2.  ])
```

```
In [3]: np.arange(1, 2, 0.25)
```

spacing = 0.25
(last number is excluded)

```
Out[3]: array([1. , 1.25, 1.5 , 1.75])
```

```
In [4]: np.zeros((2,3)) # Also try: np.ones((2,3))
```

```
Out[4]: array([[0., 0., 0.],
               [0., 0., 0.]])
```

```
In [5]: np.identity(3)
```

```
Out[5]: array([[1., 0., 0.],
               [0., 1., 0.],
               [0., 0., 1.]])
```

```
In [6]: np.random.random((2,3))
```

```
Out[6]: array([[0.31061503, 0.71400417, 0.73183976],
               [0.49391461, 0.70985227, 0.21991922]])
```

```
In [7]: np.random.normal(loc=2, scale=0.7, size=4)
```

```
Out[7]: array([1.68576625, 2.18509204, 1.57062214, 2.35008457])
```

Importing / exporting arrays

```
In [2]: a = np.random.random((1000,3))
        np.save("myarray", a)
```

```
In [3]: b = np.load("myarray.npy")
        b
```

```
Out[3]: array([[0.92311745, 0.61963683, 0.99603435],
               [0.46528023, 0.2557813 , 0.95800069],
               [0.22207387, 0.99658931, 0.43508635],
               ...,
               [0.54641681, 0.61120011, 0.78739067],
               [0.49190005, 0.30582528, 0.89564933],
               [0.86677168, 0.60828137, 0.57487395]])
```

```
In [4]: np.savetxt("myfile.txt", a)
        c = np.loadtxt("myfile.txt")
        c
```

```
Out[4]: array([[0.92311745, 0.61963683, 0.99603435],
               [0.46528023, 0.2557813 , 0.95800069],
               [0.22207387, 0.99658931, 0.43508635],
               ...,
               [0.54641681, 0.61120011, 0.78739067],
               [0.49190005, 0.30582528, 0.89564933],
               [0.86677168, 0.60828137, 0.57487395]])
```

```
In [6]: d = np.loadtxt("file.csv", delimiter=",", skiprows=1) # csv file
        d
```

```
Out[6]: array([[0.92311745, 0.61963683, 0.99603435],
               [0.46528023, 0.2557813 , 0.95800069],
               [0.22207387, 0.99658931, 0.43508635],
               ...,
               [0.54641681, 0.61120011, 0.78739067],
               [0.49190005, 0.30582528, 0.89564933],
               [0.86677168, 0.60828137, 0.57487395]])
```

```
In [9]: np.savetxt("myfile.csv", c, delimiter=",")
```


Some array methods

```

1  >>> import numpy as np
2  >>> b = np.array([[1, 2, 3],[4, 5, 6]])
3  >>> b
4  array([[1, 2, 3],
5         [4, 5, 6]])
6  >>> b.mean()
7  3.5
8  >>> b.mean(axis = 0)
9  array([2.5, 3.5, 4.5])
10 >>> b.mean(axis = 1)
11 array([2., 5.])
12 >>> b.std()
13 1.707825127659933
14 >>> b.std(axis = 1)
15 array([0.81649658, 0.81649658])
16 >>> b.max(), b.min()
17 (6, 1)
18 >>> b.reshape(1,6)
19 array([[1, 2, 3, 4, 5, 6]])
20 >>> b.T
21 array([[1, 4],
22        [2, 5],
23        [3, 6]])
24 >>> b.sum()
25 21

```

Note: `.T` (or `.transpose()`) inverts the shape of the array independently of its dimension!
E.g. shape (2,3,4) => `.T` => shape (4,3,2)

Try out `.cumsum()` and `.sort()`

b

([[1, 2, 3],
[4, 5, 6]])

axis = 0 => average of each column

b

([[1, 2, 3],
[4, 5, 6]])

axis = 1 => average of each row

Note: *b* has 6 elements. We can only reshape *b* if the product of the shape numbers (dimensions along each axis) is also 6 (here, 1 x 6 = 6).

E.g. Try `b.reshape(1,5)`



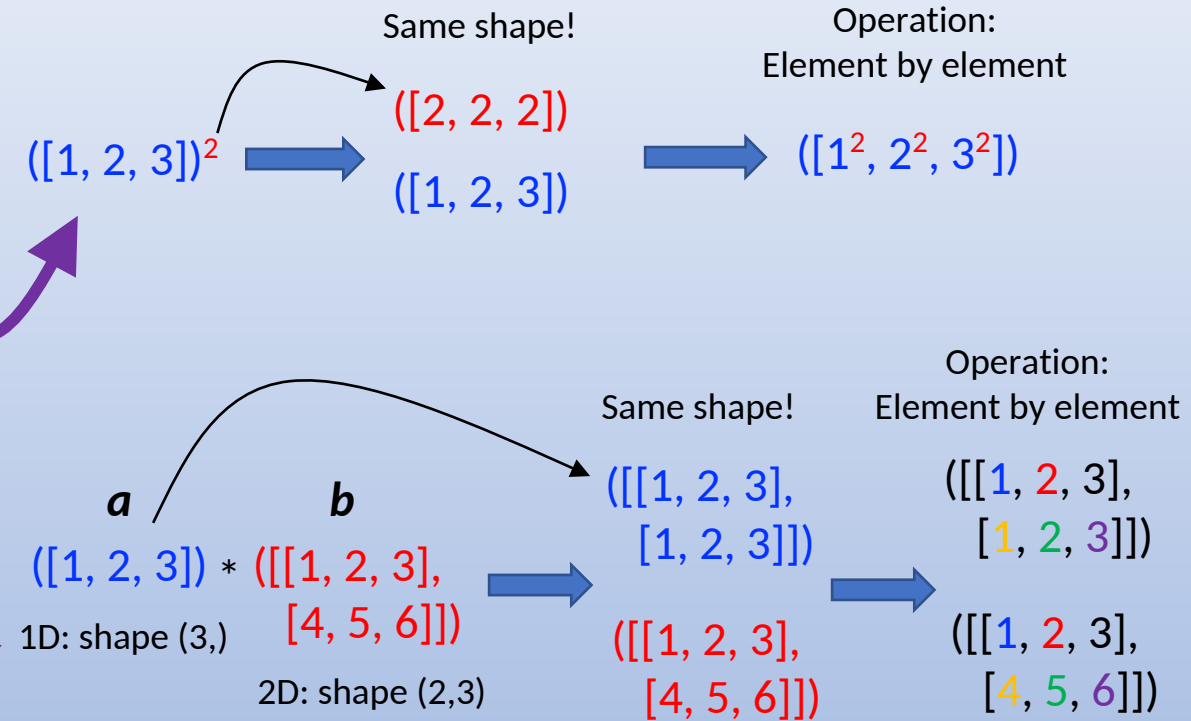
Some array methods

```

1  >>> import numpy as np
2  >>> a = np.array([1,2,3])
3  >>> b = np.array([[1,2,3],[4,5,6]])
4  >>> b
5  array([[1, 2, 3],
6         [4, 5, 6]])
7  >>> a ** 2
8  array([1, 4, 9])
9  >>> 2 ** a
10 array([2, 4, 8])
11 >>> a - 1
12 array([0, 1, 2])
13 >>> a * b
14 array([[ 1,  4,  9],
15        [ 4, 10, 18]])
16 >>> a ** b
17 array([[ 1,  4, 27],
18        [ 1, 32, 729]])
19 >>> a + b
20 array([[2, 4, 6],
21        [5, 7, 9]])

```

Broadcasting



Note: Sometimes one array cannot be broadcast to have the shape of the other array: **Error!**

E.g., Try `array([1,2]) * array([1,2,3])`



Exercise

1) 2D array of shape (3,3) with all odd numbers between 20 and 38.

```
In [2]: np.arange(21, 38, 2).reshape(3,3)
Out[2]: array([[21, 23, 25],
               [27, 29, 31],
               [33, 35, 37]])
```

2) Create an array of shape (2,3,4) with integer random numbers between 3 and 5.

```
In [4]: np.random.randint(3, 6, size = (2,3,4))
Out[4]: array([[[5, 3, 4, 5],
                [4, 4, 3, 4],
                [4, 3, 4, 4]],
               [[5, 5, 4, 3],
                [3, 3, 3, 4],
                [3, 5, 5, 5]])])
```

3) Use the `np.random.normal()` function using any average/variance to create 3 different arrays with shape (4,1) and combine them column-wise to generate an array of shape (4,3).

Tip: check the `np.c_[]` function.

```
a = np.random.normal(size = 4).reshape(4,1)
b = np.random.normal(size = 4).reshape(4,1)
c = np.random.normal(size = 4).reshape(4,1)
np.c_[a, b, c]
array([[ 1.45747619,  0.65092474, -0.87365949],
       [ 1.78365878,  1.01880445, -1.25639063],
       [ 0.34060236, -0.37920261,  0.62521058],
       [ 0.56423595,  0.73355626,  2.47309436]])
```

```
In [14]: a = np.random.normal(size = 4).reshape(4,1)
         b = np.random.normal(size = 4).reshape(4,1)
         c = np.random.normal(size = 4).reshape(4,1)
         d = np.c_[a, b, c]
         d.mean(axis = 0), d.std(axis = 0) # or np.mean()
Out[14]: (array([ 0.40053918, -0.46262531,  0.102725  ]),
          array([1.18849932, 0.74008606, 1.1196697 ]))
```

4) Repeat the previous exercise, each time using individual arrays with 4, 400 and 40000 elements and calculate the average and standard deviation of each column of the final combined array. Do you see any **trend**?

400: (array([0.10088182, -0.01910432, 0.02557122]),
array([0.97849846, 1.0412029 , 0.963044]))

4E4: (array([0.00274767, -0.00148074, -0.01154878]),
array([0.99674189, 1.00053144, 1.00194965]))



Average/std converge to 0/1

Exercise

- 5) Generate a simple 1D array of shape (1000000,) with random numbers in the range [0,1].
Use a slice of this array to find out how many elements have values smaller than 0.1.

```
In [2]: a = np.random.random(size = 1000000)
a < 0.1 # This generates a mask
Out[2]: array([False, False, False, ..., False, False, False])
```

```
In [3]: mask = a < 0.1
```

```
In [4]: a[mask]
Out[4]: array([0.02932379, 0.06917394, 0.07276824, ..., 0.05980439, 0.02901995,
              0.09303711])
```

```
In [5]: a[mask].size
Out[5]: 99835
```

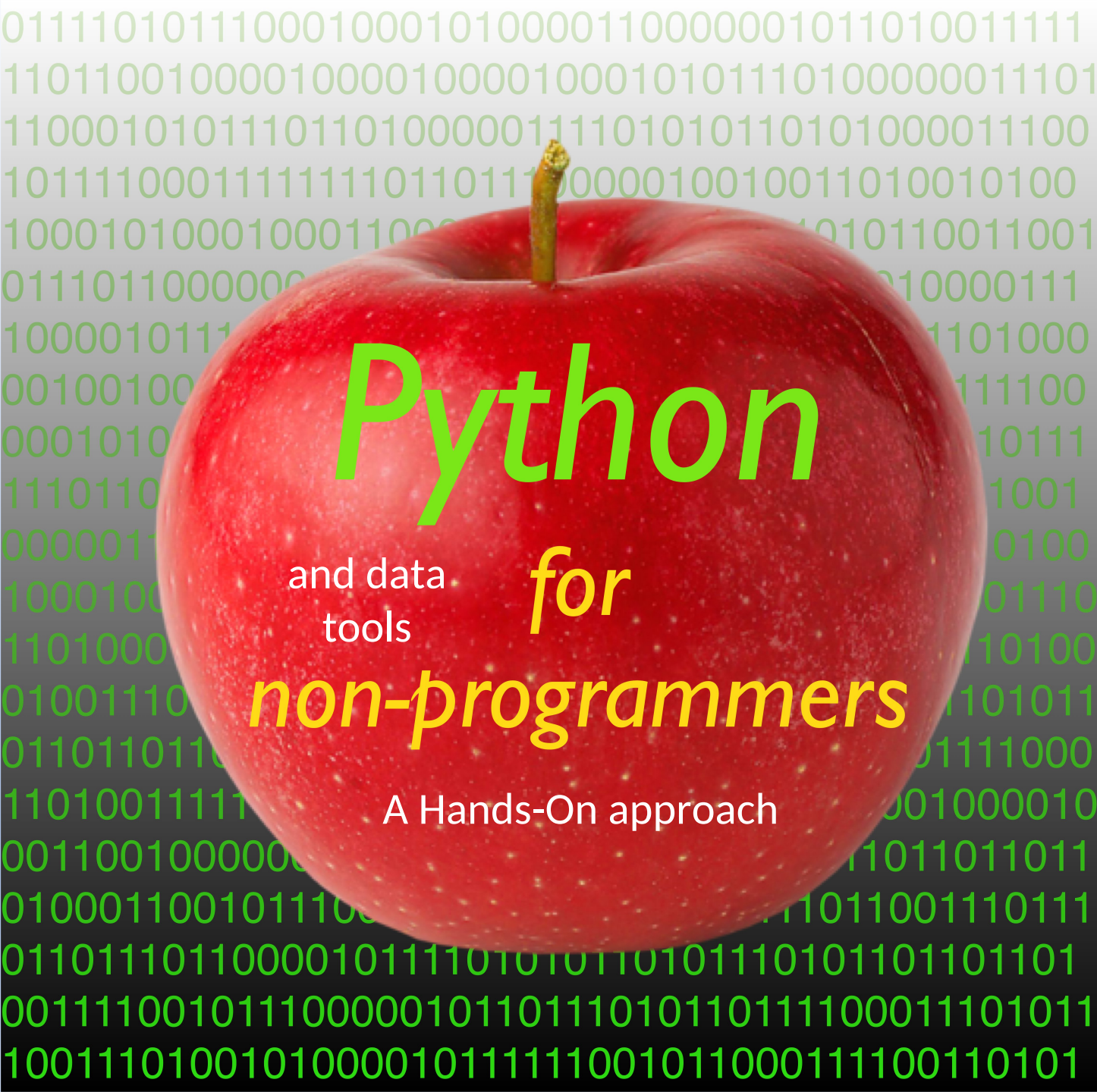
Because of the randomness, you will not get exactly 99835...

In practice, we solve the slicing problem in one single line:

```
In [6]: # alternative
a[a < 0.1].size
Out[6]: 99835
```

For higher dimensions (2D, 3D, etc), one can check such conditions for different axes.

E.g., If **b** is a 2D array, the condition **b[:, 1] < 0.1** is checked only on the elements of the second column. Therefore, **b[b[:, 1] < 0.1]** would give all rows whose elements of the second column are smaller than 0.1.



UNIVERSITÄT
BAYREUTH

Delivered by

Dr. Rodrigo Albuquerque

(AG Ruckdäschel)

&

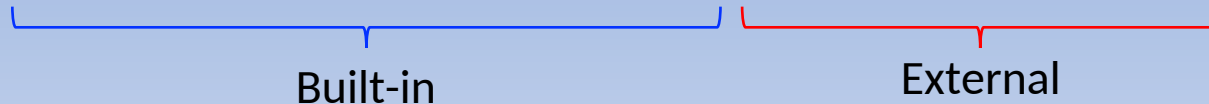
AG Künneth

Part of the project
“digitale Methoden”
(bidt-digikollegs)



- Brief overview of Python
- Installation and graphical interfaces
- Data types and operators
- Objects, modules and classes
- Strings
- Lists, sets and tuples
- Dictionary
- Arrays and **matrices**
- Loops (for, while)
- If statements
- Functions, pandas
- Files
- Plotting graphics

In Python, data can be grouped using **DATA STRUCTURES:**
lists, sets, tuples, dictionaries, arrays, matrices, etc



Matrix: 2D container (same data type) with the well-known matrix properties

```
In [2]: a = [[1,2],[3,4]]
        m = np.matrix(a)
        m
```

```
Out[2]: matrix([[1, 2],
               [3, 4]])
```

```
In [3]: np.matrix("1 2; 3 4")
```

```
Out[3]: matrix([[1, 2],
               [3, 4]])
```

```
In [4]: m * m # Equivalent: m @ m
```

```
Out[4]: matrix([[ 7, 10],
               [15, 22]])
```

Matrix multiplication:

$$\begin{bmatrix} a & b \\ c & d \end{bmatrix} \begin{bmatrix} e & f \\ g & h \end{bmatrix} = \begin{bmatrix} ae + bg & af + bh \\ ce + dg & cf + dh \end{bmatrix}$$

```
In [5]: np.linalg.det(m)
```

```
Out[5]: -2.000000000000000004
```

Determinant

$$\text{Det} \begin{bmatrix} a & b \\ c & d \end{bmatrix} = ad - bc$$

```
In [6]: np.linalg.inv(m)
```

```
Out[6]: matrix([[ -2. ,  1. ],
               [ 1.5, -0.5]])
```

$$\begin{matrix} m & m^{-1} & \text{Identity} \\ \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} & \begin{bmatrix} -2 & 1 \\ 1.5 & -0.5 \end{bmatrix} & = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \end{matrix}$$

```
In [7]: np.linalg.eig(m) # val, vec = np.linalg.eig(m)
```

```
Out[7]: (array([-0.37228132,  5.37228132]),
        matrix([[ -0.82456484, -0.41597356],
               [ 0.56576746, -0.90937671]]))
```

$$\begin{matrix} \text{Matrix} & \text{Eigenvector 1} & \text{Eigenvalue 1} & \text{Eigenvector 1} \\ \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} & \begin{bmatrix} -0.8246 \\ 0.5658 \end{bmatrix} & = -0.3723 & \begin{bmatrix} -0.8246 \\ 0.5658 \end{bmatrix} \end{matrix}$$

Covariance matrix

```
In [16]: np.cov(m)
```

```
Out[16]: array([[0.5, 0.5],
               [0.5, 0.5]])
```

```
In [11]: m.T
```

```
Out[11]: matrix([[1, 3],
               [2, 4]])
```



Exercises

Use the matrix m of the previous slide and:

- 1) Calculate $m \cdot m^{-1}$
- 2) Check that $m \cdot \text{eigenvector}_{(i)} = \text{eigenvalue}_{(i)} \cdot \text{eigenvector}_{(i)}$ for both pairs of eigenvector/eigenvalue
- 3) In the matrix a given below, the columns represent variables and the rows represent samples).
 - 3.1) Subtract each column of a by its average and then divide the column by its standard deviation to get matrix b .
 - 3.2) Calculate the eigenvalues of the covariance matrix of b . Knowing that each eigenvalue represents the amount of information (or variance) related to the corresponding eigenvector (here called **principal component**), calculate how much percent of the information is carried by the first, second and third principal components.

Do you see any trend? Tip: Use the option “rowvar = False” inside the function np.cov().

```
a = np.matrix([[5.1, 3.8, 1.9, 0.4],
               [4.9, 3.6, 1.4, 0.1],
               [6.5, 3. , 5.2, 2. ],
               [6.4, 2.7, 5.3, 1.9],
               [6.5, 3. , 5.8, 2.2],
               [5. , 3.6, 1.4, 0.2],
               [6.1, 2.8, 4. , 1.3],
               [5.5, 2.5, 4. , 1.3],
               [5.1, 3.5, 1.4, 0.3],
               [5. , 2. , 3.5, 1. ]])
```

```
In [2]: a = a - a.mean(axis = 0)

In [3]: b = a / a.std(axis = 0)

In [4]: cov_b = np.cov(b, rowvar = False)

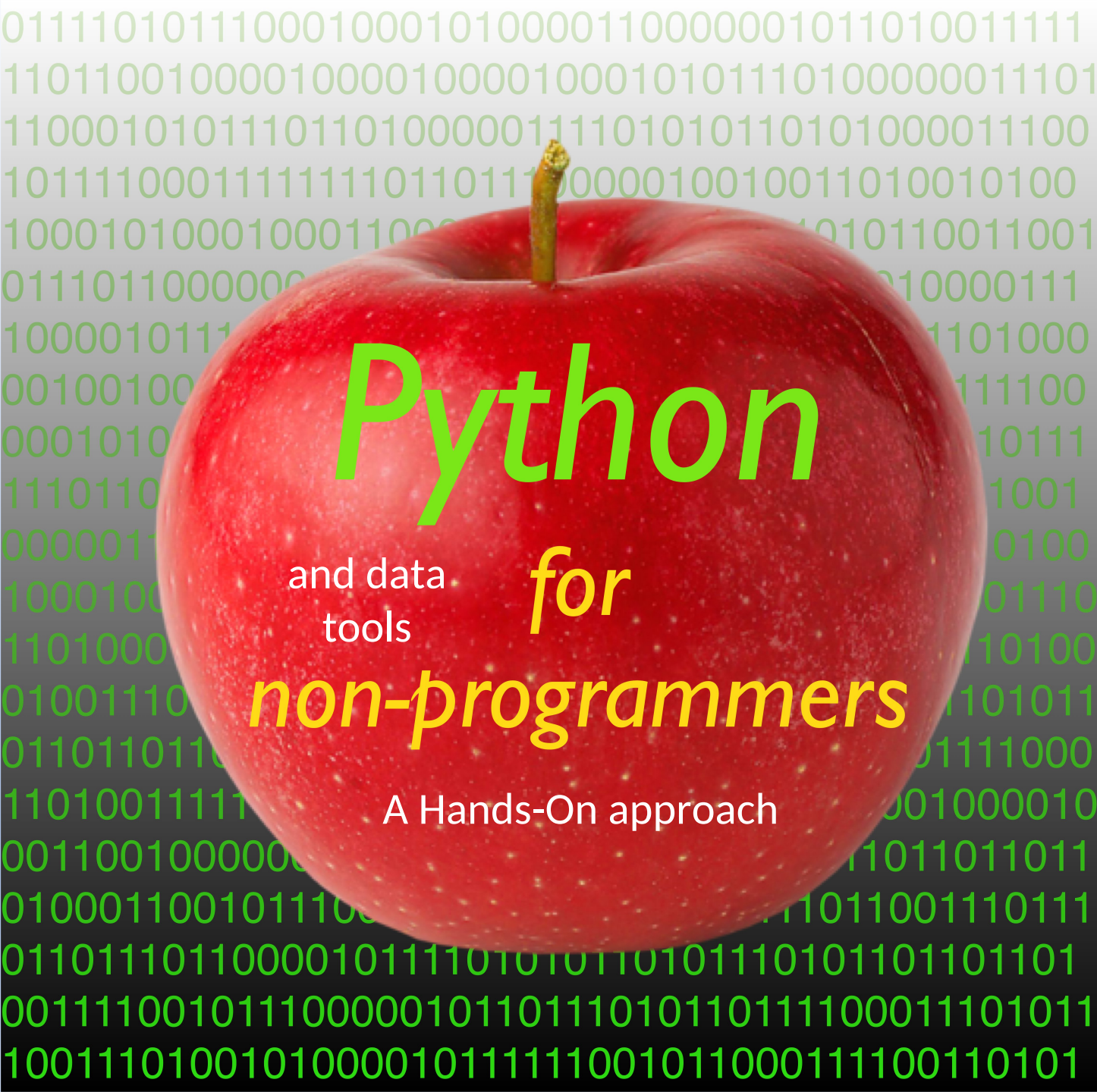
In [5]: val, vec = np.linalg.eig(cov_b)

In [6]: val * 100 / np.sum(val)

Out[6]: array([8.10461919e+01, 1.80852033e+01, 8.00354824e-01, 6.82500133e-02])
```

81 % => 18 % => 0.8 %





UNIVERSITÄT
BAYREUTH

Delivered by

Dr. Rodrigo Albuquerque

(AG Ruckdäschel)

&

AG Künneth

Part of the project
“digitale Methoden”
(bidt-digikollegs)



- Brief overview of Python
- Installation and graphical interfaces
- Data types and operators
- Objects, modules and classes
- Strings
- Lists, sets and tuples
- Dictionary
- Arrays and matrices
- Loops (for, while)
- If statements
- Functions, pandas
- Files
- Plotting graphics

`for` blocks are used to iterate sequentially over elements a finite number of times. The internal lines (tasks) are indented. In Python, the `for` block ends when a non-indented line is found.

```
In [1]: mylist = ['red', 'green', 'blue']
        for color in mylist:
            print(color)
```

```
red
green
blue
```

```
In [2]: for color in mylist:
        print(f"The color is {color}.")
```

```
The color is red.
The color is green.
The color is blue.
```

```
In [3]: for number in range(3):
        print(number)
```

```
0
1
2
```

```
In [4]: for n1 in range(2):
        for n2 in range(2):
            print(f'{n1}+{n2}={n1+n2}')
```

```
0+0=0
0+1=1
1+0=1
1+1=2
```

```
In [8]: a = np.empty((3,3))
        for i in range(3):
            for j in range(3):
                a[i,j] = i + j
        a
```

```
Out[8]: array([[0., 1., 2.],
               [1., 2., 3.],
               [2., 3., 4.]])
```

```
In [9]: [k for k in range(5)]
```

```
Out[9]: [0, 1, 2, 3, 4]
```

```
In [16]: for letter in 'Hello!':
          print(letter)
```

```
H
e
l
l
o
!
```

```
In [25]: a = {'H':1, 'He':2, 'Li':3}
        for key in a:
            print(key)
```

```
H
He
Li
```

```
In [26]: for key in a:
          print(key, a[key])
```

```
H 1
He 2
Li 3
```

For blocks with some useful commands

```
In [10]: colors = ['red', 'green', 'blue']  
gifts = ['car', 'bike', 'plane']  
for gift, color in zip(gifts, colors):  
    print(f'The {gift} is {color}.')
```

```
The car is red.  
The bike is green.  
The plane is blue.
```

```
In [11]: for i, gift in enumerate(gifts):  
         print(i, gift)
```

```
0 car  
1 bike  
2 plane
```

```
In [14]: for i, (color, gift) in enumerate(zip(colors, gifts)):  
         print(i, color, gift)
```

```
0 red car  
1 green bike  
2 blue plane
```

```
In [15]: for i, (color, gift) in enumerate(zip(colors, gifts)):  
         print(f'Yesterday I saw {i+5} {color} {gift}s.')
```

```
Yesterday I saw 5 red cars.  
Yesterday I saw 6 green bikes.  
Yesterday I saw 7 blue planes.
```

```
In [24]: sentence = 'My name is john'  
for word in sentence.split():  
    print(word)
```

```
My  
name  
is  
john
```

Note 1: The command `break` can be used to leave the `for` block before fully completed (to be seen later on...)

```
In [25]: for letter in 'Hello!':  
         print(letter)  
         break
```

```
H
```

Note 2: The command `continue` can be used to go to the next iteration without executing the lines below.

```
In [30]: for letter in 'Wow':  
         print(letter)  
         continue  
         print('Oh No!!!')
```

```
W  
o  
w
```



Exercises

- 1) Generate 3 different lists of anything and iterate over their elements inside a `for` loop. Use both the `zip` and `enumerate` commands.

```
In [2]: things = ['car', 'bike', 'plane']
        colors = ['red', 'green', 'blue']
        adjectives = ['nice', 'beautiful', 'heavy']
        for i, (thing,color,adjective) in enumerate(zip(things,colors,adjectives)):
            print(f'She has {i+2} {adjective} {color} {thing}s!')
```

She has 2 nice red cars!
She has 3 beautiful green bikes!
She has 4 heavy blue planes!

- 2) Fill an empty dictionary with information coming from two lists (defined by you) using a `for` loop.

```
In [8]: mydict = {}
        keys = ['car', 'bike', 'plane']
        values = ['red', 'green', 'blue']
        for key, value in zip(keys, values):
            mydict[key] = value
```

```
In [10]: mydict = {}
         keys = ['car', 'bike', 'plane']
         values = ['red', 'green', 'blue']
         for i in range(len(keys)):
             mydict[keys[i]] = values[i]
```

```
In [11]: mydict
Out[11]: {'car': 'red', 'bike': 'green', 'plane': 'blue'}
```

- 3) Use nested `for` loops to generate a 3D array named `a` of shape (3,2,4) with elements given by $a[i, j, k] = (i + j + k)^2$

```
In [20]: a = np.empty((3,2,4))
         for i in range(3):
             for j in range(2):
                 for k in range(4):
                     a[i,j,k] = (i + j + k)**2

In [21]: a
Out[21]: array([[[ 0.,  1.,  4.,  9.],
                  [ 1.,  4.,  9., 16.]],
                [[ 1.,  4.,  9., 16.],
                  [ 4.,  9., 16., 25.]],
                [[ 4.,  9., 16., 25.],
                  [ 9., 16., 25., 36.]])
```

Make it more automatic in case you need to change the shape of `a`, so that all loops are automatically modified

```
In [22]: a = np.empty((3,2,4)) # only change line 1 if needed
         for i in range(a.shape[0]):
             for j in range(a.shape[1]):
                 for k in range(a.shape[2]):
                     a[i,j,k] = (i + j + k)**2
```

Note: `>>> a.shape`
`(3, 2, 4)`





UNIVERSITÄT
BAYREUTH

Delivered by

Dr. Rodrigo Albuquerque

(AG Ruckdäschel)

&

AG Künneth

Part of the project
“digitale Methoden”
(bidt-digikollegs)



- Brief overview of Python
- Installation and graphical interfaces
- Data types and operators
- Objects, modules and classes
- Strings
- Lists, sets and tuples
- Dictionary
- Arrays and matrices
- **Loops** (for, **while**)
- If statements
- Functions, pandas
- Files
- Plotting graphics

LOOP (WHILE BLOCKS)

The **while** loop is used to execute commands as long as a condition is satisfied (or True). The commands inside the **while** block must be indented. The commands **break** and **continue** can also be used here similarly to the **for** block.

```
In [1]: i = 0
while i < 3:
    print('i is smaller than 3.')
    i += 1 # increment i in 1 unit

i is smaller than 3.
i is smaller than 3.
i is smaller than 3.
```

Note: The last time the condition `i < 3` was tested, `i` was still 2. Therefore, both command lines were executed one last time, from where `i` became 3 (**tricky!**)

```
In [2]: i
Out[2]: 3
```

```
In [3]: i = 0
while i < 3:
    print(f'{i} is smaller than 3.')
    i += 1 # increment i in 1 unit

0 is smaller than 3.
1 is smaller than 3.
2 is smaller than 3.
```

```
In [6]: # Note: In the last iteration, len(a) = 2 (still < 3)
# then, the 2 lines are executed one last time,
# from where a is updated and len(a) becomes 3 (tricky!)
a = []
while len(a) < 3:
    print(f'len(a) = {len(a)}')
    a.append('hi')

len(a) = 0
len(a) = 1
len(a) = 2
```

```
In [7]: a
Out[7]: ['hi', 'hi', 'hi']
```

```
In [8]: len(a)
Out[8]: 3
```

```
In [10]: i = 0
while i < 3:
    n = input('Digit any number: ')
    print(f'You wrote {n}')
    i += 1 # delete this line to get an infinite loop

Digit any number: 123
You wrote 123
Digit any number: 234
You wrote 234
Digit any number: 345
You wrote 345
```

Command **input**

→ **Infinite loop**
(also works with "**while True:**")



Exercises

- 1) Use `while` to print all odd numbers smaller than 12.
Update the number inside the `while` loop

```
In [2]: a = 1
        while a < 12:
            print(a)
            a += 2
```

1
3
5
7
9
11

- 2) Use the string 'Hi! How are you? My name is John. And yours?' and iterate over the words inside a `while` block to print only the first 6 words.

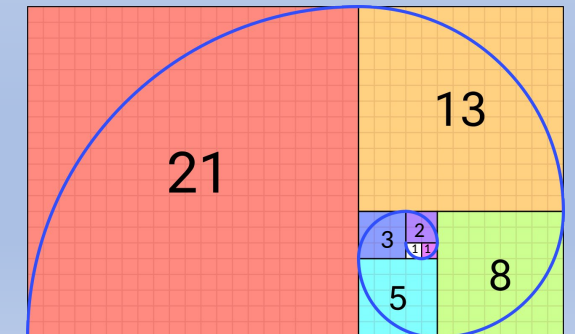
```
In [3]: a = 'Hi! How are you? My name is John. And yours?'
        b = a.split()
        i = 0 # index of the first word in the list b
        while i < 6:
            print(b[i])
            i += 1
```

Hi!
How
are
you?
My
name

```
In [5]: a, b = 0, 1
        while a < 100:
            print(a)
            a, b = b, a+b
```

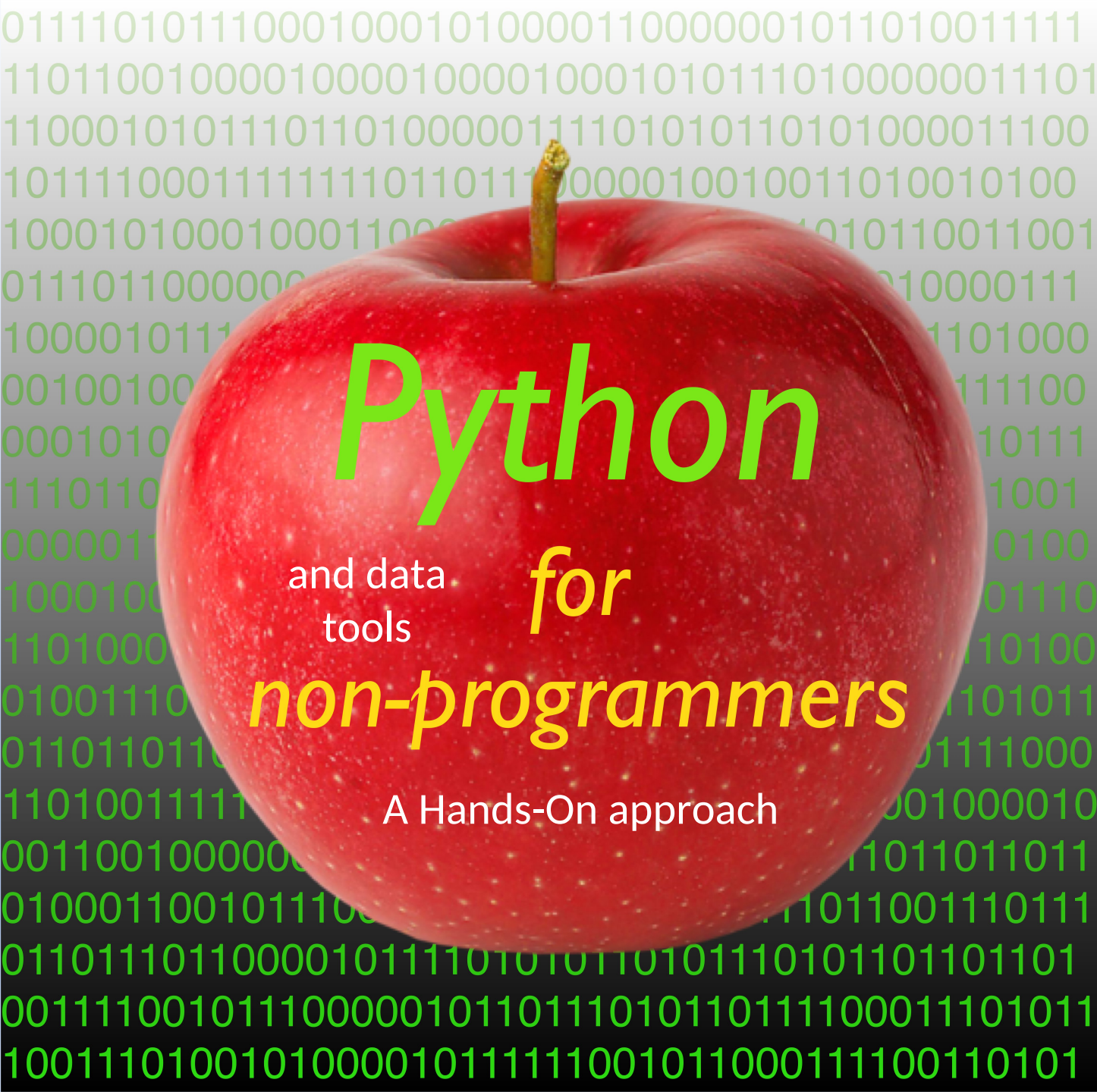
0
1
1
2
3
5
8
13
21
34
55
89

Challenge: Use `while` to grow (and print) the Fibonacci sequence up to 100. This series is a sequence of numbers where any number is the sum of the previous two. Adopt $a = 0$ and $b = 1$ for the first two numbers. **Tip:** Update a and b in the same line.



Length of each square

Figure source: Wikipedia: Fibonacci sequence



UNIVERSITÄT
BAYREUTH

Delivered by

Dr. Rodrigo Albuquerque

(AG Ruckdäschel)

&

AG Künneth

Part of the project
“digitale Methoden”
(bidt-digikollegs)



- Brief overview of Python
- Installation and graphical interfaces
- Data types and operators
- Objects, modules and classes
- Strings
- Lists, sets and tuples
- Dictionary
- Arrays and matrices
- Loops (for, while)
- If statements
- Functions, pandas
- Files
- Plotting graphics

IF STATEMENTS

The **if** block checks a condition and executes the commands below (and indented to the right) only if the condition is **True**

```
In [2]: a = 1
        if a == 1:
            print('a is equal to 1')

a is equal to 1
```

Comparison
operator

(Check/revise other comparison operators!)

"=" versus "=="

Assignment
operator

(Check/revise other assignment operators!)

```
In [16]: a = 2
        if a != 0:
            print("a is different from 0!")

a is different from 0!
```

!= means "different from" or "not equal to"
(Comparison operator)

```
In [5]: a = b = 1
        if a == 1 and b == 2:
            print("Condition achieved!")
```

and logical
operator

```
In [6]: a = b = 1
        if a == 1 or b == 2:
            print("Condition achieved!")

Condition achieved!
```

or logical
operator

```
In [7]: a, b = 0, 1
        if a > b:
            print("a is larger than b")
        else:
            print("a is NOT larger than b")

a is NOT larger than b
```

else command

Multiple (self-excluding) conditions

```
In [9]: a = 3
        if a == 1:
            print("a is equal to 1")
        elif a == 2:
            print("a is equal to 2")
        elif a == 3:
            print("a is equal to 3")
        else:
            print("a is NOT equal to 1, 2 or 3")

a is equal to 3
```

The commands of the first condition
that is evaluated as True are executed

Note: You don't need
to use **else** in the end!

```
In [10]: a = 3
         if a == 1:
             print("a is equal to 1")
         elif a > 2:
             print("a is larger than 2")
         elif a == 3:
             print("a is equal to 3")

a is larger than 2
```



Checking expressions + multiple command lines:

```
In [13]: a = 2
         if 2*a + 1 == 5:
             print("I forgot what to do!")
             print(f"I remembered! a**2 = {a**2}")

I forgot what to do!
I remembered! a**2 = 4
```

Above, $2*a + 1 == 5$ is evaluated as **True**, causing the indented lines to be executed. To invert this behavior (i.e., to execute commands if the expression is **False**, just add a **not** (logical operator) before the expression.



```
In [14]: a = 2
         if not 2*a + 1 == 10:
             print("I forgot what to do!")
             print(f"I remembered! a**2 = {a**2}")

I forgot what to do!
I remembered! a**2 = 4
```

Note: if $\text{not } 2*a + 1 == 10$ in fact means if " $2*a + 1 == 10$ " is not True, execute the lines below.

Alternative: if $2*a + 1 != 10$

Nested if conditions

```
In [1]: size = 'huge'
       color = 'gray'
       if size == 'huge':
           if color == 'blue':
               print('This is a blue whale!')
           elif color == 'gray':
               print('This is a dinosaur!')

This is a dinosaur!
```

Searching inside lists or strings

```
In [2]: mining = ['Pt', 'Ag', 'Au']
       if 'Au' in mining:
           print('We found Gold!')

We found Gold!
```

```
In [4]: 'Au' in mining
Out[4]: True
```

```
In [5]: mining = 'Pt Ag Au'
       if 'Au' in mining:
           print('We found Gold!')

We found Gold!
```


Verifying the truth value of objects

```
In [1]: dog = True
        if dog:
            print('I found a dog!')

I found a dog!
```

```
In [2]: a = 1
        b = [1,2,3]
        if a:
            print('Variable a is True')
        if b:
            print('Variable b is True')

Variable a is True
Variable b is True
```

ERROR!

```
In [3]: if c:
        print('Variable c exists')

-----
NameError                                Traceback (most recent call last)
Input In [3], in <cell line: 1>()
----> 1 if c:
      2     print('Variable c exists')

NameError: name 'c' is not defined
```

Note: True and False can also be represented by 1 and 0, respectively:

```
In [6]: a, b = 0, []
        if a: # a = 0 (False)
            print('Variable a is True')
        if b: # b = [] (False)
            print('Variable b is True')
```

In [7]: bool(a), bool(b)
Out[7]: (False, False)

Objects considered False:

- Empty sequences and collections: E.g., [], ' ', (), {}, etc.
- Zero of any numeric type (e.g. 0, 0.0, 0j)
- None and False

Otherwise the objects are considered True

Multiple logical operators (and, or, not)

```
In [1]: flight = False
bus     = True
hotel   = False
airbnb  = True
if (flight or bus) and (hotel or airbnb):
    print("Transport + Accomodation!")

Transport + Accomodation!
```



equivalent
(lengthy!)

```
In [2]: flight = False
bus     = True
hotel   = False
airbnb  = True
if (flight == True or bus == True) and (hotel == True or airbnb == True):
    print("Transport + Accomodation!")

Transport + Accomodation!
```

```
In [3]: age = 80
has_family = False
if age > 85 and has_family:
    print("Extra care, extra family bed")
elif age > 85 and not has_family:
    print("Extra care, no extra family bed")
elif age < 85 and has_family:
    print("Normal care, extra family bed")
else:
    print("Normal care, no extra family bed")

Normal care, no extra family bed
```

Exercises

Extra exercises, e.g.: <https://www.w3resource.com/python-exercises/python-conditional-statements-and-loop-exercises.php>

1) Use an **if** block and a **for** loop to generate two separate lists (named strings and integers) from the list **a**, each of them containing elements of the same data type. **a** = [1, 'Hi', 132, 447, 'Hello!', 'Nice'].

```
In [2]: a = [1, 'Hi', 132, 447, 'Hello!', 'Nice' ]
        integers, strings = [], []
        for item in a:
            if type(item) == int:
                integers.append(item)
            elif type(item) == str:
                strings.append(item)
```

```
In [3]: strings
Out[3]: ['Hi', 'Hello!', 'Nice']

In [4]: integers
Out[4]: [1, 132, 447]
```

2) Use an **if** clause and a **for** loop to print the numbers that are multiples of 7 (e.g. 7, 14, 21,...) in the range 0 to 50. **Tip**: Use the modulus operator (%).

```
In [5]: for number in range(50):
        if number % 7 == 0:
            print(number)

0
7
14
21
28
35
42
49
```

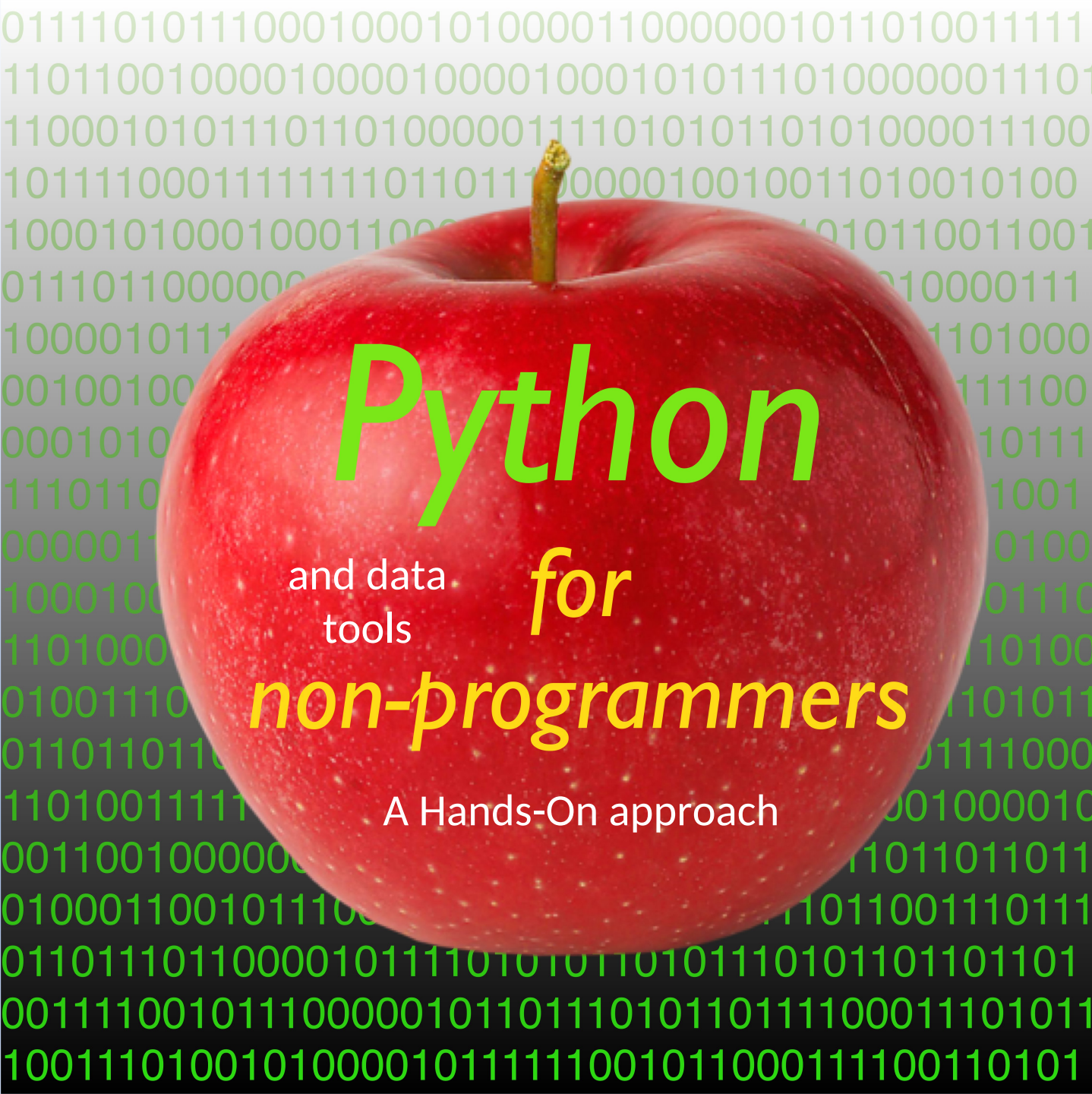
3) Print all possible tuples of the type (a,b,c), where a, b and c are in the range 0-10 and adopting the restrictions: a + b + c is smaller than 20, c is larger than or equal to a + b, and a > 7.

Tip: Use nested **for** loops and **if**.

```
In [1]: for a in range(11):
        for b in range(11):
            for c in range(11):
                if a + b + c < 20 and a + b <= c and a > 7:
                    print(a,b,c)

8 0 8
8 0 9
8 0 10
8 1 9
8 1 10
9 0 9
9 0 10
```





UNIVERSITÄT
BAYREUTH

Delivered by

Dr. Rodrigo Albuquerque

(AG Ruckdäschel)

&

AG Künneth

Part of the project
“digitale Methoden”
(bidt-digikollegs)



- Brief overview of Python
- Installation and graphical interfaces
- Data types and operators
- Objects, modules and classes
- Strings
- Lists, sets and tuples
- Dictionary
- Arrays and matrices
- Loops (for, while)
- If statements
- Functions, **pandas**
- Files
- Plotting graphics

pandas is a popular Python library used for working with data in tables, similar to spreadsheets in Excel. It helps you:

- Load data from files (CSV, Excel, databases, etc.)
- Organize data into rows and columns
- Clean messy data
- Analyze data to find patterns and answers
- Visualize data (often together with other libraries like matplotlib)

The main tools in pandas are:

Series → a one-dimensional labeled list (like a single column of data)

DataFrame → a two-dimensional table (like a full spreadsheet)

Important: Install pandas before importing it!

- In jupyter notebook, execute the command *!pip install pandas matplotlib*
- In Anaconda Powershell, execute the command *conda install pandas matplotlib*

Series creation

```
import pandas as pd # !pip install pandas OR conda install pandas
```

```
s = pd.Series([11,22,33,44]) # you can add: dtype = float , index = [new indices]..
```

```
s # automatic index ID (=0,1,2,3) were added!
```

```
0    11
1    22
2    33
3    44
dtype: int64
```

```
type(s)
```

```
pandas.core.series.Series
```

```
s.shape # or use len(s)
```

```
(4,)
```

A **pandas Series** is a simple sequence of elements displayed as one column (+ index ID)

Change row name/ID (or index)

```
# Access the 'index' attribute and change it to any unique ID labels!  
s.index = ['trial 1', 'trial 2', 'trial 3', 'trial 4']
```



```
s  
  
trial 1    11  
trial 2    22  
trial 3    33  
trial 4    44  
dtype: int64
```

Access an element by its row name/ID

```
s['trial 1'] # check the element at unique ID 'trial 1'
```

```
11
```

Redefine an element

```
s['trial 1'] = 20 # pandas objects are mutable, we can change their elements!  
s
```

```
trial 1    20  
trial 2    22  
trial 3    33  
trial 4    44  
dtype: int64
```


Get general statistics

```
s.describe() # check general statistics of this Series
```

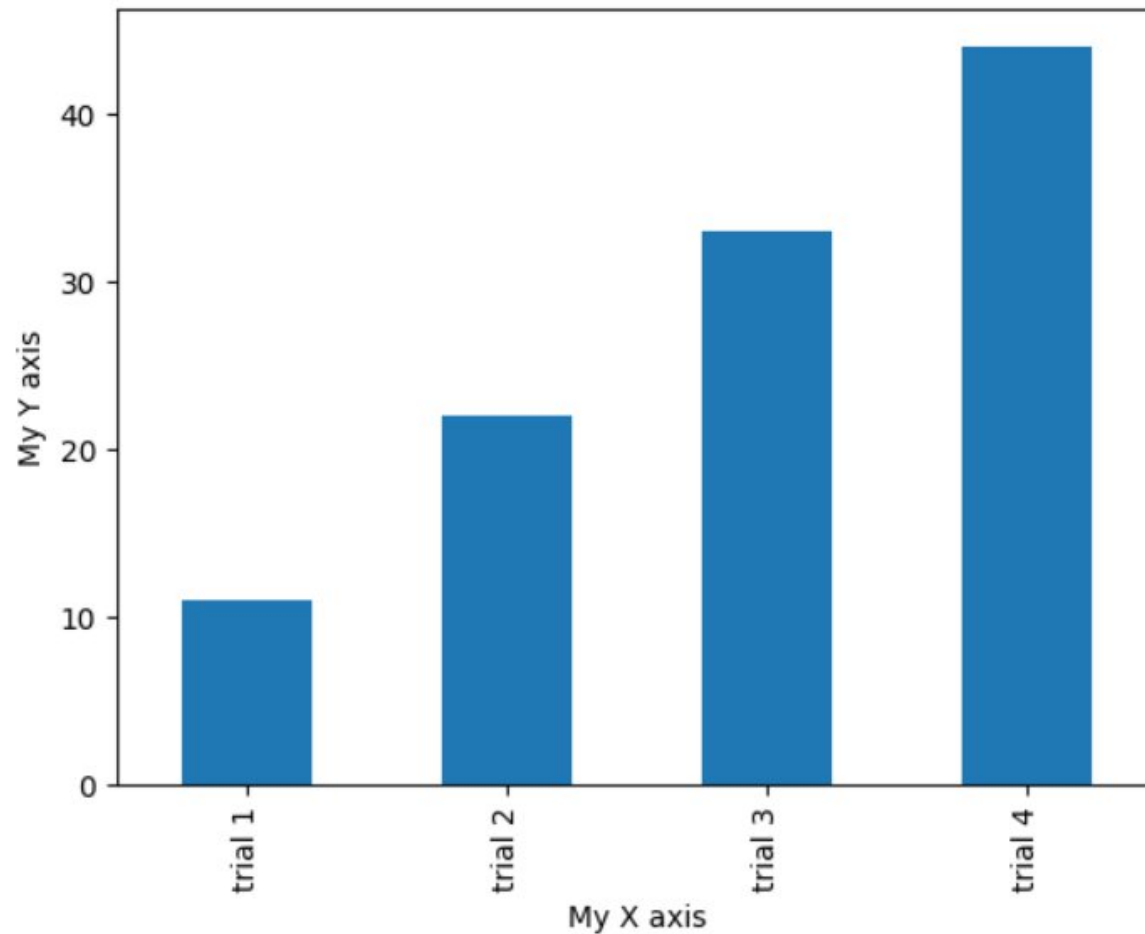
```
count      4.000000  
mean       27.500000  
std        14.200939  
min        11.000000  
25%        19.250000  
50%        27.500000  
75%        35.750000  
max        44.000000  
dtype: float64
```

```
s.mean(), s.std() # or check individual statistics
```

```
(27.5, 14.200938936093861)
```

Make graphs

```
# plot the data inside the Series s. Press Shift + TAB to access the docstring for more parameters  
# Check the docstring also via the command help(s.plot)  
s.plot(kind = 'bar', xlabel = 'My X axis', ylabel = 'My Y axis');
```



Apply restrictions via masks

```
s > 25 # Which elements are larger than 25?
```

```
trial 1    False
trial 2    False
trial 3     True
trial 4     True
dtype: bool
```

```
mask = s > 25 # save this "mask" to a new object named mask
mask # check the content of mask
```

```
0    False
1    False
2     True
3     True
dtype: bool
```

```
s[mask] # Use the mask to show only desired elements
```

```
trial 3    33
trial 4    44
dtype: int64
```

Save to / load external files

```
s.to_excel('test.xlsx') # save the content of s to an excel file
# Note: digit s.to and press the TAB key (in Jupyter) to see other
# file types supported by pandas
```

```
# read a new excel file
s_new = pd.read_excel('test.xlsx')
s_new # DataFrame!
```

	Unnamed: 0	0
0	trial 1	11
1	trial 2	22
2	trial 3	33
3	trial 4	44

s_new is not a Series anymore.
Check its type!

Tip: You can read all sheets of a given excel file by adding the parameter `sheet_name = None`:

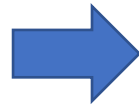
```
d1 = pd.read_excel('test.xlsx', sheet_name = None)
```

d1 is now a dictionary containing pandas objects!

DataFrame creation

```
# Each key of the dictionary below is a column name and its value are the column elements
data_dict = {'Speed': [2, 3, 4, 5, 6], 'Weight': [25, 30, 35, 40, 45], 'Type': ['A', 'B', 'B', 'A', 'A']}
df = pd.DataFrame(data_dict) # convert the dictionary to a pandas DataFrame
df # check the DataFrame
```

	Speed	Weight	Type
0	2	25	A
1	3	30	B
2	4	35	B
3	5	40	A
4	6	45	A



A DataFrame is basically a multi-column table

```
type(df) # This is a "pandas DataFrame"
```

```
pandas.core.frame.DataFrame
```

```
df.shape # df has 5 rows and 3 columns
```

```
(5, 3)
```

Note: You can also use `pd.DataFrame()` to convert 2D arrays or nested lists to DataFrames!

Checking attributes, getting statistics

```
# Check the attribute columns (it shows the column names)
df.columns # you can also define column names after df creation
```

```
Index(['Speed', 'Weight', 'Type'], dtype='object')
```

```
df.describe() # only numerical columns have statistics!
```

	Speed	Weight
count	5.000000	5.000000
mean	4.000000	35.000000
std	1.581139	7.905694
min	2.000000	25.000000
25%	3.000000	30.000000
50%	4.000000	35.000000
75%	5.000000	40.000000
max	6.000000	45.000000

Checking a column

```
df['Speed'] # check column 'Speed'
```

```
0    2
1    3
2    4
3    5
4    6
Name: Speed, dtype: int64
```

Note: You can also use custom labels as indices instead of the automatically generated unique ID integers. Just redefine your indices via `df.index = ['label1', 'label2', ..., 'labeln']`



Creating new columns

Redefining a column

```
# Redefine ALL elements of column 'Type'
df['Type'] = ['A', 'A', 'B', 'B', 'A']
df
```

	Speed	Weight	Type
0	2	25	A
1	3	30	A
2	4	35	B
3	5	40	B
4	6	45	A

```
df['Amount'] = [21,23,43,45,65] # Create a new column
df
```

	Speed	Weight	Type	Amount
0	2	25	A	21
1	3	30	B	23
2	4	35	A	43
3	5	40	B	45
4	6	45	A	65

```
# create a new column as a combination of other ones
df['Amount2'] = df['Speed'] * 5 / df['Weight']
df
```

	Speed	Weight	Type	Amount	Amount2
0	2	25	A	21	0.400000
1	3	30	B	23	0.500000
2	4	35	A	43	0.571429
3	5	40	B	45	0.625000
4	6	45	A	65	0.666667

Removing columns

```
# delete columns 'Amount' and 'Amount2', redefine df
df = df.drop(['Amount', 'Amount2'], axis = 1)
df # note: axis = 1 mean that we remove columns,
   # not rows
```

	Speed	Weight	Type
0	2	25	A
1	3	30	B
2	4	35	A
3	5	40	B
4	6	45	A

Getting elements or slices

```
# element on row labeled 1, column labeled 'Speed'
df.loc[1, 'Speed']
```

3

```
# slice with loc: second label is INCLUDED!
df.loc[1:3, 'Speed': 'Weight']
```

	Speed	Weight
1	3	30
2	4	35
3	5	40

```
# index of location (iloc) requires array-like indices
df.iloc[2,1] # row of index 2, column of index 1
```

35

```
# slices with iloc exclude the second index!
df.iloc[2:4, 0:2] # rows 2, 3, and columns 0 and 1
```

	Speed	Weight
2	4	35
3	5	40

```
# get the elements as a 2D array
df.iloc[2:4, 0:2].values
```

```
array([[ 4, 35],
       [ 5, 40]])
```



Get array or transpose it

```
# get the elements as a 2D array
df.values
```

```
array([[2, 25, 'A'],
       [3, 30, 'B'],
       [4, 35, 'A'],
       [5, 40, 'B'],
       [6, 45, 'A']], dtype=object)
```

```
# columns become rows and vice-versa!
df.T # transpose the DataFrame
```

	0	1	2	3	4
Speed	2	3	4	5	6
Weight	25	30	35	40	45
Type	A	B	A	B	A

Selecting columns

```
# select specific columns
selection = ['Type', 'Speed']
df[selection]
```

	Type	Speed
0	A	2
1	B	3
2	A	4
3	B	5
4	A	6

Applying restrictions / masks

```
# which elements of column 'Speed' are >= 4?
df['Speed'] >= 4 # This generates a mask!
```

```
0    False
1    False
2     True
3     True
4     True
Name: Speed, dtype: bool
```

```
# save this mask as mask1
mask1 = df['Speed'] >= 4
```

```
df[mask1] # Apply the mask
```

	Speed	Weight	Type
2	4	35	A
3	5	40	B
4	6	45	A

Making graphs

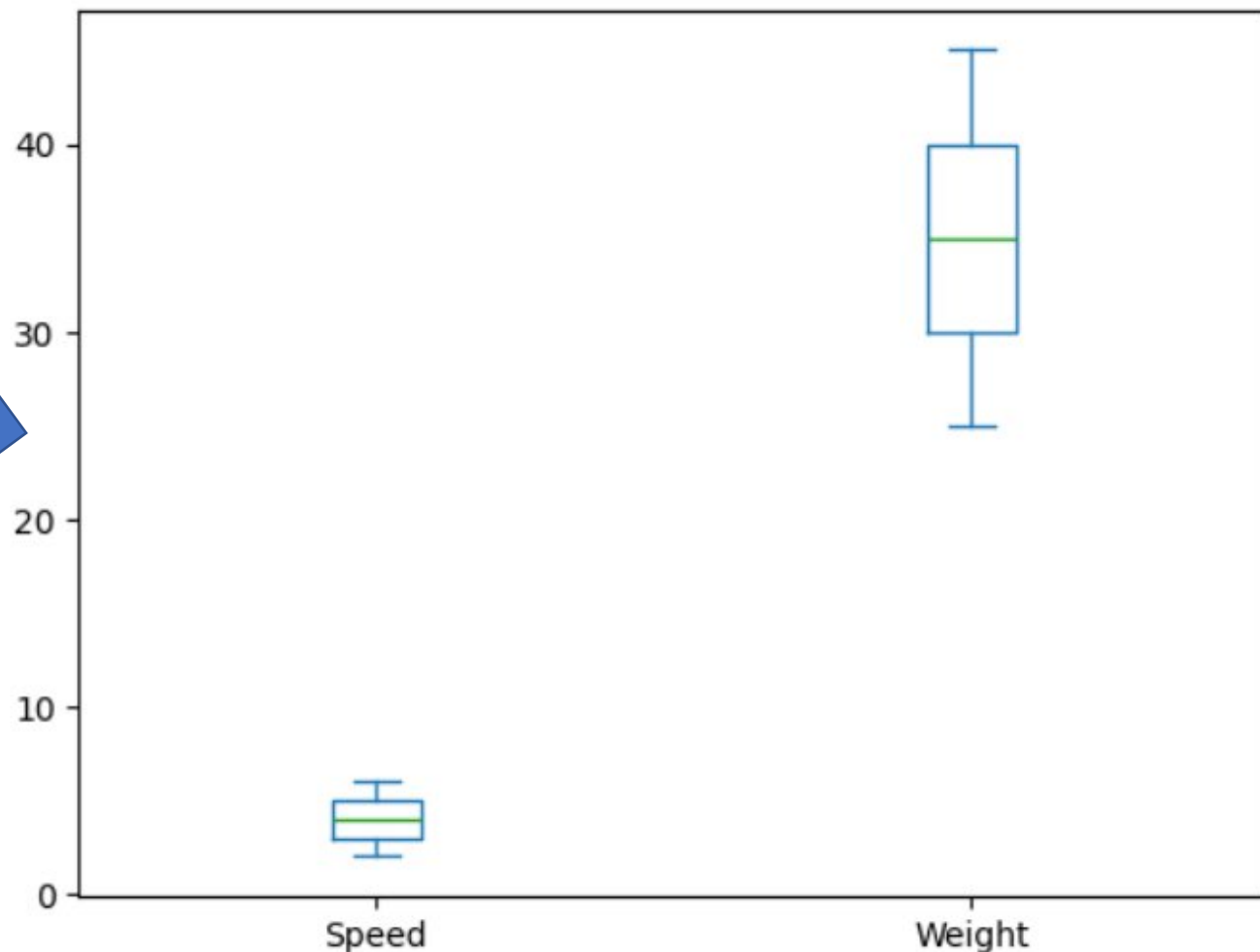
Applying restrictions / masks

```
# make restrictions over different columns  
mask2 = (df['Speed'] >= 4) & (df['Type'] == 'A')  
df[mask2] # you can use many restrictions!
```

	Speed	Weight	Type
2	4	35	A
4	6	45	A

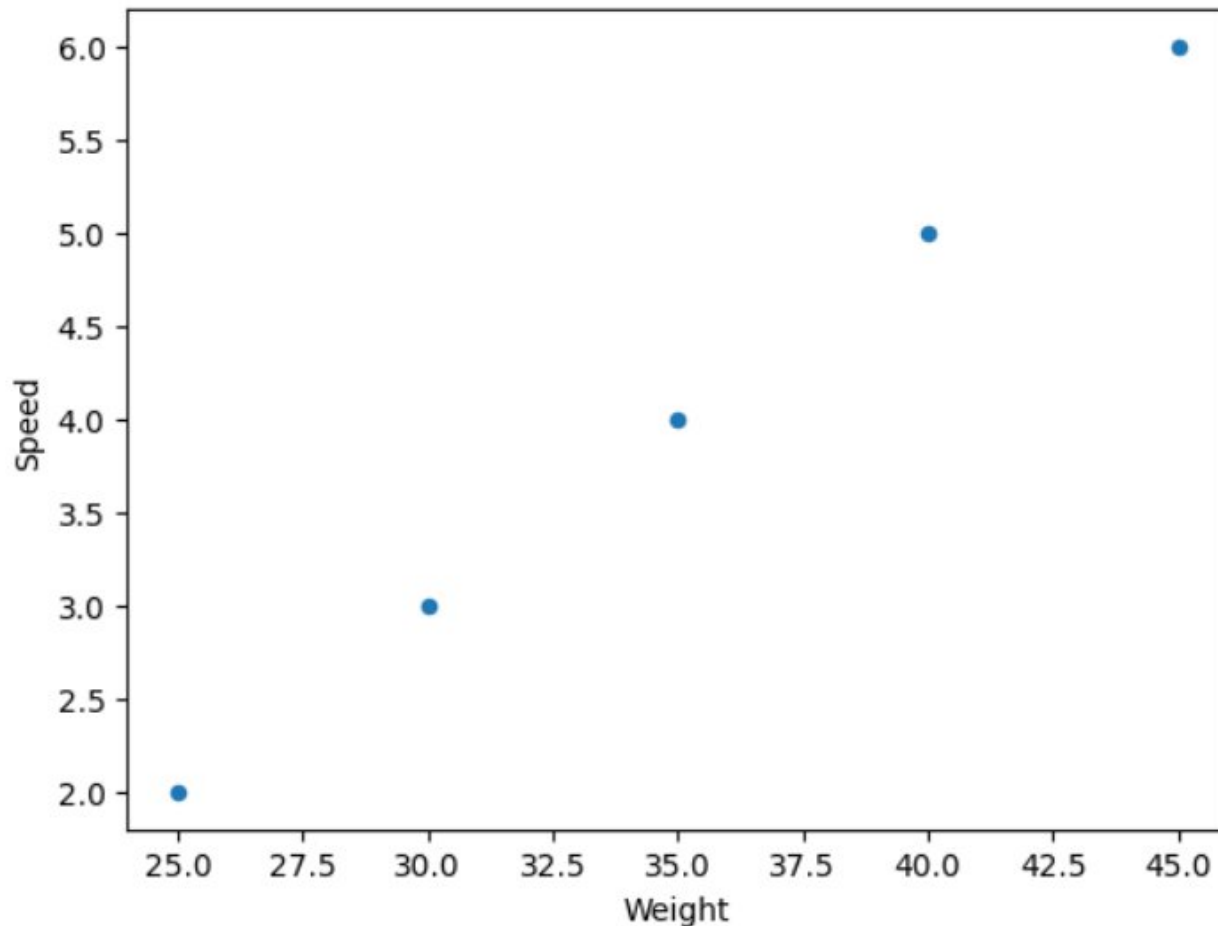
Box plots can easily (visually) identify outliers, which would appear outside (above or below) the confidence intervals. E.g., repeat the box plot on the right after redefining an element via `df.iloc[0,4] = 20`

```
# Make a box plot of the numerical columns  
df.plot(kind = 'box'); # note: ";" avoids undesired messages
```



Making graphs

```
# plot the weight against the speed in a scatter plot  
df.plot(x = 'Weight', y = 'Speed', kind = 'scatter');
```



Tip1: Check `help(df.plot)` to see the full documentation (or docstring) of the plot method. You can adjust many more parameters in the plot!

Tip2: Save your plot (e.g. as a png figure):

```
>>> ax = df.plot(kind='box')
```

```
>>> fig = ax.get_figure()
```

```
>>> fig.savefig("plot.png", dpi=300)
```



Managing missing values

```
# Main list elements become rows.
nested_list = [[2,25,'A'],[3,30,'B'],[4,np.nan,'B'],[5,35,'A'],[6,45,'A']]
col_names = ['Speed', 'Weight', 'Type']
df2 = pd.DataFrame(nested_list, columns = col_names)
df2 # NaN = missing value ("Not a Number")
```

	Speed	Weight	Type
0	2	25.0	A
1	3	30.0	B
2	4	NaN	B
3	5	35.0	A
4	6	45.0	A

Note: numpy was used to generate a missing value: `np.nan`

```
# are there missing values?
df2.isna() # this generates a mask
```

	Speed	Weight	Type
0	False	False	False
1	False	False	False
2	False	True	False
3	False	False	False
4	False	False	False

```
# missing values per column
df2.isna().sum()
```

```
Speed      0
Weight     1
Type       0
dtype: int64
```

```
# remove rows with missing values
df2.dropna()
```

	Speed	Weight	Type
0	2	25.0	A
1	3	30.0	B
3	5	35.0	A
4	6	45.0	A

Managing missing values

```
# Alternatively, replace all missing values by 'value'
value = df2['Weight'].mean() # average of the Weight column
df2['Weight'] = df2['Weight'].fillna(value) # redefine column
df2 # check the updated DataFrame (no missing values now)
```

	Speed	Weight	Type
0	2	25.00	A
1	3	30.00	B
2	4	33.75	B
3	5	35.00	A
4	6	45.00	A

Grouping rows

```
# group rows by 'Type', then get the mean
# of the column 'Weight' for each type
df.groupby('Type')['Weight'].aggregate('mean')
```

```
Type
A    33.333333
B    37.500000
Name: Weight, dtype: float64
```

```
# group rows by 'Type', then get the mean/std/min/max of the column
# 'Weight' for each type found in column 'Type'
df.groupby('Type')['Weight'].aggregate(['mean', 'std', 'min', 'max'])
```

	mean	std	min	max
Type				
A	33.333333	10.408330	25	45
B	37.500000	3.535534	35	40

Grouping rows

```
# group rows by 'Type', then get the mean of column 'Weight' and
# the maximum of the column 'Speed', both for each type
df.groupby('Type').aggregate({'Weight': 'mean', 'Speed': 'max'})
```

	Weight	Speed
Type		
A	33.333333	6
B	34.375000	5

Applying functions to columns

```
# define a simple function that returns 1 or 2
# depending on its argument ('A' or 'B')
def f(item):
    if item == 'A':
        return 1
    elif item == 'B':
        return 2
```

```
f('A') # check if this function works
1
```

Tip: You can apply any function to any column and use the new data generated as a new column for the DataFrame!

```
# apply function f to column 'Type'
df['Type'].apply(f)
```

```
0    1
1    2
2    1
3    2
4    1
Name: Type, dtype: int64
```

This output is a Series!



Managing date & times

```
# column Time has date/time infos as strings
d = {"Sensor": [12, 5, 42, 22, 17],
     "Time": [ "2025-01-10 08:30:00",
               "2025-01-10 09:15:00",
               "2025-01-10 09:45:30",
               "2025-01-10 10:00:00",
               "2025-01-10 10:30:45" ]}

df = pd.DataFrame(d)
df
```

	Sensor	Time
0	12	2025-01-10 08:30:00
1	5	2025-01-10 09:15:00
2	42	2025-01-10 09:45:30
3	22	2025-01-10 10:00:00
4	17	2025-01-10 10:30:45

```
# convert strings to datetime objects
df['Time'] = pd.to_datetime(df['Time'])
df # column time was redefined!
```

	Sensor	Time
0	12	2025-01-10 08:30:00
1	5	2025-01-10 09:15:00
2	42	2025-01-10 09:45:30
3	22	2025-01-10 10:00:00
4	17	2025-01-10 10:30:45

```
# Show only the hour in column Time
df['Time'].dt.hour # or .day, .minute, etc.
```

```
0      8
1      9
2      9
3     10
4     10
Name: Time, dtype: int32
```


Managing date & times

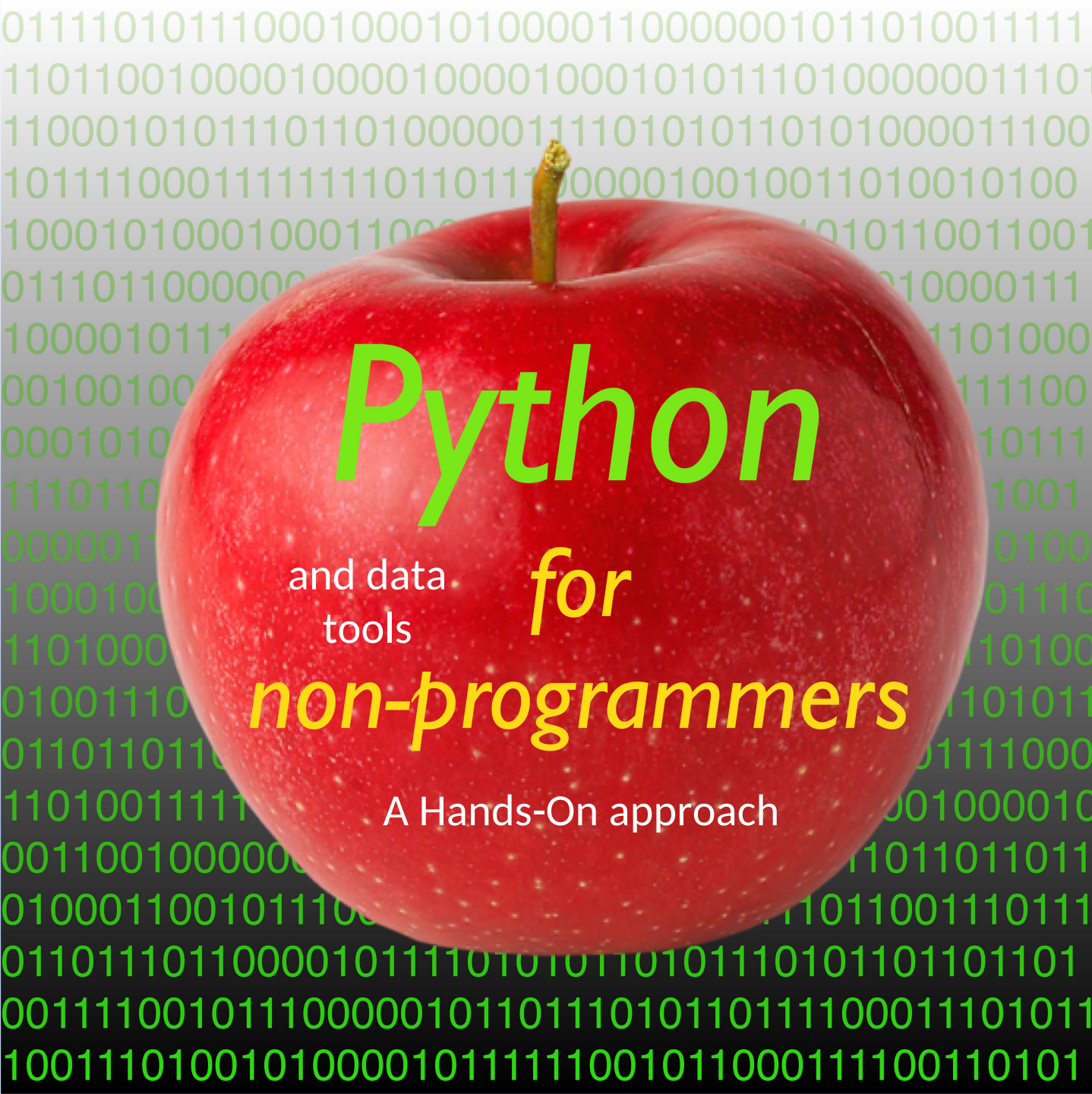
```
# get time differences between consecutive rows
df['Time'].diff() # NaT = Not a Time, equivalent to NaN
```

```
0          NaT
1    0 days 00:45:00
2    0 days 00:30:30
3    0 days 00:14:30
4    0 days 00:30:45
Name: Time, dtype: timedelta64[ns]
```

```
# select experiments done in a specific period (of date and time)
start = "2025-01-10 09:00:00"
end   = "2025-01-10 10:00:00"
mask = (df['Time'] >= start) & (df['Time'] <= end)
df[mask] # only experiments done between start and end are shown!
```

	Sensor	Time
1	5	2025-01-10 09:15:00
2	42	2025-01-10 09:45:30
3	22	2025-01-10 10:00:00

Note: pandas is a very large and important library that has really many classes, methods and submodules! It takes time to learn, so **keep your hands busy!**



UNIVERSITÄT
BAYREUTH

Delivered by

Dr. Rodrigo Albuquerque

(AG Ruckdäschel)

&

AG Künneth

Part of the project
"digitale Methoden"
(bidt-digikollegs)



- Brief overview of Python
- Installation and graphical interfaces
- Data types and operators
- Objects, modules and classes
- Strings
- Lists, sets and tuples
- Dictionary
- Arrays and matrices
- Loops (for, while)
- If statements
- **Functions**, pandas
- Files
- Plotting graphics

FUNCTIONS

A **function** is a block of code that is executed whenever it is called. It is useful to repeat two or more lines of code multiple times inside your final program.

```
In [1]: def myfunction():  
        print('My first function!')
```

```
In [2]: myfunction()  
  
My first function!
```

```
In [3]: def f():  
        """  
        This function prints something!  
        """  
        print("My second function!")
```

```
In [4]: f()  
  
My second function!
```

```
In [5]: def g(a):  
        """  
        This returns a^2  
        """  
        return a**2
```

```
In [6]: g(3)  
Out[6]: 9
```

Note: If you don't know what to write inside the function, just write the command **pass**.

docstring (= Your documentation.
Write whatever you want!)

Tip: In Jupyter, press **SHIFT + TAB** close your function to read information about it (e.g. the docstring). Or just **help(f)**.

functions can have a **return** command!



Unpacking the result of a function:

```
In [8]: b = g(3)
```

```
In [9]: b
```

```
Out[9]: 9
```



Multiple arguments, multiple outputs

```
In [1]: def h(a,b,c):
        """
        Return the product and sum of a, b and c
        """
        product = a*b*c
        addition = a+b+c # note, "sum" is a command
        return product, addition
```

```
In [2]: h(1, 2, 5) # one can write h(1,2,5) too...
Out[2]: (10, 8)
```

```
In [3]: p, s = h(1, 2,5) # unpacking the results
```

```
In [4]: p, s
Out[4]: (10, 8)
```

```
In [5]: def create_list(a):
        """
        It returns a list from the input 'a' (integer)
        'a' cannot be smaller than 1.
        """
        if a < 1:
            return # leave the function and do nothing!
        else:
            mylist = [a, a**2, a**3]
        return mylist
```

```
In [6]: create_list(0)
```

no output!

```
In [7]: create_list(2)
```

```
Out[7]: [2, 4, 8]
```

```
In [8]: mylist # variables defined inside functions are
          # not remembered outside of them
          # check the command "global"
```

mylist is in the local namespace,
as well as the temporary variable a

```
-----
NameError                                Traceback (most recent call last)
/var/folders/g0/9w3xrlnd031ftmvx29wmvw9w0000gn/T/ipykernel_19327/2306592467.py in <module>
----> 1 mylist # variables defined inside functions are
      2         # not remembered outside of them
      3         # check the command "global"

NameError: name 'mylist' is not defined
```

Tip: To make a variable created inside a function to be used outside the function, define it (inside the function) in the global namespace with the command e.g. `global mylist`



Making a variable available outside the function

```
In [1]: def create_list(a):
        """
        It returns a list from the input 'a' (integer)
        'a' cannot be smaller than 1.
        """
        global mylist # mylist is now in the global namespace!

        if a < 1:
            return # leave the function and do nothing!
        else:
            mylist = [a, a**2, a**3]

        return mylist
```

```
In [2]: create_list(3)
```

```
Out[2]: [3, 9, 27]
```

```
In [3]: mylist
```

```
Out[3]: [3, 9, 27]
```

Getting help...

```
In [4]: help(create_list)
```

Help on function create_list in module __main__:

```
create_list(a)
    It returns a list from the input 'a' (integer)
    'a' cannot be smaller than 1.
```

Updating objects inside a function

```
In [5]: a = ["hi", "my", "name", "is"]
```

```
In [6]: def update_a(x):
        """
        Function to update the list 'a'
        """
        a.append(x)
```

```
In [7]: update_a("John")
```

```
In [8]: a
```

```
Out[8]: ['hi', 'my', 'name', 'is', 'John']
```

```
In [8]: def update_f(x, a):
        """
        Function to update the list 'a'
        with the element x
        """
        a.append(x)
```

```
In [9]: b = [1,2,3]
        update_f('hi', b)
        b
```

```
Out[9]: [1, 2, 3, 'hi']
```



Multiple unspecified arguments

```
In [1]: def f(*args):
        """
        Print all arguments given.
        """
        print(f'Arguments provided: {args}')
```

`args` = tuple with all arguments provided
(`*args` = "Non-keyword arguments")

```
In [2]: f('hi', 'you', 'all', 1, 2, 3)

Arguments provided: ('hi', 'you', 'all', 1, 2, 3)
```

```
In [3]: def g(a, b = 10, c = None, *args):
        """
        Different arguments are possible here!
        """
        print(f'a = {a}, b = {b}, c = {c}')
        print(f'Extra arguments = {args}')
```

```
In [4]: g(1, 2, 3, "Hi", 2023)

a = 1, b = 2, c = 3
Extra arguments = ('Hi', 2023)
```

```
In [5]: g(a = 5, c = 15) # default b value is used

a = 5, b = 10, c = 15
Extra arguments = ()
```

Topics of interest:

- Update any object(s) inside a function
- Functions inside a function (**nested functions**)
- Function inside itself (**recursive function**)

`kwargs` = dictionary
(`**kwargs` = "Keyword arguments")

```
In [6]: def h(**kwargs):
        """
        Function to print keyword arguments,
        which are transformed into a dictionary
        """
        print(kwargs)
```

```
In [7]: h(x = 1, y = 2, z = 3)

{'x': 1, 'y': 2, 'z': 3}
```

```
In [8]: def h(**kwargs):
        """
        Function to print keyword arguments,
        which are transformed into a dictionary
        """
        for key in kwargs:
            print(key, "====>", kwargs[key])
```

```
In [9]: h(x = 1, y = 2, z = 3)

x ====> 1
y ====> 2
z ====> 3
```

Note: The words `args` and `kwargs` are default words used in Python (recommended), but you can use any other one if you prefer.



Exercises

1) Generate the function `f(full_name)` that returns a list with all words of that name, where `full_name` is the full name as one single string.

Tip: use the string method `.split()` inside your function.

```
In [1]: def f(full_name):
        """This function returns the list of
        words inside the string full_name"""
        list_of_words = full_name.split()
        return list_of_words
```

```
In [2]: f("John Kirk Johansen Junior")
Out[2]: ['John', 'Kirk', 'Johansen', 'Junior']
```

2) Generate the function `g(full_name)` that prints how many words are there in the string `full_name`. Call the above-defined `f` function inside your `g` function to help with the code.

```
In [3]: def g(full_name):
        """This function prints the number
        of words inside a given full name."""
        words = f(full_name)
        n = len(words)
        print(f'The input name has {n} words')
```

```
In [4]: g("John Kirk Johansen Junior")
The input name has 4 words
```

3) Generate a function `h(full_name)` that returns the total number of vowels (a, e and i) inside the string `full_name`.

```
In [5]: def h(full_name):
        """Function to counts the total number
        of 'a', 'e' and 'i' inside a full name"""
        a = full_name.count('a')
        e = full_name.count('e')
        i = full_name.count('i')
        total = a + e + i
        return total
```

```
In [6]: h("Johnnie Steinway")
Out[6]: 5
```

4) Create the function `f1(full_name)` to print the number of words and of selected vowels (a, e and i) inside a full name given as input. For that, use both the `g` and `h` functions inside the `f1` function.

```
In [7]: def f1(full_name):
        """Print the number of words and of
        vowels (a,e,i) inside full_name"""
        g(full_name) # call the first function
        n_vowels = h(full_name)
        print(f"The input name has {n_vowels} (a+e+i)'s")
```

```
In [8]: f1("Joanne Kirkeil Delaware")
The input name has 3 words
The input name has 9 (a+e+i)'s
```

5) **Challenge:** How to fix the bug, so that the output below would show 4 (a,e,i) vowels instead of only 2?

```
In [9]: # challenge: why f1 does not work? fix the bug...
        f1("Amy Emerrich")

The input name has 2 words
The input name has 2 (a+e+i)'s
```

```
In [11]: def h(full_name):
        """Function to counts the total number
        of 'a', 'e' and 'i' inside a full name"""
        full_name = full_name.lower()
        a = full_name.count('a')
        e = full_name.count('e')
        i = full_name.count('i')
        total = a + e + i
        return total
```

```
In [12]: f1("Amy Emerrich")

The input name has 2 words
The input name has 4 (a+e+i)'s
```



Exercises

6) Create a function with three arguments: dictionary name, key and value to update an existing dictionary

```
In [1]: def update_dict(dictionary, key, value):
        """
        Function to update a dictionary
        """
        dictionary[key] = value
```

```
In [2]: mydict = {'H': 1}
```

```
In [4]: mydict
Out[4]: {'H': 1, 'He': 2}
```

```
In [3]: update_dict(mydict, 'He', 2)
```

7) Somebody used a list as the first argument of the above function and got an error. Change the function to be sure that the first argument is a dictionary.

```
In [5]: def update_dict(dictionary, key, value):
        """
        Function to update a dictionary
        """
        if type(dictionary) == dict:
            dictionary[key] = value
        else:
            print('No dictionary found!')
```

```
In [6]: mylist = [1,2,3]
        update_dict(mylist, 'Li', 3)

        No dictionary found!
```

```
In [7]: update_dict(mydict, 'Li', 3)
```

```
In [8]: mydict
Out[8]: {'H': 1, 'He': 2, 'Li': 3}
```

8) Create a function that takes any array as argument and returns the total sum, average and standard deviation of all elements in the array.

```
In [9]: def f(myarray):
        """
        Function to get the sum, average
        and standard deviation of myarray
        """
        ave = myarray.mean()
        summ = myarray.sum()
        std = myarray.std()
        return ave, summ, std
```

```
In [10]: import numpy as np
         a = np.array([[2,4,6],[1,3,5]])
```

```
In [11]: f(a) # alternative: ave, summ, std = f(a)
Out[11]: (3.5, 21, 1.707825127659933)
```

9) **Challenge:** The roots x_1 and x_2 of a polynomial of second order of the type $y(x) = ax^2 + bx + c$ is given by $\frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$

Write a function that takes a , b and c as arguments and returns x_1 and x_2 . Make sure that a cannot be zero and that $b^2 \geq 4ac$ to get real roots only.

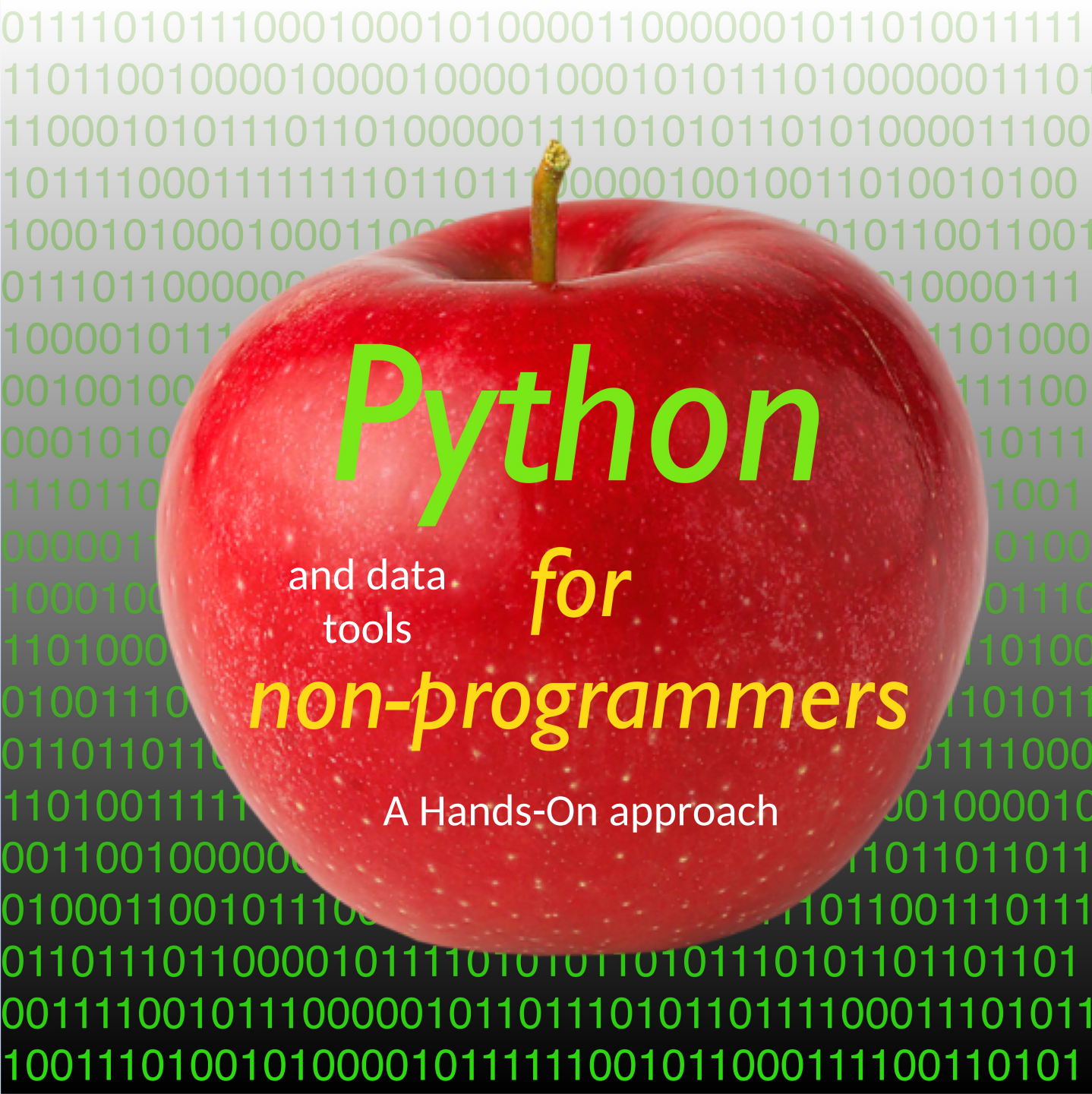
```
In [12]: def roots(a,b,c):
        """Get the real roots of the
        polynomial y(x) = ax^2 + bx + c"""
        if a == 0 or b**2 < 4*a*c:
            print("No real roots found!")
            return
        delta = np.sqrt(b**2 - 4*a*c)
        x1 = (-b + delta)/(2*a)
        x2 = (-b - delta)/(2*a)
        return x1, x2
```

```
In [13]: roots(2,3,2)

        No real roots found!
```

```
In [14]: roots(2,3,1)
Out[14]: (-0.5, -1.0)
```





UNIVERSITÄT
BAYREUTH

Delivered by

Dr. Rodrigo Albuquerque

(AG Ruckdäschel)

&

AG Künneth

Part of the project
“digitale Methoden”
(bidt-digikollegs)



- Brief overview of Python
- Installation and graphical interfaces
- Data types and operators
- Objects, modules and classes
- Strings
- Lists, sets and tuples
- Dictionary
- Arrays and matrices
- Loops (for, while)
- If statements
- Functions, pandas
- **Files**
- Plotting graphics

Reading files

```

1 Total repulsion energy = 23.435238 a.u.
2 Total energy for this iteration = 22.694720 a.u.
3 Final geometry:
4 =====
5      X      Y      Z
6 H 0.0000 0.0000 0.0000
7 H 0.0000 0.0000 0.7400
8 Wall time: 1.321 seconds
    
```

file.txt

```
In [1]: f = open("file.txt") # open("file.txt", 'r')
```

read mode (default): 'r'
write/create mode: 'w'
append mode: 'a'

```
In [2]: f.readline() # first line as a string
```

```
Out[2]: 'Total repulsion energy = 23.435238 a.u.\n'
```

new line: `\n`

```
In [3]: f.readline() # second line as a string
```

```
Out[3]: 'Total energy for this iteration = 22.694720 a.u. \n'
```

After this command, the file
"reader" went to line 3

```
In [4]: f.seek(0) # rewind the file object
```

```
Out[4]: 0
```

Rewind the file "reader"
back to the beginning

```
In [5]: f.readlines() # list with all lines as strings
```

```
Out[5]: ['Total repulsion energy = 23.435238 a.u.\n',
'Total energy for this iteration = 22.694720 a.u. \n',
'Final geometry:\n',
'===== \n',
'      X      Y      Z\n',
'H 0.0000 0.0000 0.0000\n',
'H 0.0000 0.0000 0.7400\n',
'Wall time: 1.321 seconds\n']
```

Finally,

```
In [6]: f.close()
```

Note 1: In practice, do whatever you want inside a **block** of code, after which the file is automatically closed.

```
In [7]: with open("file.txt") as f:
        lines = f.readlines()
```

```
In [8]: lines
```

```
Out[8]: ['Total repulsion energy = 23.435238 a.u.\n',
'Total energy for this iteration = 22.694720 a.u. \n',
'Final geometry:\n',
'===== \n',
'      X      Y      Z\n',
'H 0.0000 0.0000 0.0000\n',
'H 0.0000 0.0000 0.7400\n',
'Wall time: 1.321 seconds\n']
```

```
In [9]: len(lines)
```

```
Out[9]: 8
```

Note 2: The empty line is caused by two `\n` characters: one from the `print()` command and another one from `line`.

```
In [10]: for line in lines:
        if "H" in line:
            print(line)
```

```
H 0.0000 0.0000 0.0000
H 0.0000 0.0000 0.7400
```



Appending info to files

purchases.txt

```
1 List of purchased items
2
3 =====
4 Item   Price(Eur)   Date(DD.MM.YYYY)
5 =====
6 Pan      30.00      11.02.2019
7 Fridge   160.00      20.10.2020
```

append mode

```
In [1]: with open("purchases.txt", "a") as f:
        line = "Pencil      1.99      29.12.2021" + '\n'
        f.write(line)
```

```
In [2]: with open("purchases.txt") as f:
        lines = f.readlines()
```

```
In [3]: lines
```

```
Out[3]: ['List of purchased items\n',
        '\n',
        '=====\n',
        'Item   Price(Eur)   Date(DD.MM.YYYY)\n',
        '=====\n',
        'Pan      30.00      11.02.2019\n',
        'Fridge   160.00      20.10.2020\n',
        'Pencil    1.99      29.12.2021\n']
```

General notes:

- Nested file blocks (transfer info between two files)
- We only used the **text mode**, but there is also a **byte mode**

Creating/overwriting files

file1.txt => temporary file.
apple, orange, banana.

```
In [4]: with open("file1.txt", "w") as f:
        f.write('Hello!\n')
```

file1.txt was overwritten!!

```
In [5]: with open("file1.txt") as f:
        lines = f.readlines()
        lines
```

```
Out[5]: ['Hello!\n']
```

To avoid overwriting existing files:

```
In [6]: import os # os.getcwd(), os.listdir(), os.chdir()
```

```
In [7]: filename = "file1.txt"
        if os.path.isfile(filename):
            print(f"{filename} already exists!")
        else: # if file1.txt doesn't exist, then:
            with open(filename, 'w') as f:
                f.write('Hello!\n')
```

file1.txt already exists!

```
In [8]: filename = "file1.txt"
        if not os.path.isfile(filename):
            with open(filename, 'w') as f:
                f.write('Hello!\n')
```



Exercises

- 1) Write a file called filetemp.txt with any two lines, then append a third line to it

```
In [1]: with open('tempfile.txt', 'w') as f:
        f.write('I like apples\n')
        f.write('I like oranges\n')
```

```
In [2]: with open('tempfile.txt', 'a') as f:
        f.write('I like bananas too!\n')
```

```
In [3]: with open('tempfile.txt') as f:
        lines = f.readlines()
        lines
```

```
Out[3]: ['I like apples\n', 'I like oranges\n', 'I like bananas too!\n']
```

- 2) Iterate over the elements of the list ['John', 'Mary', 'Tim'] to write 3 lines to any file.

```
In [4]: mylist = ['John', 'Mary', 'Tim']
        with open('tempfile2.txt', 'w') as f:
            for name in mylist:
                f.write(f'The chosen person is {name}\n')
```

```
In [5]: with open('tempfile2.txt') as f:
        lines = f.readlines()
        lines
```

```
Out[5]: ['The chosen person is John\n',
        'The chosen person is Mary\n',
        'The chosen person is Tim\n']
```

- 3) Iterate over the lines of "filetemp_3.txt" to get the **annual** salary of John Wayne.

```
In [6]: with open('filetemp_3.txt') as f:
        lines = f.readlines()
```

```
In [8]: salary
Out[8]: 16782.6
```

```
In [7]: for line in lines:
        line = line.split()
        if line[0] == 'John' and line[1] == 'Wayne':
            salary_str = line[2]
            salary = float(salary_str) * 12
            break
```

Add a new if condition:

```
for line in lines:
    line = line.split()
    if len(line) == 3:
        if line[0] == 'John' and line[1] == 'Wayne':
            salary_str = line[2]
            salary = float(salary_str) * 12
            break
```

```
List of salaries for the company XXX
NAME      MONTHLY SALARY (EUROS)
Will Pratty      2087.13
John Carter      3290.32
John Wayne       1398.55
Tim Wayne        2672.10
```

Note: The if condition would not work if lines with less than 3 words are present! Possible [solution](#):



Exercises

4) Challenge: Iterate over the lines of `file2.txt` to find all lines starting with 'A' or 'a' and save these lines to a new file named `file3.txt`

`file2.txt`

```
A person that does not know,
what is right or wrong,
a typical thing among those,
Who didn't have a moral education.
and that's it.
```

5) Challenge 2: Overwrite the content of `file4.txt` by replacing John by Mary in the original text. Use the `open()` command.

```
John saw Mary and got interested
That was the right thing to do
Mary thought it was wrong
And decided to tell John
```

`file4.txt`

```
In [9]: with open('file2.txt') as fin:
        with open('file3.txt', 'w') as fout:
            lines = fin.readlines()
            for line in lines:
                if line[0].lower() == 'a':
                    fout.write(line)
```

```
In [10]: with open('file3.txt') as f:
          lines = f.readlines()
```

```
In [11]: lines
```

```
Out[11]: ['A person that does not know,\n',
          'a typical thing among those,\n',
          "and that's it.\n"]
```

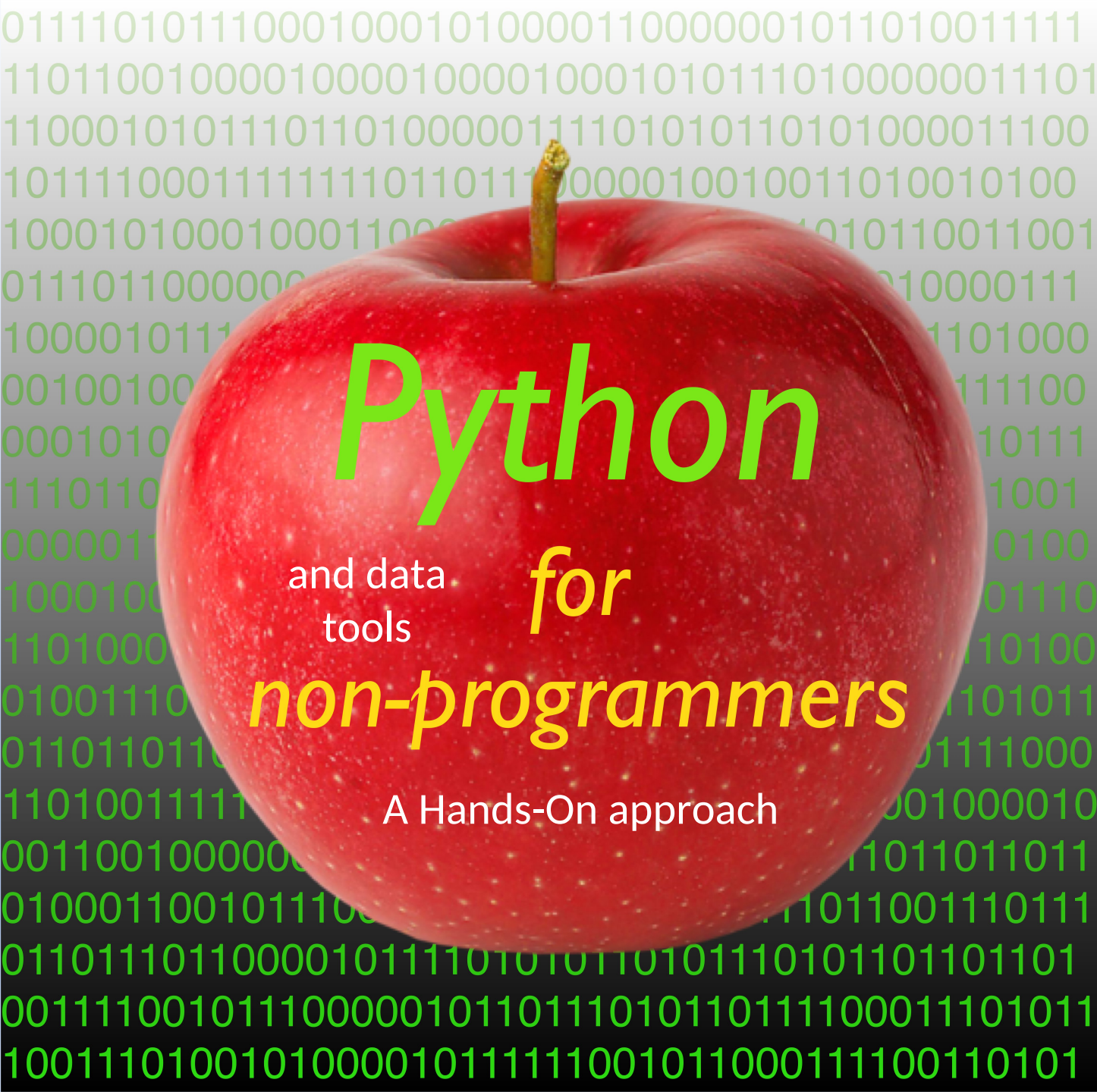
```
In [12]: with open("file4.txt") as f:
          text = f.read()
          with open("file4.txt", 'w') as f:
              text = text.replace('Mary', 'XXX')
              text = text.replace('John', 'Mary')
              text = text.replace('XXX', 'John')
              f.write(text)
```

```
In [13]: with open('file4.txt') as f:
          lines = f.readlines()
          lines
```

```
Out[13]: ['Mary saw John and got interested\n',
          'That was the right thing to do\n',
          'John thought it was wrong\n',
          'And decided to tell Mary\n']
```

Note: The `.read()` method generates one single string with the whole text and including '\n' characters.





UNIVERSITÄT
BAYREUTH

Delivered by

Dr. Rodrigo Albuquerque

(AG Ruckdäschel)

&

AG Künneth

Part of the project
“digitale Methoden”
(bidt-digikollegs)



- Brief overview of Python
- Installation and graphical interfaces
- Data types and operators
- Objects, modules and classes
- Strings
- Lists, sets and tuples
- Dictionary
- Arrays and matrices
- Loops (for, while)
- If statements
- Functions, pandas
- Files
- Plotting graphics

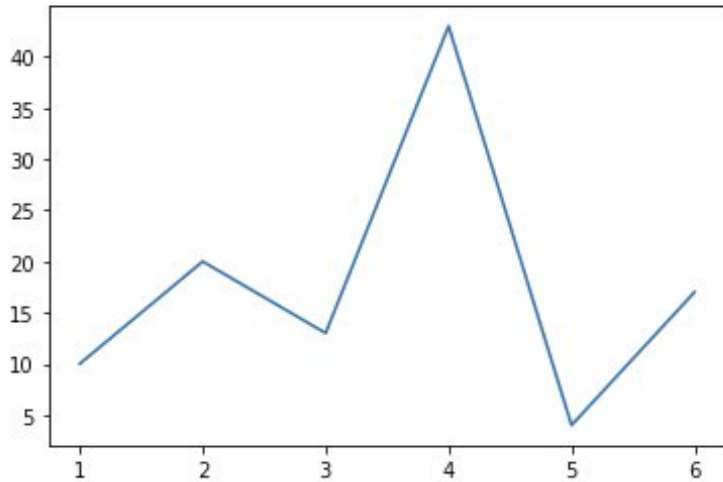
Library Matplotlib → <https://matplotlib.org/stable/gallery/index.html>

Note: Check the library **seaborn** too!

```
In [1]: import matplotlib.pyplot as plt
```

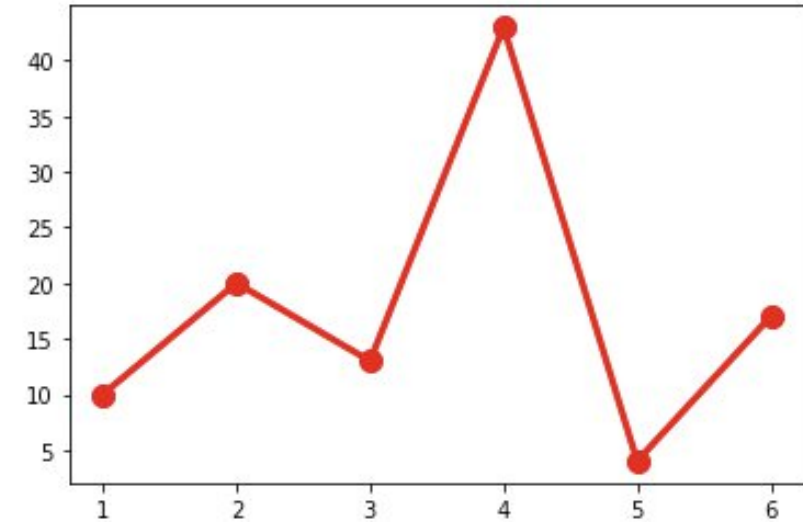
```
In [2]: x = [1,2,3,4,5,6]
        y = [10,20,13,43,4,17]
        plt.plot(x, y);
```

; = suppress text messages



Note: You could also plot only one coordinate e.g. `plt.plot(y)`, where the corresponding x values would be automatically generated.

```
In [3]: plt.plot(x, y, '-o', color="red", lw = 3, ms = 10);
```

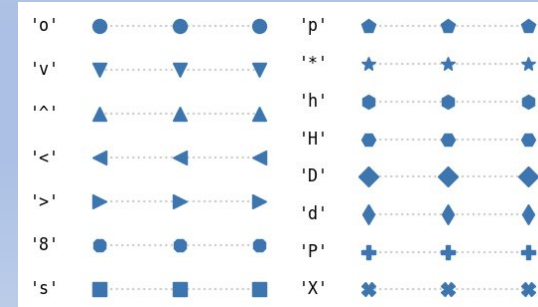


`lw` = line width
`ms` = marker size

`'-o'`: { continuous line
marker shape = circle

Tips:

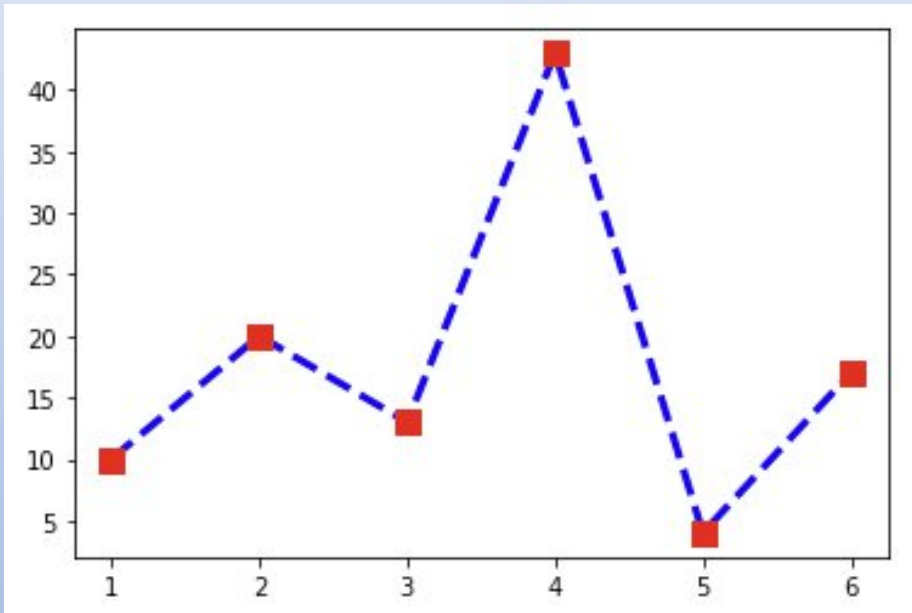
`'-bo'` = blue circle marker with continuous line
`'--gs'` = green square marker with dashed line
etc... (check the matplotlib **documentation!**)



➡ In a Python session, you would need `plt.show()`

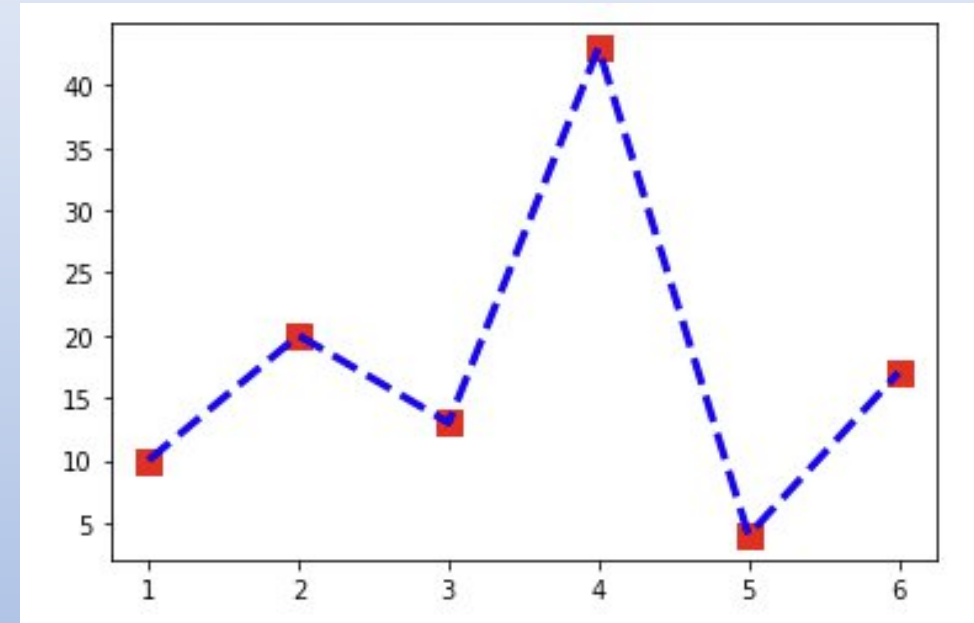
Multiple plots in the same area

```
In [4]: plt.plot(x, y, '--', color="blue", lw = 3)  
plt.plot(x, y, 's', color="red", ms = 10);
```



Note: you can add as many plots as you want in the same area, each with its own configuration.

```
In [5]: plt.plot(x, y, 's', color="red", ms = 10)  
plt.plot(x, y, '--', color="blue", lw = 3);
```

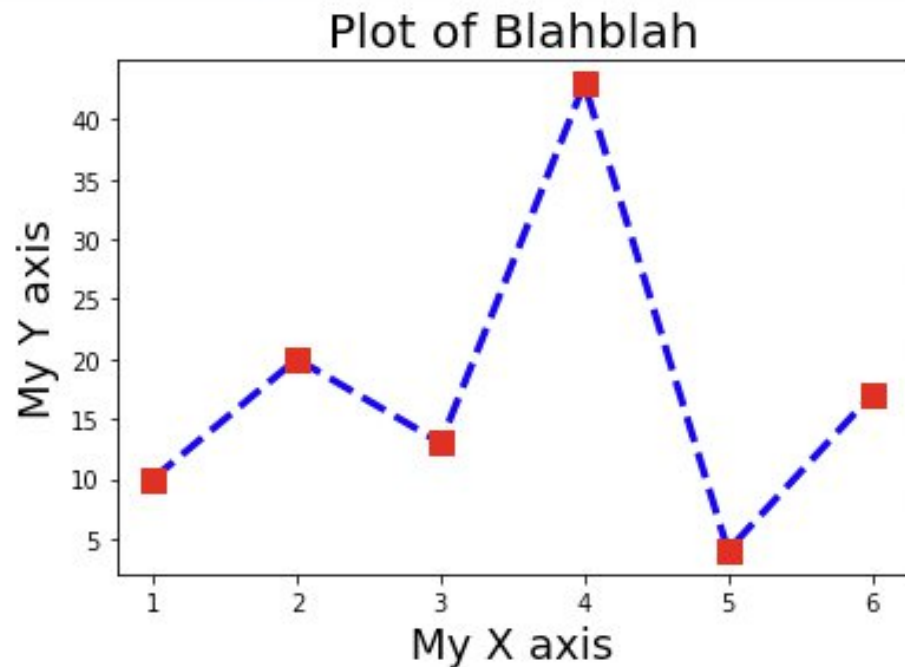


The red markers are now **behind** the line

Tip: You can also control which elements are in the front via the keyword `zorder = integer` (e.g. `zorder = 20`)

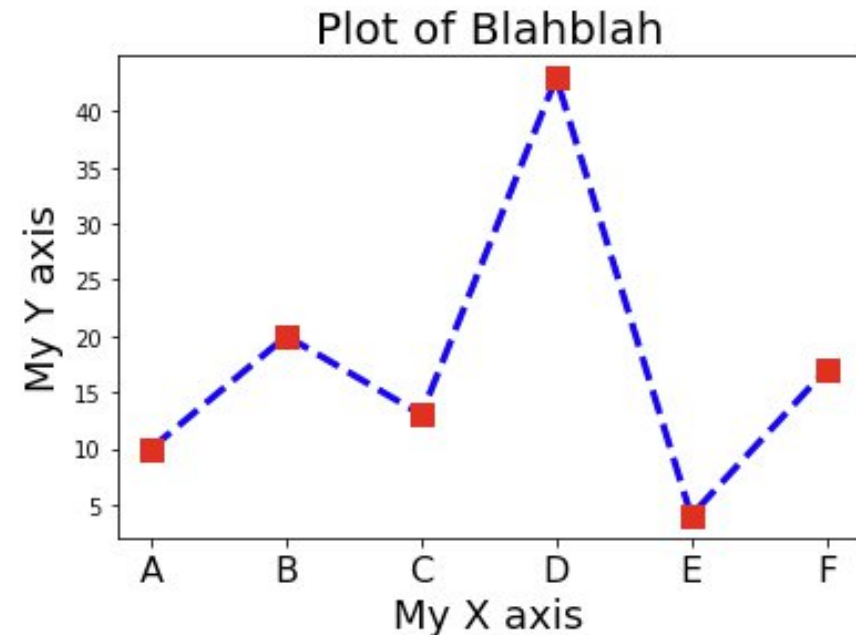
Adding labels and title

```
In [6]: plt.plot(x, y, '--', color="blue", lw = 3)
plt.plot(x, y, 's', color="red", ms = 10)
plt.title('Plot of Blahblah', fontsize = 20)
plt.xlabel('My X axis', fontsize=18)
plt.ylabel('My Y axis', fontsize=18);
```



Customizing the ticks

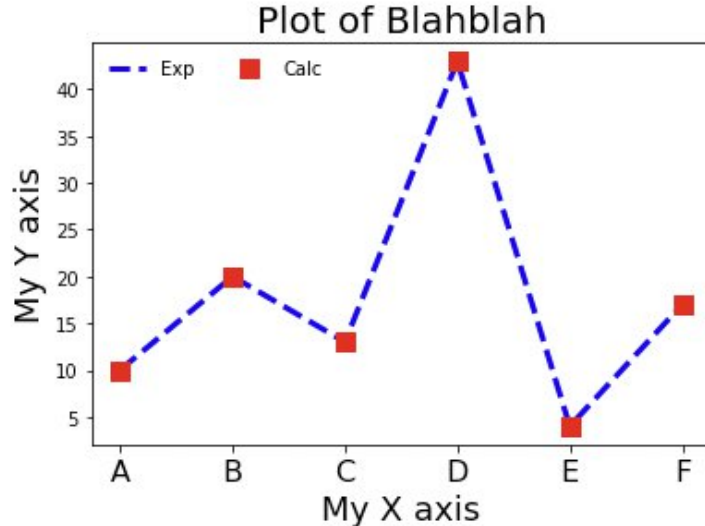
```
In [7]: plt.plot(x, y, '--', color="blue", lw = 3)
plt.plot(x, y, 's', color="red", ms = 10)
plt.title('Plot of Blahblah', fontsize = 20)
plt.xlabel('My X axis', fontsize=18)
plt.ylabel('My Y axis', fontsize=18)
plt.xticks(x, ['A', 'B', 'C', 'D', 'E', 'F'], fontsize = 16);
```



Note: Any text printed in the graph accepts the beautiful LaTeX equations!

Adding legend

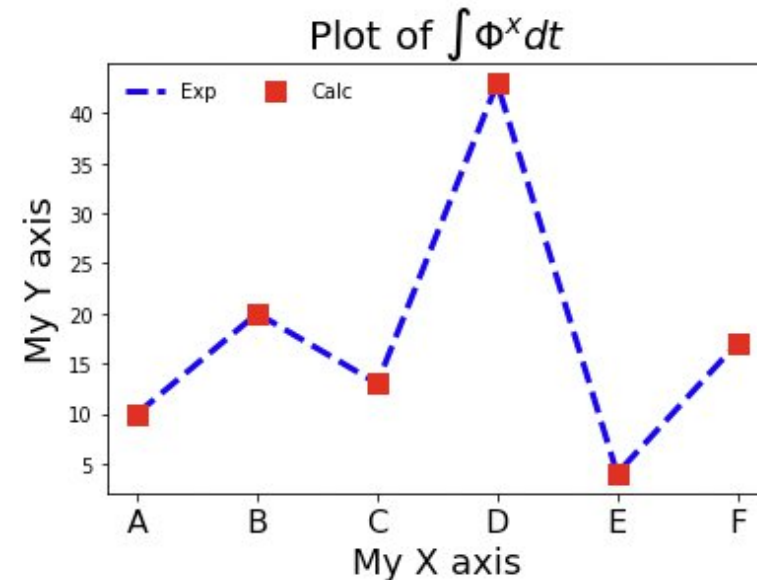
```
In [8]: plt.plot(x, y, '--', color="blue", lw = 3, label = "Exp")
plt.plot(x, y, 's', color="red", ms = 10, label = "Calc")
plt.title('Plot of Blahblah', fontsize = 20)
plt.xlabel('My X axis', fontsize=18)
plt.ylabel('My Y axis', fontsize=18)
plt.xticks(x, ['A', 'B', 'C', 'D', 'E', 'F'], fontsize = 16)
plt.legend(frameon = False, ncol = 2, loc = "upper left");
```



`frameon = False`: Do not show a rectangle around the legend
`ncol` = number of columns (default = 1)
`loc` = location of the label (default = best visible place)

Adding LaTeX-type equations

```
In [9]: plt.plot(x, y, '--', color="blue", lw = 3, label = "Exp")
plt.plot(x, y, 's', color="red", ms = 10, label = "Calc")
plt.title(r'Plot of $\int \Phi^x dt$', fontsize = 20)
plt.xlabel('My X axis', fontsize=18)
plt.ylabel('My Y axis', fontsize=18)
plt.xticks(x, ['A', 'B', 'C', 'D', 'E', 'F'], fontsize = 16)
plt.legend(frameon = False, ncol = 2, loc = "upper left");
```



`$ \int \Phi^x $`

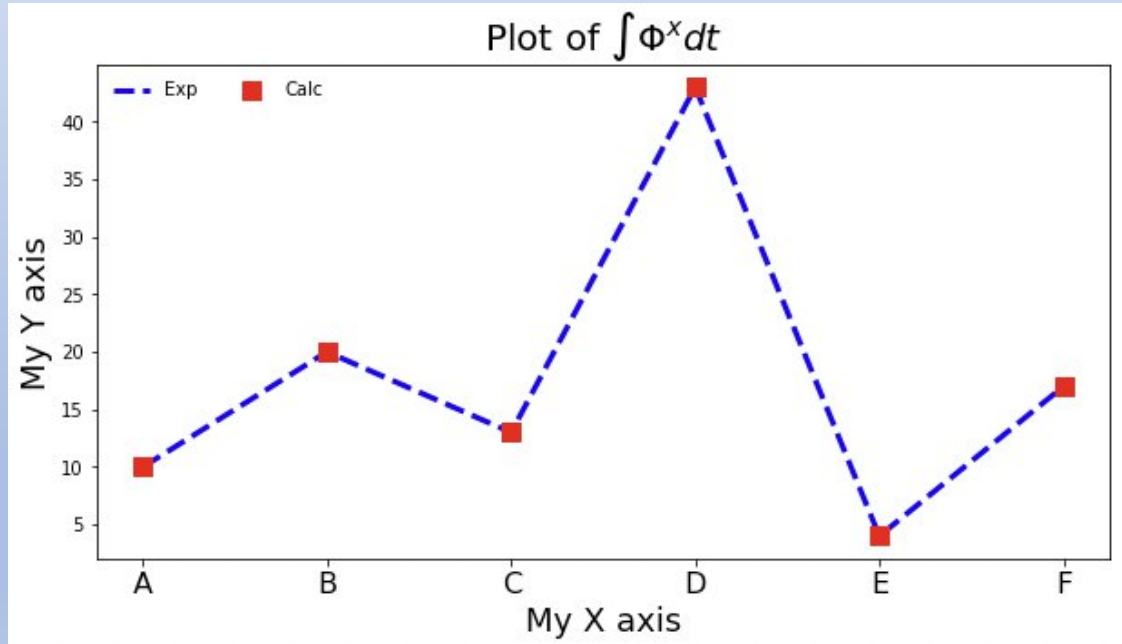
Notice `r' '`

Check some nice LaTeX symbols

<https://www.cmor-faculty.rice.edu/~heinken/latex/symbols.pdf>

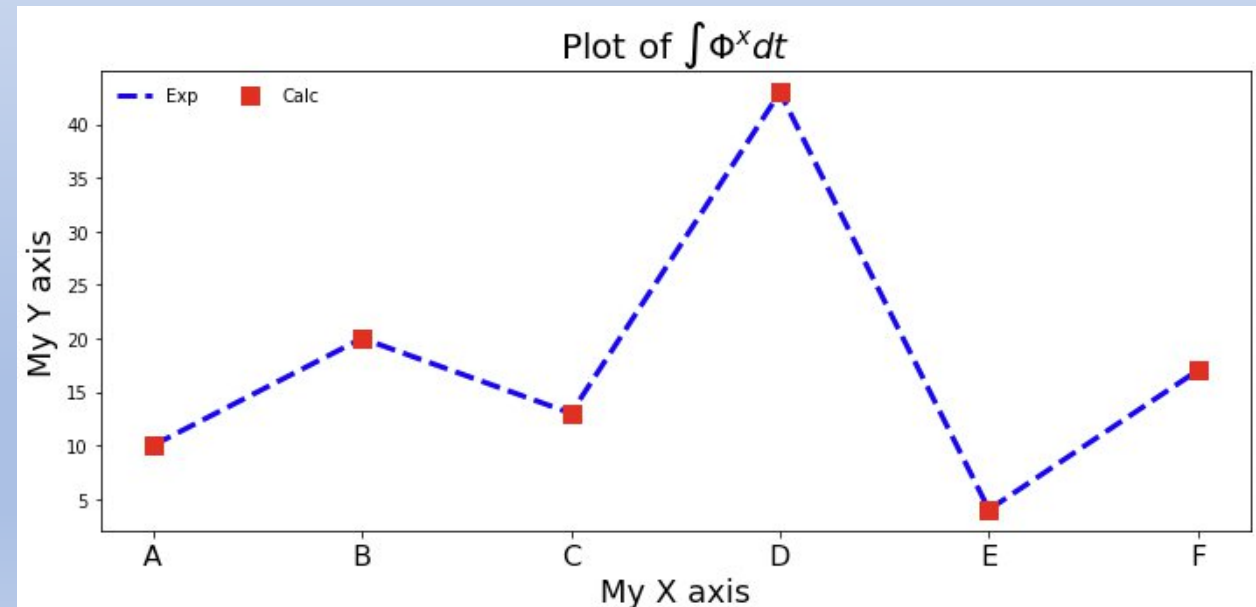
Shape of the graph

```
In [10]: plt.figure(figsize = (10,5)) # width, height
plt.plot(x, y, '--', color="blue", lw = 3, label = "Exp")
plt.plot(x, y, 's', color="red", ms = 10, label = "Calc")
plt.title(r'Plot of $\int \Phi^x dt$', fontsize = 20)
plt.xlabel('My X axis', fontsize=18)
plt.ylabel('My Y axis', fontsize=18)
plt.xticks(x, ['A', 'B', 'C', 'D', 'E', 'F'], fontsize = 16)
plt.legend(frameon = False, ncol = 2, loc = "upper left");
```



Improving layout and saving

```
In [11]: plt.figure(figsize = (10,5)) # width, height
plt.plot(x, y, '--', color="blue", lw = 3, label = "Exp")
plt.plot(x, y, 's', color="red", ms = 10, label = "Calc")
plt.title(r'Plot of $\int \Phi^x dt$', fontsize = 20)
plt.xlabel('My X axis', fontsize=18)
plt.ylabel('My Y axis', fontsize=18)
plt.xticks(x, ['A', 'B', 'C', 'D', 'E', 'F'], fontsize = 16)
plt.legend(frameon = False, ncol = 2, loc = "upper left")
plt.tight_layout()
plt.savefig("myfig.png", dpi = 400);
```



`.tight_layout()` accepts arguments like e.g. `pad = 2`

Grid subplots

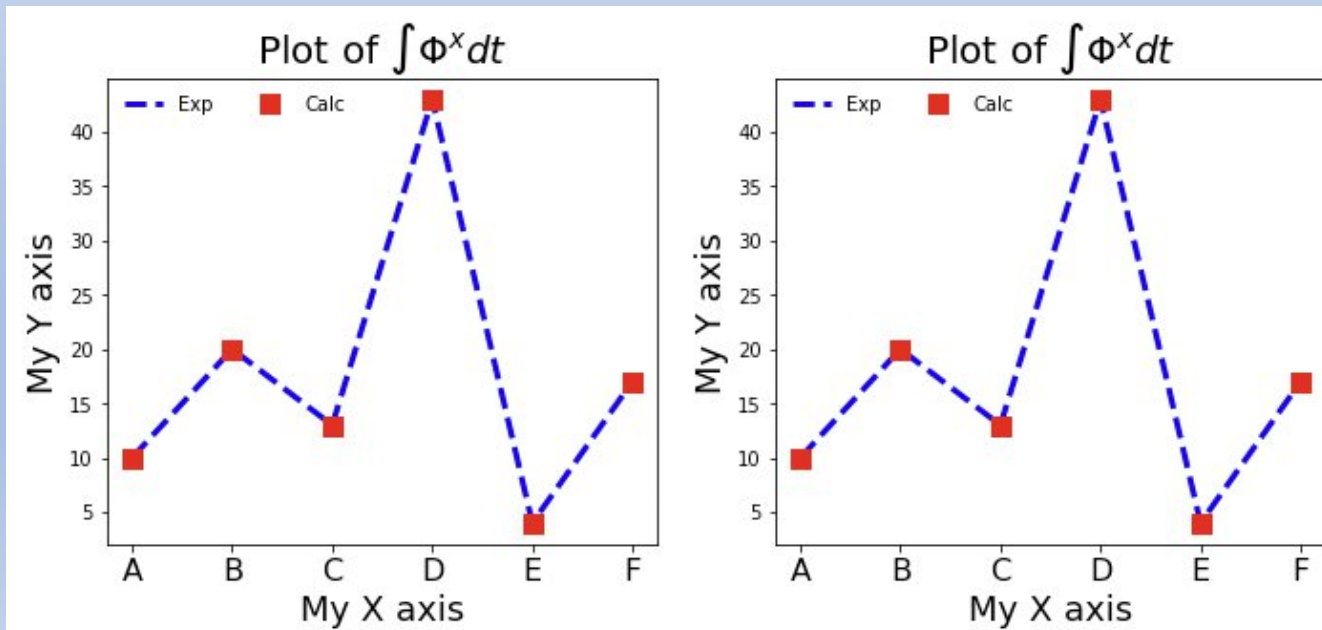
```
In [12]: plt.figure(figsize = (10,5)) # width, height

plt.subplot(1,2,1) # grid 1x2, plot #1
plt.plot(x, y, '--', color="blue", lw = 3, label = "Exp")
plt.plot(x, y, 's', color="red", ms = 10, label = "Calc")
plt.title(r'Plot of $\int \Phi^x dt$', fontsize = 20)
plt.xlabel('My X axis', fontsize=18)
plt.ylabel('My Y axis', fontsize=18)
plt.xticks(x, ['A', 'B', 'C', 'D', 'E', 'F'], fontsize = 16)
plt.legend(frameon = False, ncol = 2, loc = "upper left")

plt.subplot(1,2,2) # grid 1x2, plot #2
plt.plot(x, y, '--', color="blue", lw = 3, label = "Exp")
plt.plot(x, y, 's', color="red", ms = 10, label = "Calc")
plt.title(r'Plot of $\int \Phi^x dt$', fontsize = 20)
plt.xlabel('My X axis', fontsize=18)
plt.ylabel('My Y axis', fontsize=18)
plt.xticks(x, ['A', 'B', 'C', 'D', 'E', 'F'], fontsize = 16)
plt.legend(frameon = False, ncol = 2, loc = "upper left")

plt.tight_layout(pad=2)
```

`plt.subplot(rows, columns, plot #)`

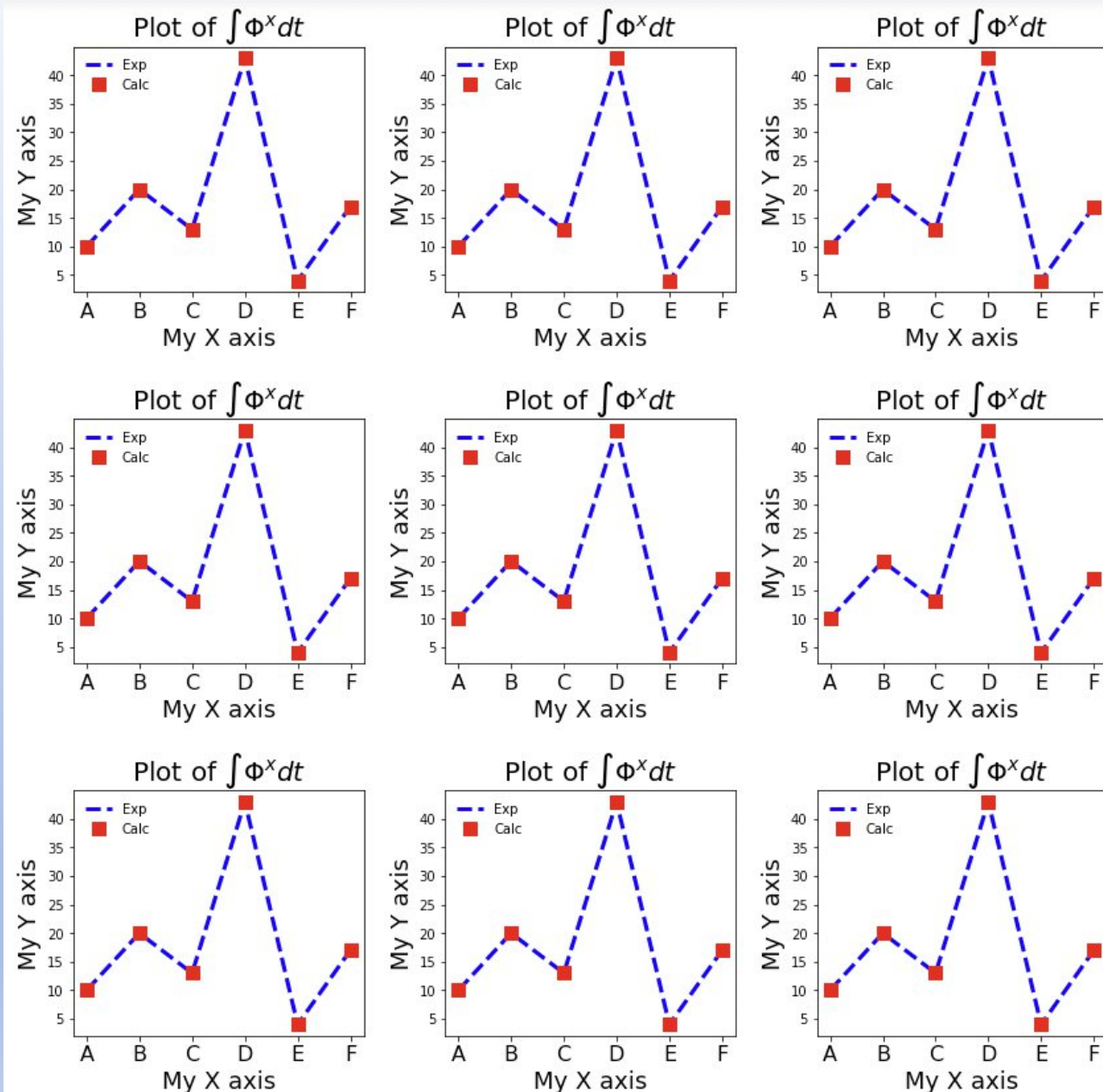


Grid subplots

```
In [13]: plt.figure(figsize = (12,12)) # width, height

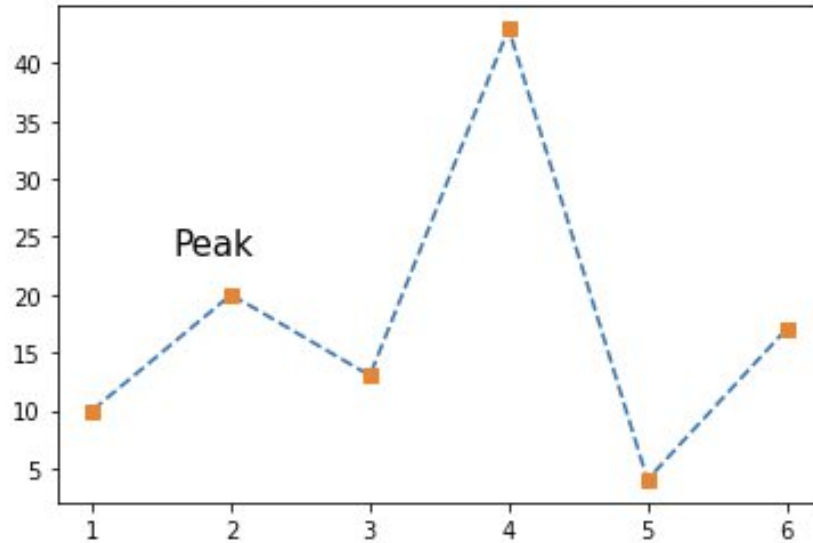
for i in range(9):
    plt.subplot(3, 3, i+1) # grid 3 x 3, plot #i+1
    plt.plot(x, y, '--', color="blue", lw = 3, label = "Exp")
    plt.plot(x, y, 's', color="red", ms = 10, label = "Calc")
    plt.title(r'Plot of $\int \Phi^x dt$', fontsize = 20)
    plt.xlabel('My X axis', fontsize=18)
    plt.ylabel('My Y axis', fontsize=18)
    plt.xticks(x, ['A', 'B', 'C', 'D', 'E', 'F'], fontsize = 16)
    plt.legend(frameon = False, loc = "upper left")

plt.tight_layout(pad=2)
```



Annotate plots

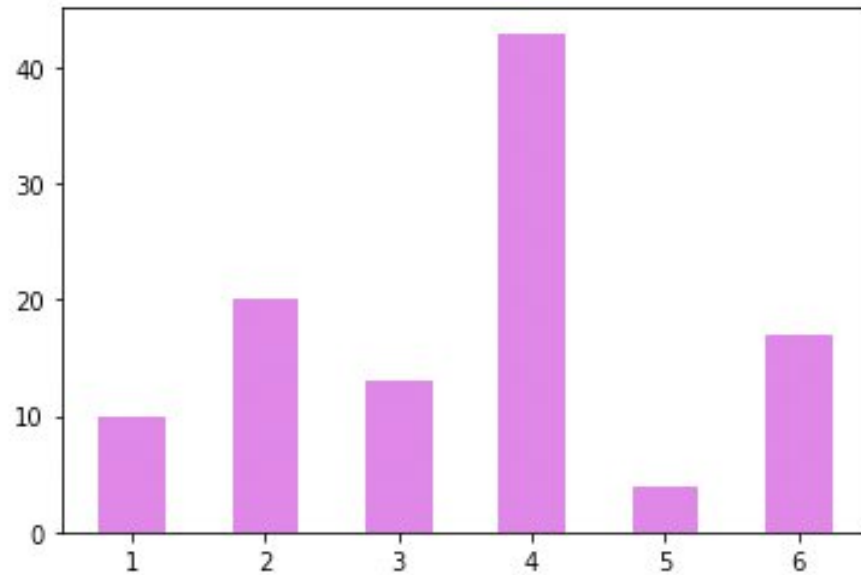
```
In [14]: plt.plot(x, y, '--')  
plt.plot(x, y, 's')  
plt.annotate('Peak', xy = (0.15, 0.5), xycoords=("axes fraction", "axes fraction"), size = 15);
```



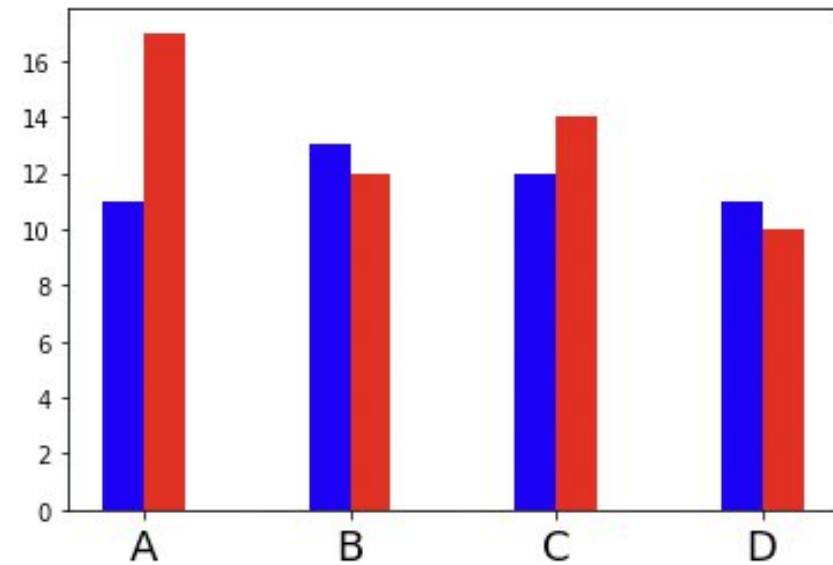
Note: You can also use the x,y coordinates related to the data itself to place your annotation. Just remove the `xycoords` parameter, e.g.: `plt.annotate('Peak', xy = (2, 25), size = 15)`

Bar plots

```
In [15]: plt.bar(x, y, width=0.5, color = 'violet');
```

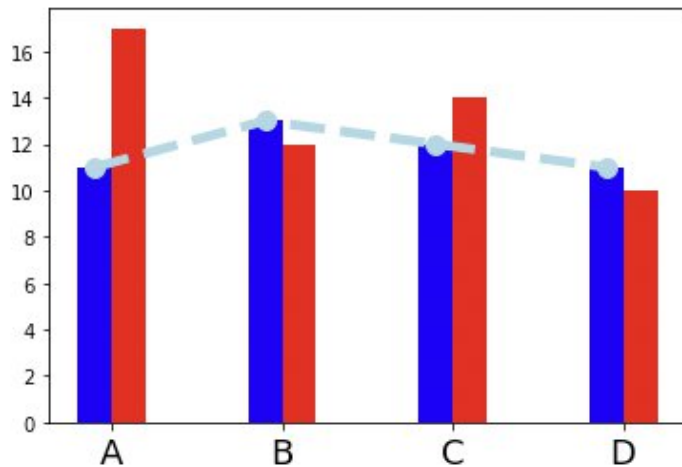


```
In [16]: x1 = [0.9, 1.9, 2.9, 3.9]
y1 = [11,13,12,11]
x2 = [1.1, 2.1, 3.1, 4.1]
y2 = [17,12,14,10]
plt.bar(x1, y1, width=0.2, color = 'blue')
plt.bar(x2, y2, width=0.2, color = 'red')
plt.xticks([1,2,3,4], ['A', 'B', 'C', 'D'], fontsize=18);
```



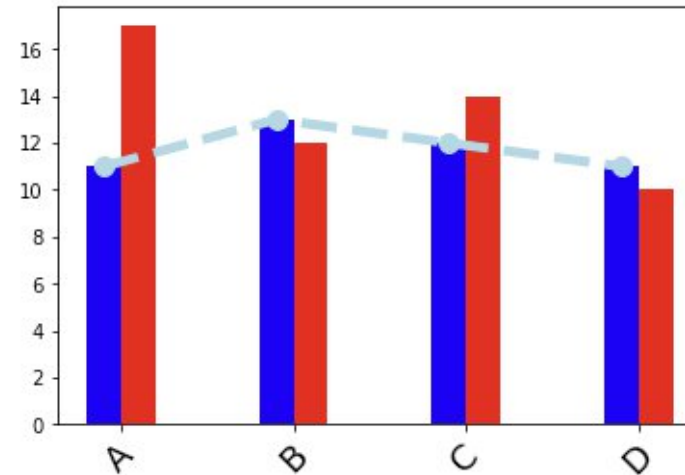
Bar + bar + line

```
In [17]: plt.bar(x1, y1, width=0.2, color = 'blue')
plt.bar(x2, y2, width=0.2, color = 'red')
plt.plot(x1,y1,'--o', lw=5, ms=10, color='lightblue')
plt.xticks([1,2,3,4], ['A', 'B', 'C', 'D'], fontsize=18);
```



Rotating the tick symbols

```
In [4]: plt.bar(x1, y1, width=0.2, color = 'blue')
plt.bar(x2, y2, width=0.2, color = 'red')
plt.plot(x1,y1,'--o', lw=5, ms=10, color='lightblue')
plt.xticks([1,2,3,4], ['A', 'B', 'C', 'D'], rotation=45, fontsize=18);
```



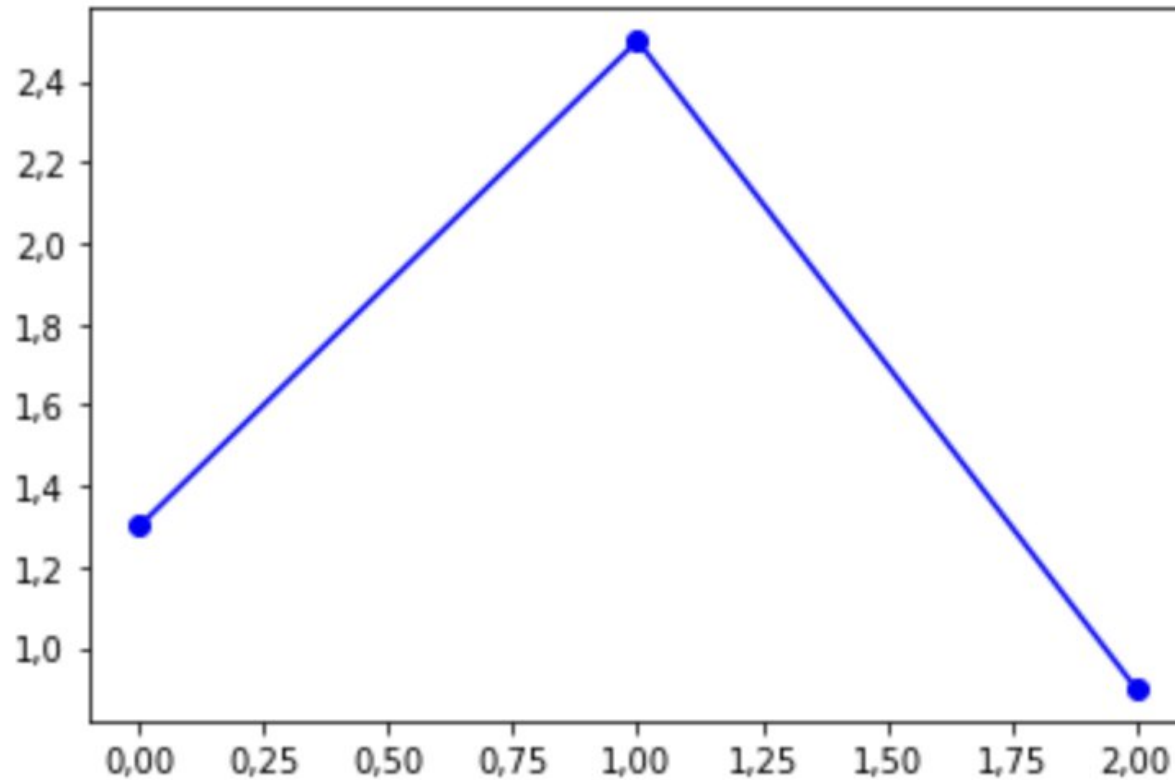
Tip 1: To fine tune the scale of the y axis to the range e.g. [8-17], use `plt.ylim(8,17)`

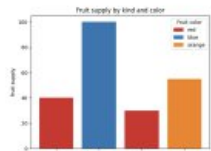
Tip 2: To show the X, Y or both axes in logarithmic scales, you can use e.g. `plt.semilogx(x,y)`, `plt.semilogy(x,y)` and `plt.loglog(x,y)`, respectively.

Using comma as decimal separator

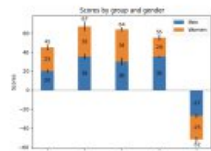
```
import matplotlib.pyplot as plt
import locale
locale.setlocale(locale.LC_NUMERIC, ('de_DE', 'UTF-8'))
plt.rcParams.update({'axes.formatter.use_locale' : True})

plt.plot([1.3, 2.5, 0.9], '-bo');
```

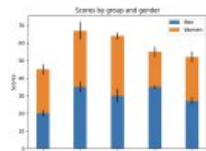




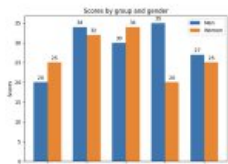
Bar color demo



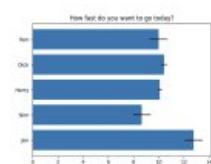
Bar Label Demo



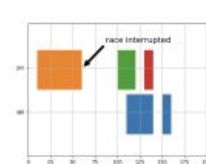
Stacked bar chart



Grouped bar chart with labels



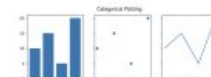
Horizontal bar chart



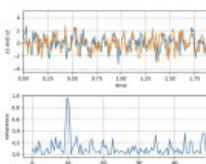
Broken Barh



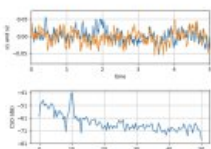
CapStyle



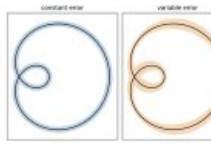
Plotting categorical variables



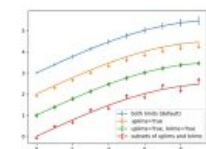
Plotting the coherence of two signals



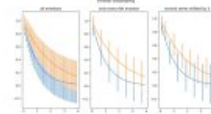
CSD Demo



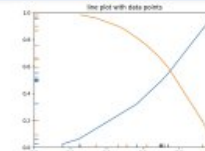
Curve with error band



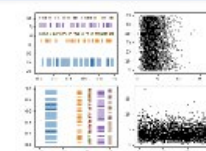
Errorbar limit selection



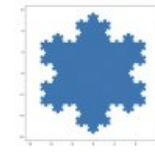
Errorbar subsampling



EventCollection Demo



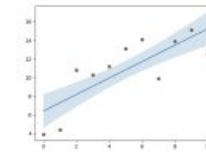
Eventplot Demo



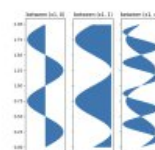
Filled polygon



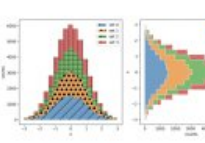
Fill Between and Alpha



Filling the area between lines



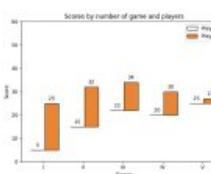
Fill Betweenx Demo



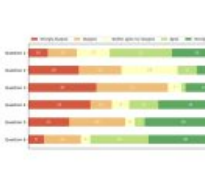
Hatch-filled histograms



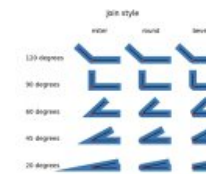
Bar chart with gradients



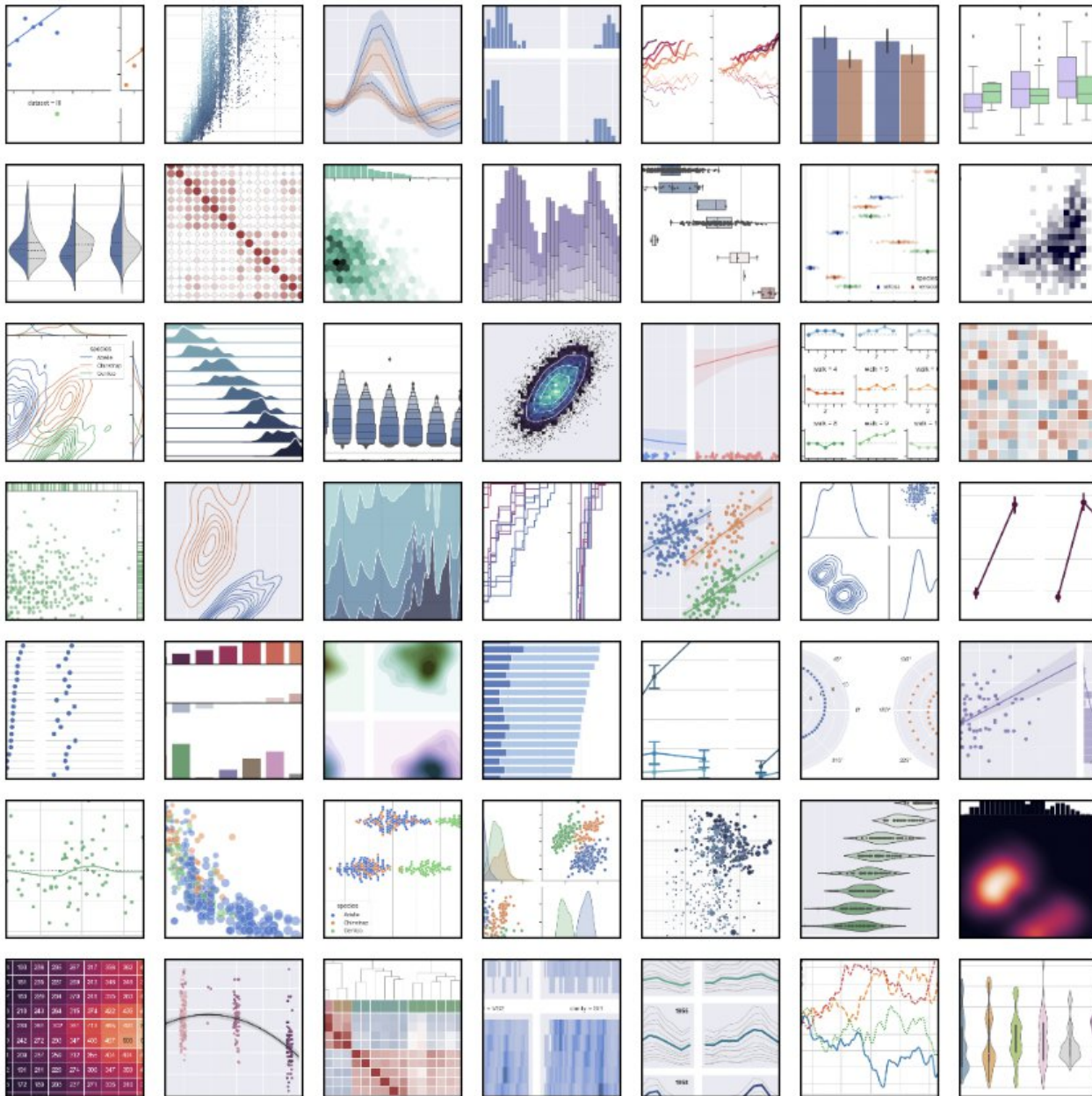
Hat graph



Discrete distribution as horizontal bar chart



JoinStyle



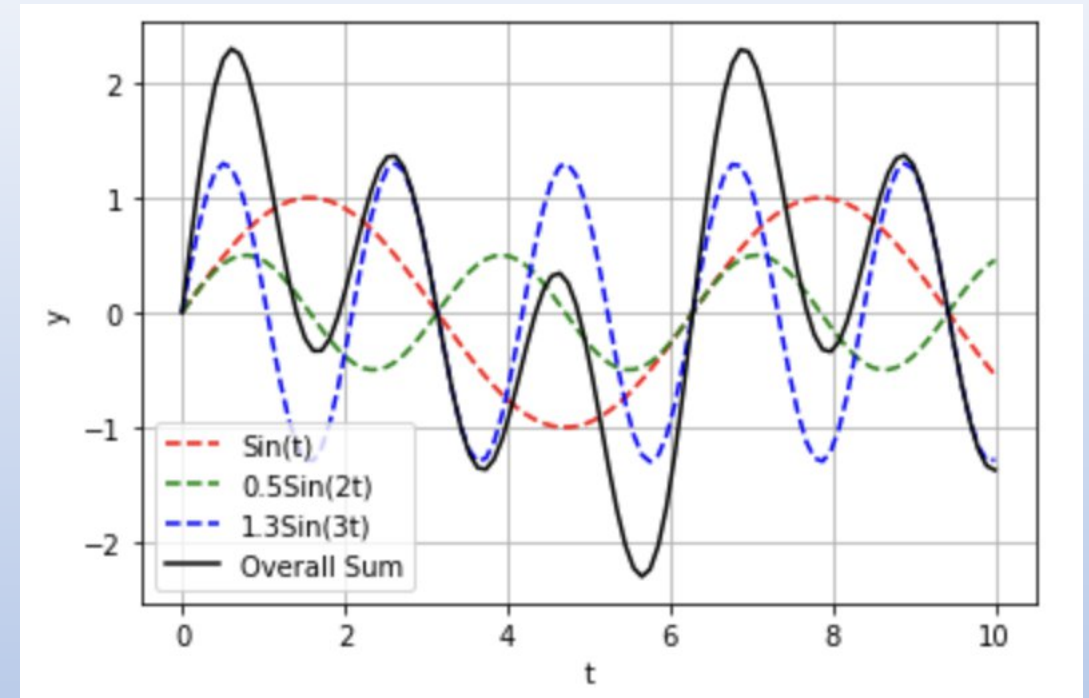
- Pre-configured nice plots (based on matplotlib)
- Little code needed

Another alternative: **plotly**

Exercises

- 1) In the same graph area, make plots of $y = \sin(t)$ with red dashed line, $y = \sin(2t)$ with green dashed line, $y = \sin(3t)$ with blue dashed line and the sum of the previous y values as a black continuous line. t must be in the interval $[0,10]$. Add legend and grid lines.

```
In [2]: t = np.linspace(0,10,100)
y1 = np.sin(t)
y2 = 0.5*np.sin(2*t)
y3 = 1.3*np.sin(3*t)
y4 = y1 + y2 + y3
plt.plot(t, y1, 'r--', label = "Sin(t)")
plt.plot(t, y2, 'g--', label = "0.5Sin(2t)")
plt.plot(t, y3, 'b--', label = "1.3Sin(3t)")
plt.plot(t, y4, 'k-', label = "Overall Sum")
plt.grid()
plt.legend()
plt.xlabel('t')
plt.ylabel('y');
```



Note: The black curve could be seen as a signal (e.g. audio, visible spectrum, etc), which is composed of the three individual dashed lines. Starting from the black curve and without explicitly knowing the dashed lines, one can find (or recover) the underlining curves (more precisely, the underlining frequencies and amplitudes) via the so called **Fourier transform** (available in numpy and scipy).

Exercises

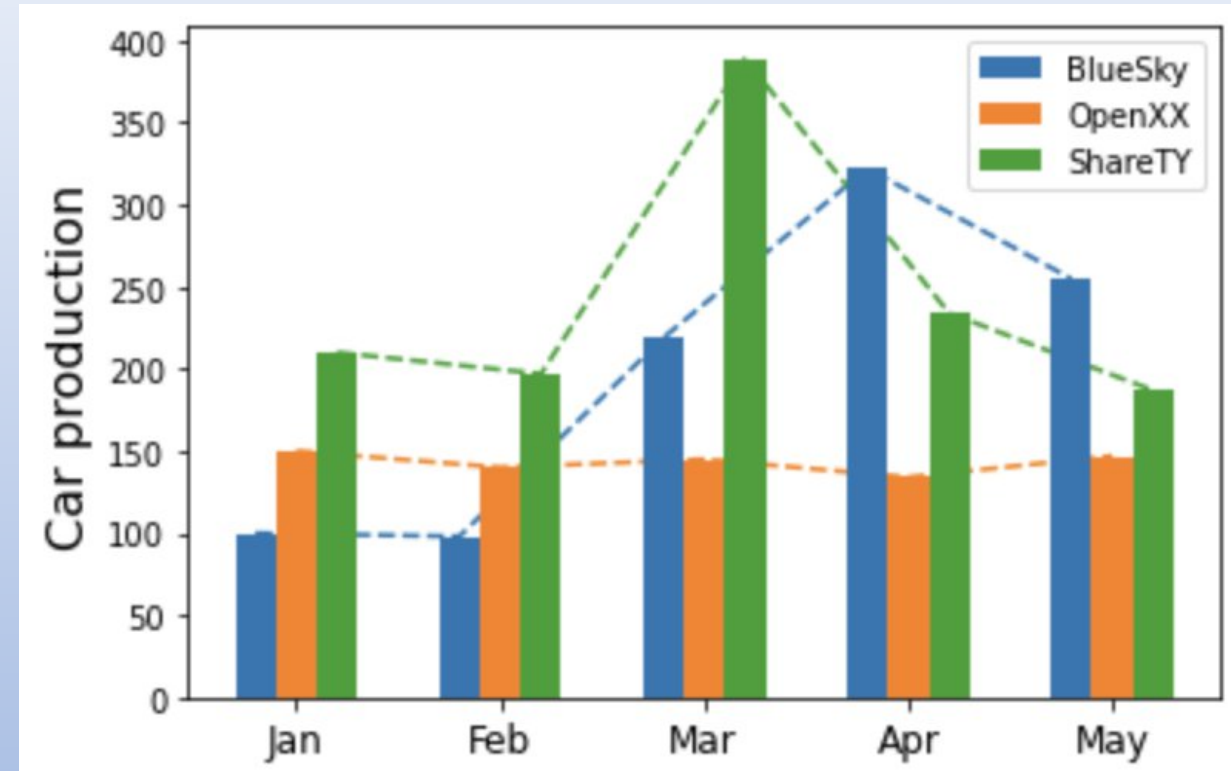
2) From January to May, the monthly car production (in car units) of the companies BlueSky, OpenXX and ShareTY was [100,98,219,322,255], [150,140,145,134,147] and [210,197,389,234,187], respectively. Show this information on one single bar plot with legend. In addition, add to the same plot continuous lines with markers (behind the bars!) with markers to better highlight the trends of each company.

```
In [3]: bluesky = [100,98,219,322,255]
openxx = [150,140,145,134,147]
sharety = [210,197,389,234,187]
x = np.array([1,2,3,4,5])
months = ["Jan", "Feb", "Mar", "Apr", "May"]

plt.bar(x - 0.2, bluesky, width = 0.2, label='BlueSky')
plt.bar(x, openxx, width = 0.2, label='OpenXX')
plt.bar(x + 0.2, sharety, width = 0.2, label='ShareTY')

plt.plot(x - 0.2, bluesky, '--', zorder=0)
plt.plot(x, openxx, '--', zorder=0)
plt.plot(x + 0.2, sharety, '--', zorder=0)

plt.xticks(x, months, fontsize=12)
plt.ylabel('Car production', fontsize=16)
plt.legend();
```



Notes:

- Automatic color generation for bars and lines
- Precise especification of the bar centers
- Parameter `zorder`



Exercises

- 3) Use the function $y = \exp(-x/10)$ evaluated in the range $x = 1$ to 1000 and plot this function (with continuous lines and markers) using: i) normal axes, ii) logarithmic X axis, iii) logarithmic Y axis and iv) logarithmic X and Y axes.

```
In [2]: x = np.arange(1,1000)
y = np.exp(-x/10)
plt.figure(figsize=(8,8))

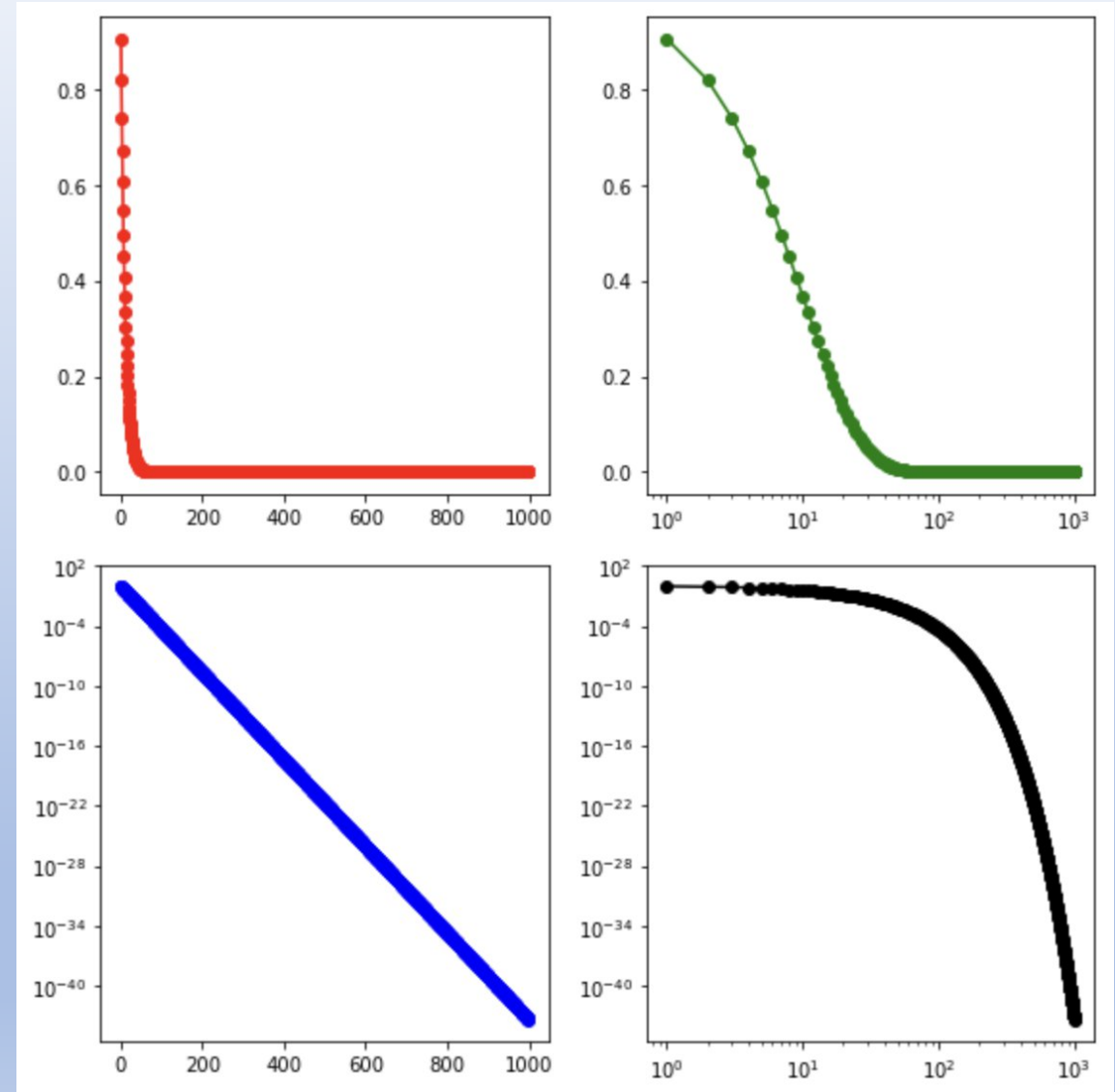
plt.subplot(2,2,1)
plt.plot(x, y, '-ro')

plt.subplot(2,2,2)
plt.semilogx(x, y, '-go')

plt.subplot(2,2,3)
plt.semilogy(x,y, '-bo')

plt.subplot(2,2,4)
plt.loglog(x,y, '-ko')

plt.tight_layout()
```



Note: The code above is **very small**. You can add MANY other visual elements, of course!

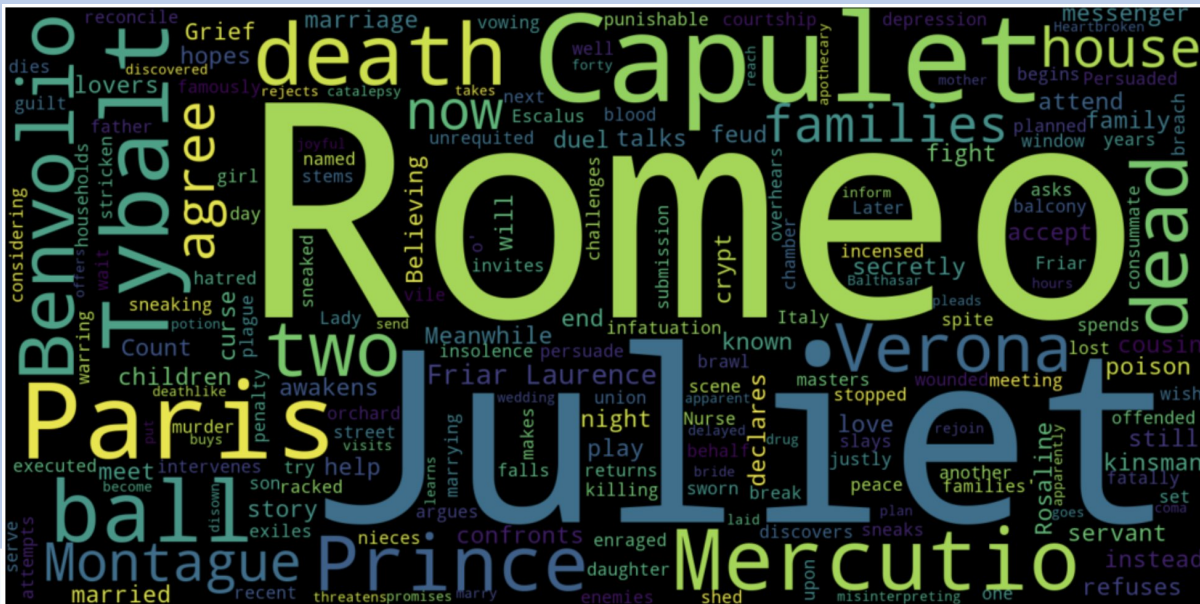


Exercises

4) **Challenge:** Install the package wordcloud (search it in anaconda) and generate a word cloud with any text you like with more than 100 words (suggestion: the Synopsys of Romeo and Juliet entry of wikipedia) to generate a word cloud of the text. Tip: the text must be used as one single string. Use the module "WordCloud" of the package "wordcloud" and the imshow() method of matplotlib. Use triple quotes to define your string.

```
In [2]: text = """The play, set in Verona, Italy, begins with a street brawl between Montague and Capulet servants who, like th
```

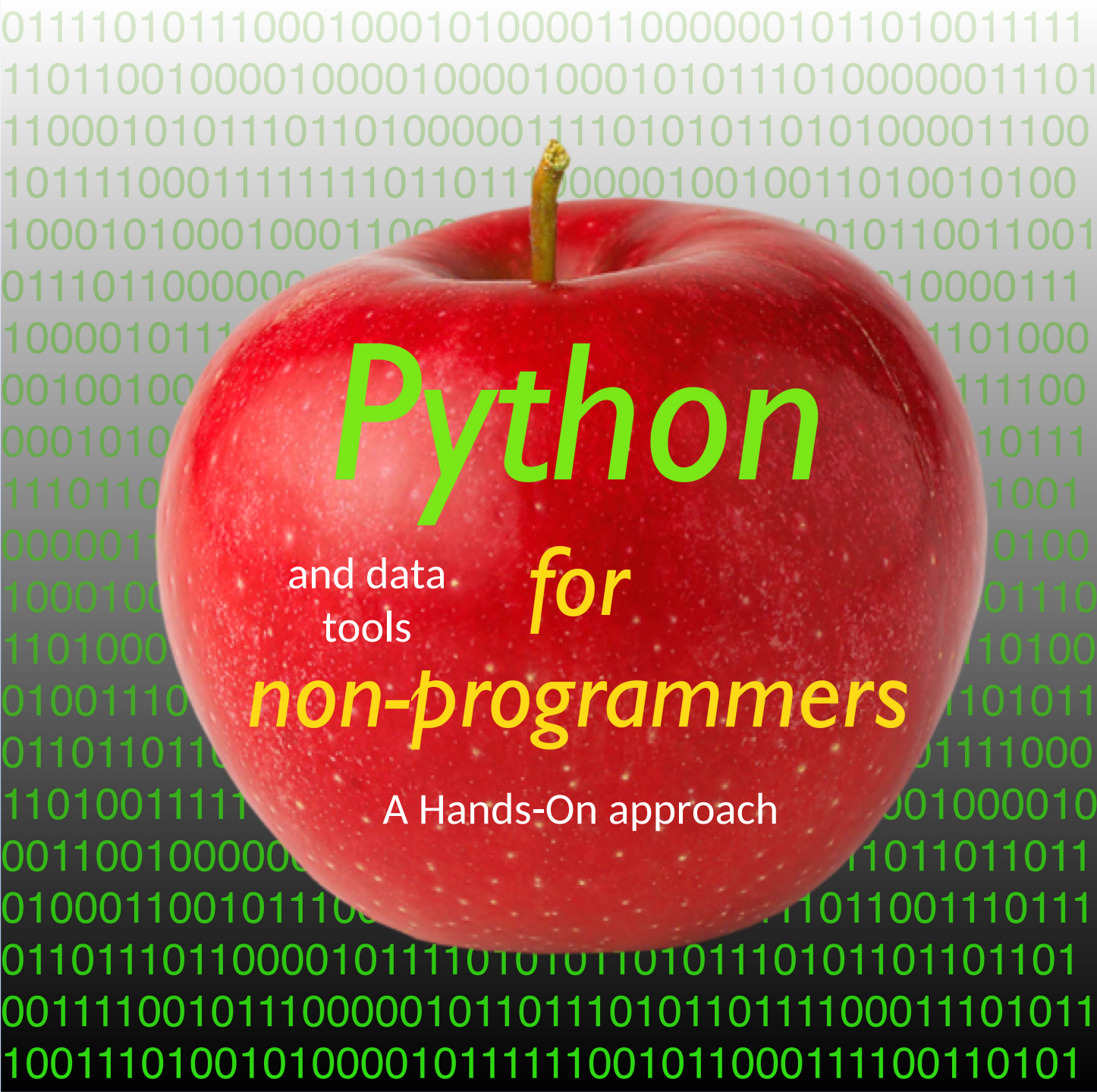
```
In [3]: from wordcloud import WordCloud
wordcloud = WordCloud(width = 1600, height = 800).generate(text)
plt.figure(figsize = (20,10))
plt.imshow(wordcloud, interpolation = 'bilinear')
plt.axis("off");
```



Tip: To save directly the word cloud as a .png file, use the code:

```
wordcloud = WordCloud(width = 1600, height = 800).generate(text)
wordcloud.to_file('figure.png')
```





UNIVERSITÄT
BAYREUTH

Delivered by

Dr. Rodrigo Albuquerque

(AG Ruckdäschel)

&

AG Künneth

Part of the project
“digitale Methoden”
(bidt-digikollegs)



- Brief overview of Python
- Installation and graphical interfaces
- Data types and operators
- Objects, modules and classes
- Strings
- Lists, sets and tuples
- Dictionary
- Arrays and matrices
- Loops (for, while)
- If statements
- Functions, pandas
- Files
- Plotting graphics

Concluding Remarks

- We saw a **tiny fraction** of what Python can do!
- There are more Python libraries than one can count! (and they keep increasing!)
Check: Pandas, Scipy, OpenCV, Scikit-Learn, Tensorflow, NLTK
- Keep practicing to make your daily work **more efficient** and **fun!**
- Take advantage of the **HUGE** Python documentation/resources available!
- Visit (enroll on) our courses at UBT:
 - "Machine Learning for Beginners" (Lecturer: Dr. Rodrigo Albuquerque)
 - "Machine Learning in Materials Science" (Lecturer: Prof. Christopher Künneth)